

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO ASSIGNMENT 3

Giảng viên hướng dẫn : Trần Đình Quế
Họ và tên sinh viên : Hoàng Văn Dũng
Mã sinh viên : B23DCCN199
Lớp : 05

Mục Lục

I. Hệ thống chẩn đoán bệnh tiểu đường.....	6
1. DATASET VÀ MÔ TẢ BÀI TOÁN	6
1.1. Tên và nguồn Dataset	6
1.2. Định nghĩa bài toán	6
1.3. Mô tả các thuộc tính	6
2. PHÂN TÍCH DỮ LIỆU VÀ CHẤT LƯỢNG DỮ LIỆU.....	7
2.1. Kích thước và missing values.....	7
2.2. Phân bố lớp mục tiêu	9
2.3. Phân tích tương quan.....	9
3. THIẾT KẾ THÍ NGHIỆM VÀ TIỀN XỬ LÝ	11
3.1. Chia Train / Validation / Test.....	11
3.2. Chuẩn hóa đặc trưng.....	12
4. BA MÔ HÌNH MACHINE LEARNING TRUYỀN THỐNG.....	13
4.1. Logistic Regression	13
4.2. K-Nearest Neighbors (KNN).....	13
4.3. Random Forest.....	14
5. MÔ HÌNH DEEP LEARNING 5 LỚP.....	15
5.1. Kiến trúc mạng	15
5.2. Forward Propagation	16
5.3. Hàm kích hoạt ReLU	16
5.4. Loss Function: Binary Cross-Entropy	17
5.5. Backpropagation, Gradient Descent và Optimizer	17
5.6. TensorDataset, DataLoader và Mini-batch	17
5.7. Quá trình huấn luyện và dấu hiệu Overfitting.....	18
6. ĐÁNH GIÁ VÀ SO SÁNH KẾT QUẢ	22
6.1. Các chỉ số đánh giá.....	22

6.2. Kết quả cuối trên Test Set	23
6.3. Confusion Matrix.....	25
6.4. ROC Curve và ROC-AUC	28
6.5. Chi phí tính toán	29
7. KHẢ NĂNG DIỄN GIẢI VÀ REPRESENTATION LEARNING.....	31
7.1. Random Forest Feature Importance	31
7.2. Learned Representation của Deep Learning	32
8. Kết luận.....	34
II. Hệ thống dự đoán giá nhà	35
1. DATASET VÀ MÔ TẢ BÀI TOÁN	35
1.1. Tên và nguồn Dataset	35
1.2. Phát biểu bài toán	36
2. PHÂN TÍCH DỮ LIỆU VÀ CHẤT LƯỢNG DỮ LIỆU.....	37
2.1. Cấu trúc dữ liệu và các thuộc tính	37
2.2. Missing values và dữ liệu thiếu	37
2.3. Thống kê mô tả của target Price	38
3. KHÁM PHÁ DỮ LIỆU (EDA).....	39
3.1. Phân bố giá nhà	39
3.2. Quan hệ giữa Area và Price.....	39
3.3. Outlier và tương quan.....	40
4. THIẾT KẾ THÍ NGHIỆM VÀ BIỂU DIỄN DỮ LIỆU.....	42
4.1. Xác định input và target	42
4.2. Chia Train / Validation / Test.....	42
4.3. Preprocessing và Representation.....	43
5. METRICS CHO BÀI TOÁN HỒI QUY	44
5.1. Mean Absolute Error (MAE).....	44
5.2. Mean Squared Error và RMSE.....	44
5.3. Hệ số R^2	44
6. BA MÔ HÌNH MACHINE LEARNING TRUYỀN THỐNG.....	44
6.1. Linear Regression.....	44

6.2. Random Forest Regressor.....	44
6.3. Gradient Boosting Regressor.....	44
6.4. Kết quả Validation của 3 ML	45
7. MÔ HÌNH DEEP LEARNING 5 LỚP.....	45
7.1. Kiến trúc mạng	45
7.2. Forward Propagation và ReLU.....	46
7.3. Tensor shapes và mini-batch	46
7.4. Số lượng parameters	47
7.5. Loss function và Optimizer	47
7.6. Backpropagation và chu trình huấn luyện.....	47
8. HÀNH VI HUẤN LUYỆN VÀ GENERALIZATION	48
8.1. Training / Validation Loss.....	48
8.2. Training / Validation RMSE	48
9. SO SÁNH VALIDATION GIỮA ML VÀ DL.....	50
10. Visualization metrics	50
11. PHÂN TÍCH SAI SỐ	53
11.1. Actual vs Predicted.....	53
11.2. Residual Analysis	56
12. KHẢ NĂNG DIỄN GIẢI VÀ FEATURE IMPORTANCE	57
13. REPRESENTATION LEARNING CỦA DEEP LEARNING.....	59
13.1. Hidden Representation H4	59
13.2. PCA visualization của learned representation.....	59
15. KẾT LUẬN	60
16. LƯU MÔ HÌNH VÀ TÁI SỬ DỤNG.....	60
III. Hệ thống dự đoán hành vi khách hàng	61
1. DATASET VÀ MÔ TẢ BÀI TOÁN	61
1.1. Tên và nguồn Dataset	61
1.2. Định nghĩa bài toán	61
2. PHÂN TÍCH DỮ LIỆU VÀ LÀM SẠCH	62
2.1. Cấu trúc dữ liệu và chất lượng dữ liệu	62

2.2. Quy trình làm sạch dữ liệu	63
2.3. Phân bố lớp mục tiêu	63
2.4. Phân bố độ dài review	64
2.5. Rating và recommendation intention	65
3. THIẾT KẾ THÍ NGHIỆM VÀ BIỂU DIỄN VĂN BẢN.....	67
3.1. Chia Train / Validation / Test.....	67
3.2. Biểu diễn văn bản bằng TF-IDF	68
4. CÁC METRIC ĐÁNH GIÁ	69
5. BA MÔ HÌNH MACHINE LEARNING TRUYỀN THỐNG.....	70
5.1. Logistic Regression	70
5.2. Multinomial Naive Bayes.....	70
5.3. Linear SVM	70
5.4. So sánh ba mô hình ML trên Validation Set	70
6. MÔ HÌNH DEEP LEARNING 5 LỚP.....	71
6.1. Kiến trúc mạng	71
6.2. Forward Propagation và learned representation	72
6.3. Loss Function	72
6.4. Backpropagation và Optimizer.....	72
6.5. Tensor, DataLoader và cấu hình huấn luyện.....	73
6.6. Training / Validation Curves	75
7. ĐÁNH GIÁ VÀ SO SÁNH ML VỚI DL	77
7.1. So sánh trên Validation Set	77
7.2. Final Retraining và Test Metrics	80
7.3. Classification Report và Confusion Matrix.....	80
7.4. ROC Curve và ROC-AUC	83
8. KHẢ NĂNG DIỄN GIẢI VÀ REPRESENTATION LEARNING.....	84
8.1. Diễn giải TF-IDF terms bằng Logistic Regression	84
8.2. Learned Representation H4	86
9. KẾT LUẬN	87

Link Github chứa code:

https://github.com/HoangDung05/assignment_03.git

I. Hệ thống chẩn đoán bệnh tiểu đường

1. DATASET VÀ MÔ TẢ BÀI TOÁN

1.1. Tên và nguồn Dataset

Dataset sử dụng trong thí nghiệm là **Pima Indians Diabetes Dataset**. File dữ liệu được sử dụng trong notebook có tên **diabetes_dataset.csv**. Đây là bộ dữ liệu dạng bảng, trong đó mỗi dòng đại diện cho một bệnh nhân và mỗi cột mô tả một đặc trưng y khoa hoặc nhân khẩu học liên quan đến bài toán dự đoán bệnh tiểu đường.

Tên Dataset: Pima Indians Diabetes Dataset

Nguồn Dataset: <https://www.kaggle.com/datasets/jamaltariqcheema/pima-indians-diabetes-dataset>

1.2. Định nghĩa bài toán

Mục tiêu của hệ thống là dự đoán một bệnh nhân có mắc bệnh tiểu đường hay không dựa trên tám đặc trưng đầu vào. Đây là bài toán phân loại nhị phân (**binary classification**), trong đó Outcome = 0 biểu thị không mắc bệnh và Outcome = 1 biểu thị mắc bệnh.

$$X = [x_1, x_2, \dots, x_8]$$

$$y \in \{0, 1\}$$

Để đánh giá cả phương pháp Machine Learning truyền thống và Deep Learning, thí nghiệm huấn luyện bốn mô hình gồm:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Random Forest
- Multi-Layer Perceptron (MLP) có 5 lớp Linear học được tham số.

Mục tiêu của Assignment 3 không chỉ là tìm mô hình có độ chính xác cao nhất, mà còn phải phân tích kiến trúc, tensor shape, loss function, optimizer, training loop, khả năng generalization, chi phí huấn luyện và learned representation của mạng Neural Network.

1.3. Mô tả các thuộc tính

Thuộc tính	Ý nghĩa trong dữ liệu
Pregnancies	Số lần mang thai.

Glucose	Mức glucose trong máu.
BloodPressure	Huyết áp.
SkinThickness	Độ dày nếp gấp da.
Insulin	Mức insulin.
BMI	Chỉ số khối cơ thể (Body Mass Index).
DiabetesPedigreeFunction	Chỉ số phản ánh yếu tố tiền sử/gia đình liên quan đến tiểu đường.
Age	Tuổi của bệnh nhân.
Outcome	Nhãn mục tiêu: 0 = No Diabetes, 1 = Diabetes.

2. PHÂN TÍCH DỮ LIỆU VÀ CHẤT LƯỢNG DỮ LIỆU

2.1. Kích thước và missing values

Dataset có 768 quan sát và 9 cột, bao gồm 8 cột đầu vào và 1 cột mục tiêu Outcome.

```
candidate_paths = [
    "diabetes_dataset.csv",
    "/mnt/data/diabetes_dataset.csv"
]

DATA_PATH = next((p for p in candidate_paths if os.path.exists(p)), None)
if DATA_PATH is None:
    raise FileNotFoundError("Place diabetes_dataset.csv in the same folder as this notebook.")

df = pd.read_csv(DATA_PATH)
print("Path :", DATA_PATH)
print("Shape:", df.shape)
df.head()
```

Path : diabetes_dataset.csv

Shape: (768, 9)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

Kết quả kiểm tra cho thấy không có missing value ở bất kỳ cột nào trong file dữ liệu đang sử dụng.

```
zero_check_columns = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
print("Zero-value check:")
for col in zero_check_columns:
    if col in df.columns:
        print(f"{col:15s}: {(df[col] == 0).sum()}")
```

```
Zero-value check:
Glucose      : 0
BloodPressure : 0
SkinThickness : 0
Insulin      : 0
BMI          : 0
```

Ngoài missing values, các cột Glucose, BloodPressure, SkinThickness, Insulin và BMI cũng được kiểm tra giá trị 0. Trong file hiện tại, các cột này không còn giá trị 0 bất thường. Vì vậy notebook không cần thực hiện bước thay thế hoặc imputation bổ sung cho các biến này.

```
print("Columns:", df.columns.tolist())
print("\nMissing values:")
print(df.isnull().sum())
print("\nTarget distribution:")
print(df["Outcome"].value_counts().sort_index())
print("\nTarget ratio:")
print(df["Outcome"].value_counts(normalize=True).sort_index().round(4))
```

```
Columns: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']
```

```
Missing values:
Pregnancies      0
Glucose          0
BloodPressure     0
SkinThickness     0
Insulin          0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

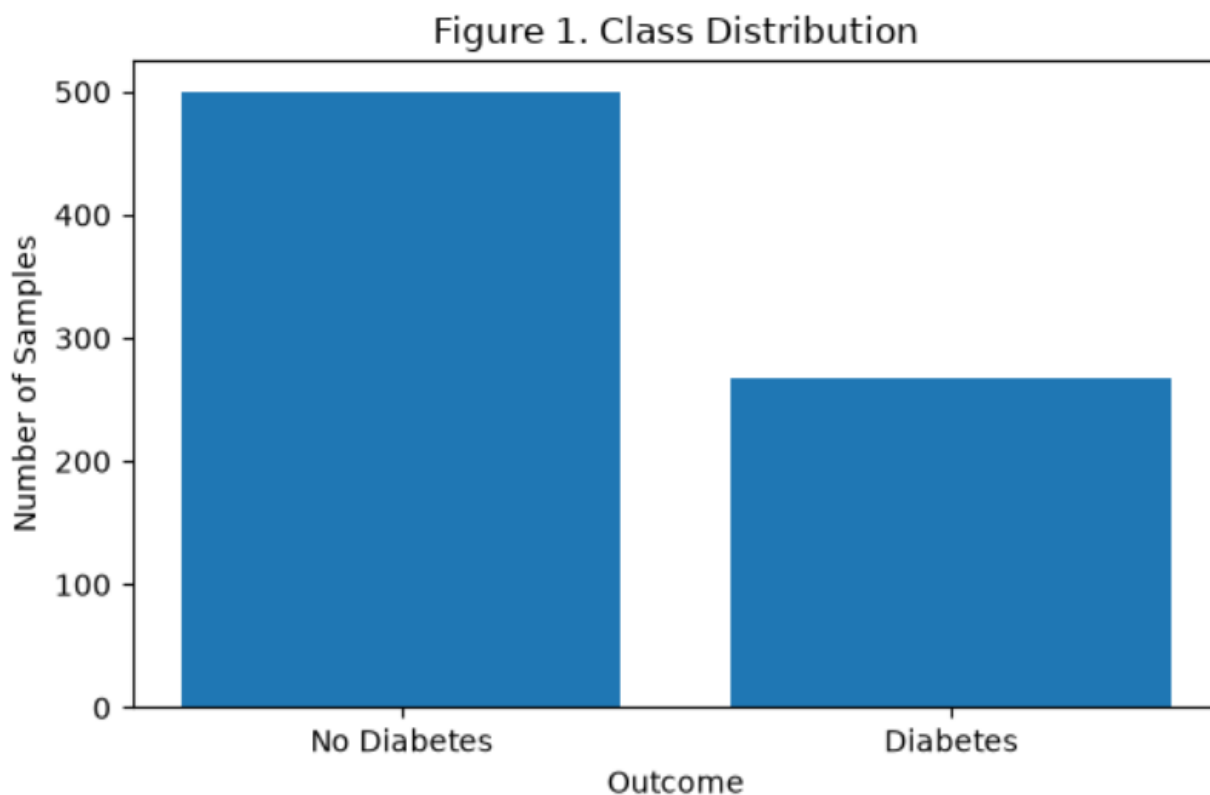
```
Target distribution:
Outcome
0      500
1      268
Name: count, dtype: int64
```

```
Target ratio:
Outcome
0      0.651
1      0.349
Name: proportion, dtype: float64
```


2.2. Phân bố lớp mục tiêu

Trong 768 quan sát, lớp No Diabetes (Outcome = 0) có 500 mẫu, chiếm khoảng 65,1%; lớp Diabetes (Outcome = 1) có 268 mẫu, chiếm khoảng 34,9%. Dữ liệu vì vậy có mức mất cân bằng vừa phải giữa hai lớp.

```
class_counts = df["Outcome"].value_counts().sort_index()
plt.figure(figsize=(6,4))
plt.bar(["No Diabetes", "Diabetes"], class_counts.values)
plt.title("Figure 1. Class Distribution")
plt.xlabel("Outcome")
plt.ylabel("Number of Samples")
plt.tight_layout()
plt.show()
```



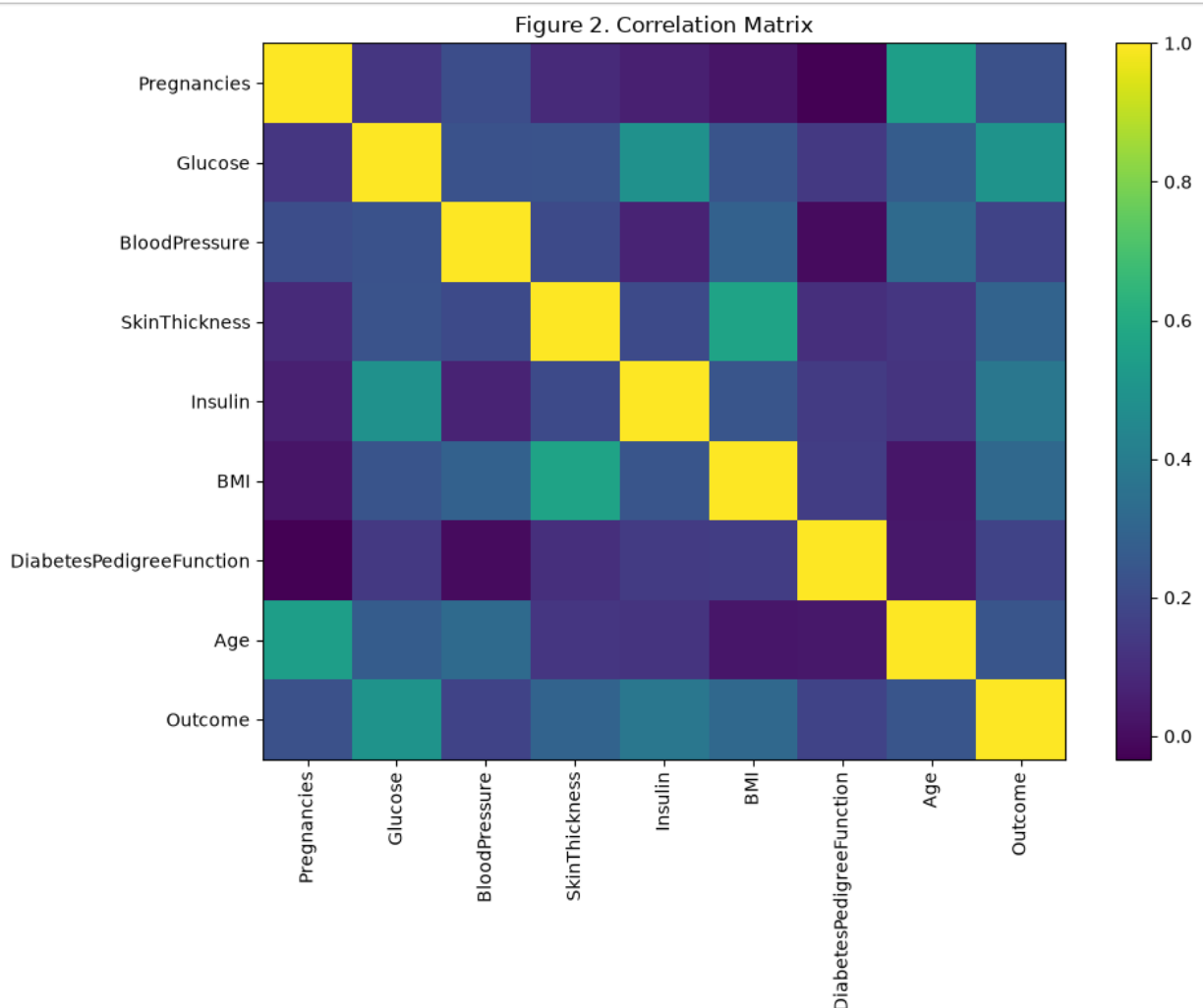
Do lớp mục tiêu không hoàn toàn cân bằng, Accuracy không đủ để mô tả toàn bộ hiệu năng của mô hình. Báo cáo cần đồng thời sử dụng Precision, Recall, F1-score, ROC-AUC và Confusion Matrix. Đặc biệt, trong bài toán phát hiện bệnh, Recall của lớp Diabetes cần được chú ý vì False Negative tương ứng với trường hợp bệnh nhân thực sự mắc bệnh nhưng hệ thống dự đoán là không mắc bệnh.

2.3. Phân tích tương quan

Ma trận tương quan được sử dụng để quan sát mức liên hệ tuyến tính giữa các biến số. Phân tích này giúp phát hiện các đặc trưng có xu hướng liên hệ với Outcome hoặc liên hệ

mạnh với nhau. Tuy nhiên, correlation chỉ phản ánh mối liên hệ thống kê và không được diễn giải thành quan hệ nhân quả y khoa.

```
corr = df.corr(numeric_only=True)
plt.figure(figsize=(10,8))
plt.imshow(corr, aspect="auto")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.colorbar()
plt.title("Figure 2. Correlation Matrix")
plt.tight_layout()
plt.show()
```



Giải thích: Ma trận tương quan này cho biết mức độ liên hệ tuyến tính giữa từng cặp biến, với giá trị thường nằm trong khoảng -1 đến 1 :

- Gần 1 : tương quan dương mạnh.

- Gần **0**: tương quan tuyến tính yếu.
- Gần **-1**: tương quan âm mạnh.

Trong hình này, phần lớn các biến có tương quan **thấp đến trung bình**, tức là không có nhiều cặp biến phụ thuộc tuyến tính quá mạnh vào nhau.

Với biến mục tiêu **Outcome**, các biến có tương quan dương tương đối nổi bật hơn là:

- **Glucose**: có tương quan với Outcome mạnh nhất trong các biến đầu vào.
- Insulin
- BMI
- SkinThickness
- Age

Điều này cho thấy khi các giá trị này tăng, xác suất thuộc lớp Outcome = 1 có xu hướng tăng theo, nhưng mức tương quan chưa quá mạnh nên **không thể chỉ dựa vào một biến để dự đoán Diabetes**.

3. THIẾT KẾ THÍ NGHIỆM VÀ TIỀN XỬ LÝ

3.1. Chia Train / Validation / Test

Dữ liệu được chia thành ba tập theo tỷ lệ xấp xỉ **70% Train, 15% Validation và 15% Test**. Cụ thể, Train có 537 mẫu, Validation có 115 mẫu và Test có 116 mẫu. Phương pháp stratified split được sử dụng để duy trì tỷ lệ hai lớp tương đối ổn định giữa các tập.

Train được dùng để học tham số mô hình. Validation được dùng để lựa chọn các quyết định phát triển mô hình như giá trị k của KNN và epoch tốt nhất của mạng Deep Learning. Test được giữ độc lập và chỉ dùng ở bước đánh giá cuối nhằm phản ánh khả năng generalization trên dữ liệu chưa nhìn thấy.

```

X = df.drop(columns=["Outcome"])
y = df["Outcome"]

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=RANDOM_STATE,
    stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=RANDOM_STATE,
    stratify=y_temp
)

print("Train      :", X_train.shape, y_train.shape)
print("Validation:", X_val.shape, y_val.shape)
print("Test       :", X_test.shape, y_test.shape)

```

```

Train      : (537, 8) (537,)
Validation: (115, 8) (115,)
Test       : (116, 8) (116,)

```

3.2. Chuẩn hóa đặc trưng

Các biến đầu vào có thang đo khác nhau nên Logistic Regression, KNN và MLP sử dụng StandardScaler. Mean và standard deviation chỉ được fit trên Training Set, sau đó cùng phép biến đổi được áp dụng cho Validation và Test nhằm tránh data leakage.

$$x' = \frac{x - \mu}{\sigma}$$

Trong đó μ và σ được học từ Training Set.

Sau chuẩn hóa, mean của từng feature trên Training Set xấp xỉ 0 và standard deviation xấp xỉ 1. Random Forest không bắt buộc phải dùng scaling vì cơ chế chia node của Decision Tree không dựa trực tiếp vào khoảng cách giữa các điểm.

```

scaler_dev = StandardScaler()
X_train_scaled = scaler_dev.fit_transform(X_train)
X_val_scaled = scaler_dev.transform(X_val)
X_test_scaled_dev = scaler_dev.transform(X_test)

print("Training means after scaling:", np.round(X_train_scaled.mean(axis=0), 4))
print("Training stds after scaling :", np.round(X_train_scaled.std(axis=0), 4))

Training means after scaling: [ 0. -0.  0.  0.  0.  0.  0. -0.]
Training stds after scaling : [1. 1. 1. 1. 1. 1. 1. 1.]

```

4. BA MÔ HÌNH MACHINE LEARNING TRUYỀN THỐNG

4.1. Logistic Regression

Logistic Regression được sử dụng làm baseline tuyến tính cho bài toán phân loại nhị phân. Mô hình học một tổ hợp tuyến tính của các feature đã chuẩn hóa và biến đổi đầu ra thành xác suất thuộc lớp Diabetes. Ưu điểm của mô hình là đơn giản, huấn luyện nhanh và có mức độ diễn giải tương đối cao.

```

logistic_dev = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
logistic_dev.fit(X_train_scaled, y_train)

lr_val_pred = logistic_dev.predict(X_val_scaled)
lr_val_prob = logistic_dev.predict_proba(X_val_scaled)[: ,1]
classification_metrics("Logistic Regression", y_val, lr_val_pred, lr_val_prob)

{'Model': 'Logistic Regression',
 'Accuracy': 0.6956521739130435,
 'Precision': 0.5641025641025641,
 'Recall': 0.55,
 'F1': 0.5569620253164557,
 'ROC_AUC': 0.8046666666666665}

```

4.2. K-Nearest Neighbors (KNN)

KNN là mô hình dựa trên khoảng cách. Với một bệnh nhân mới, mô hình tìm k điểm dữ liệu gần nhất trong không gian feature và quyết định lớp dựa trên các lân cận này. Vì khoảng cách rất nhạy với scale của feature, KNN sử dụng dữ liệu đã chuẩn hóa.

Notebook thử các giá trị $k = 3, 5, 7, 9$ và 11 trên Validation Set. Trong các cấu hình này, $k = 7$ đạt F1-score validation cao nhất (0,7907) và được chọn cho mô hình cuối cùng.

```

k_candidates = [3, 5, 7, 9, 11]
knn_val_rows = []

for k in k_candidates:
    m = KNeighborsClassifier(n_neighbors=k)
    m.fit(X_train_scaled, y_train)
    pred = m.predict(X_val_scaled)
    prob = m.predict_proba(X_val_scaled)[: ,1]
    row = classification_metrics(f"KNN (k={k})", y_val, pred, prob)
    row["k"] = k
    knn_val_rows.append(row)

knn_validation = pd.DataFrame(knn_val_rows).sort_values(["F1", "ROC_AUC"], ascending=False)
knn_validation.round(4)

```

	Model	Accuracy	Precision	Recall	F1	ROC_AUC	k
2	KNN (k=7)	0.8435	0.7391	0.850	0.7907	0.9005	7
1	KNN (k=5)	0.8087	0.7045	0.775	0.7381	0.8840	5
4	KNN (k=11)	0.8000	0.6809	0.800	0.7356	0.8777	11
3	KNN (k=9)	0.7913	0.6818	0.750	0.7143	0.8753	9
0	KNN (k=3)	0.7739	0.6591	0.725	0.6905	0.8568	3

4.3. Random Forest

Random Forest là một ensemble gồm nhiều Decision Tree. Các cây học các quy tắc phân chia phi tuyến từ dữ liệu và kết quả cuối được tổng hợp từ nhiều cây. Mô hình có khả năng biểu diễn quan hệ phi tuyến tốt hơn Logistic Regression và thường hoạt động mạnh trên dữ liệu dạng bảng.

Trong notebook, Random Forest được huấn luyện với 300 cây và `class_weight="balanced"`. Việc dùng balanced class weights giúp giảm ảnh hưởng của sự mất cân bằng giữa lớp Diabetes và No Diabetes.

```

rf_dev = RandomForestClassifier(
    n_estimators=300,
    random_state=RANDOM_STATE,
    class_weight="balanced"
)
rf_dev.fit(X_train, y_train)

rf_val_pred = rf_dev.predict(X_val)
rf_val_prob = rf_dev.predict_proba(X_val)[: ,1]
classification_metrics("Random Forest", y_val, rf_val_pred, rf_val_prob)

{'Model': 'Random Forest',
 'Accuracy': 0.8173913043478261,
 'Precision': 0.7209302325581395,
 'Recall': 0.775,
 'F1': 0.7469879518072289,
 'ROC_AUC': 0.9278333333333333}

```

5. MÔ HÌNH DEEP LEARNING 5 LỚP

5.1. Kiến trúc mạng

Mô hình Deep Learning được xây dựng bằng PyTorch dưới dạng Multi-Layer Perceptron gồm 5 lớp Linear học tham số. Kiến trúc mạng là $8 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$. Bốn lớp đầu sử dụng ReLU để tạo tính phi tuyến; lớp cuối trả về một logit cho bài toán binary classification.

$8 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$

Giai đoạn	Tensor shape với batch B	Vai trò
Input	$B \times 8$	8 feature y khoa ban đầu
Layer 1	$B \times 64$	Learned representation H_1
Layer 2	$B \times 32$	Learned representation H_2
Layer 3	$B \times 16$	Learned representation H_3
Layer 4	$B \times 8$	Learned representation H_4
Output	$B \times 1$	Logit dự đoán Diabetes

Tổng số tham số học được của mô hình là 3.329. Số tham số được tính từ tổng weight và bias của năm lớp Linear.

$\#parameters = input_dim \times output_dim + output_dim$

GHI CHÚ CHÈN HÌNH

- Chụp Cell In[17]: kiến trúc FiveLayerMLP và Total trainable parameters = 3329.
- Caption gợi ý: Hình 6. Kiến trúc và số lượng tham số của mạng MLP 5 lớp.
- Ảnh kiến thức: không cần chụp slide nếu đã có sơ đồ kiến trúc và công thức trong báo cáo.

```

: dev_mlp = FiveLayerMLP(input_dim=X_train.shape[1]).to(DEVICE)
  print(dev_mlp)

total_params = sum(p.numel() for p in dev_mlp.parameters() if p.requires_grad)
print("Total trainable parameters:", total_params)

param_rows = []
for name, p in dev_mlp.named_parameters():
    param_rows.append({"Parameter": name, "Shape": str(tuple(p.shape)), "Count": p.numel()})
pd.DataFrame(param_rows)

FiveLayerMLP(
  (fc1): Linear(in_features=8, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (fc3): Linear(in_features=32, out_features=16, bias=True)
  (fc4): Linear(in_features=16, out_features=8, bias=True)
  (fc5): Linear(in_features=8, out_features=1, bias=True)
  (relu): ReLU()
)
Total trainable parameters: 3329

```

Hình 1: Kiến trúc và số lượng tham số của mạng MLP 5 lớp.

5.2. Forward Propagation

Trong forward propagation, mỗi hidden layer nhận representation từ layer trước, áp dụng phép biến đổi tuyến tính rồi ReLU. Do đó mạng không dự đoán trực tiếp từ 8 feature ban đầu mà học một chuỗi các representation trung gian trước khi tạo output cuối cùng.

$$\mathbf{H}_1 = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)$$

$$\mathbf{H}_2 = \text{ReLU}(\mathbf{H}_1\mathbf{W}_2 + \mathbf{b}_2)$$

$$\mathbf{H}_3 = \text{ReLU}(\mathbf{H}_2\mathbf{W}_3 + \mathbf{b}_3)$$

$$\mathbf{H}_4 = \text{ReLU}(\mathbf{H}_3\mathbf{W}_4 + \mathbf{b}_4)$$

$$\mathbf{Z}_5 = \mathbf{H}_4\mathbf{W}_5 + \mathbf{b}_5$$

$$\hat{y} = \sigma(\mathbf{Z}_5) = \frac{1}{1 + e^{-\mathbf{Z}_5}}$$

$$\hat{y} \geq 0.5 \Rightarrow \text{Diabetes}, \quad \hat{y} < 0.5 \Rightarrow \text{No Diabetes}$$

5.3. Hàm kích hoạt ReLU

ReLU (Rectified Linear Unit) được sử dụng sau bốn lớp Linear đầu tiên. ReLU giữ nguyên giá trị dương và đưa giá trị âm về 0. Vai trò quan trọng của ReLU là tạo tính phi

tuyến cho mạng. Nếu chỉ xếp chồng nhiều Linear layer mà không có activation phi tuyến, toàn bộ mạng vẫn có thể rút gọn thành một phép biến đổi tuyến tính.

$$\text{ReLU}(z) = \max(0, z)$$

hoặc:

$$\text{ReLU}(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$

5.4. Loss Function: Binary Cross-Entropy

Vì đây là bài toán phân loại nhị phân, hàm mất mát Binary Cross-Entropy được sử dụng để đo mức độ sai giữa xác suất dự đoán và nhãn thật. Notebook dùng

BCEWithLogitsLoss của PyTorch, kết hợp Sigmoid và Binary Cross-Entropy theo cách ổn định hơn về mặt số học.

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

5.5. Backpropagation, Gradient Descent và Optimizer

Sau khi tính loss, backpropagation sử dụng chain rule để tính gradient của loss theo từng weight và bias. Optimizer sau đó cập nhật tham số theo hướng làm giảm loss. Notebook sử dụng Adam với learning rate 0,001.

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial H_2} \frac{\partial H_2}{\partial H_1} \frac{\partial H_1}{\partial W_1}$$

$$\theta_{t+1} = \theta_t - \eta \nabla \theta L$$

Một training step có thể tóm tắt theo chuỗi:

Forward → Loss → Backward → Update

Trong PyTorch, trình tự chính là optimizer.zero_grad(), model(X_batch), criterion(...), loss.backward() và optimizer.step().

5.6. TensorDataset, DataLoader và Mini-batch

Dữ liệu sau chuẩn hóa được chuyển thành torch.Tensor kiểu float32. TensorDataset ghép input và target thành các sample; DataLoader tổ chức các sample thành mini-batch. Batch size được đặt bằng 32, nên một batch input có shape [32, 8] và target có shape [32, 1].

```
def make_tensor_dataset(X_array, y_series):
    X_tensor = torch.tensor(np.asarray(X_array), dtype=torch.float32)
    y_tensor = torch.tensor(np.asarray(y_series).reshape(-1,1), dtype=torch.float32)
    return TensorDataset(X_tensor, y_tensor)

BATCH_SIZE = 32
train_ds = make_tensor_dataset(X_train_scaled, y_train)
val_ds = make_tensor_dataset(X_val_scaled, y_val)

g = torch.Generator()
g.manual_seed(RANDOM_STATE)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, generator=g)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False)

xb, yb = next(iter(train_loader))
print("One input batch :", xb.shape)
print("One target batch:", yb.shape)

One input batch : torch.Size([32, 8])
One target batch: torch.Size([32, 1])
```

The Assignment 03 material distinguishes:

- **Dataset** : provides individual samples.
- **DataLoader** : organizes samples into mini-batches.

Mini-batches improve memory/computation efficiency and allow frequent parameter updates.

Hình 2. Kích thước tensor của một mini-batch

5.7. Quá trình huấn luyện và dấu hiệu Overfitting

Mạng được huấn luyện tối đa 150 epoch. Trong quá trình này, Training Loss liên tục giảm nhưng Validation Loss chỉ giảm trong giai đoạn đầu rồi tăng trở lại. Validation Loss thấp nhất đạt 0,4635 tại epoch 31. Sau đó Training Accuracy tiếp tục tăng tới 100% trong khi Validation Accuracy chỉ dao động quanh khoảng 77–80%.

Hiện tượng Training performance tiếp tục cải thiện nhưng Validation performance không cải thiện tương ứng là dấu hiệu rõ của overfitting. Vì vậy epoch 31 được chọn làm epoch tốt nhất cho mô hình cuối. Kết quả này minh họa đúng yêu cầu của Assignment 3: không thể chỉ nhìn Training Accuracy để đánh giá khả năng generalization.

```

LEARNING_RATE = 0.001
MAX_EPOCHS = 150
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(dev_mlp.parameters(), lr=LEARNING_RATE)

train_losses, val_losses = [], []
train_accs, val_accs = [], []
best_val_loss = float("inf")
best_epoch = 1
best_state = None

dev_start = time.perf_counter()

for epoch in range(1, MAX_EPOCHS + 1):
    dev_mlp.train()
    epoch_loss = 0.0
    train_y, train_pred = [], []

    for xb, yb in train_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)

        optimizer.zero_grad()
        logits = dev_mlp(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item() * xb.size(0)
        prob = torch.sigmoid(logits)
        pred = (prob >= 0.5).int()
        train_y.extend(yb.detach().cpu().numpy().ravel())
        train_pred.extend(pred.detach().cpu().numpy().ravel())

    train_loss = epoch_loss / len(train_loader.dataset)
    train_acc = accuracy_score(train_y, train_pred)
    val_result = evaluate_mlp(dev_mlp, val_loader, criterion, DEVICE)

    train_losses.append(train_loss)
    val_losses.append(val_result["loss"])
    train_accs.append(train_acc)
    val_accs.append(val_result["accuracy"])

```

```

if val_result["loss"] < best_val_loss:
    best_val_loss = val_result["loss"]
    best_epoch = epoch
    best_state = copy.deepcopy(dev_mlp.state_dict())

if epoch == 1 or epoch % 25 == 0:
    print(f"Epoch {epoch:3d}/{MAX_EPOCHS} | Train Loss {train_loss:.4f} | Val Loss {val_result['loss']:.4f}")

dev_training_time = time.perf_counter() - dev_start
dev_mlp.load_state_dict(best_state)
print("\nBest epoch:", best_epoch)
print("Best validation loss:", round(best_val_loss, 4))
print("Development training time (s):", round(dev_training_time, 4))

```

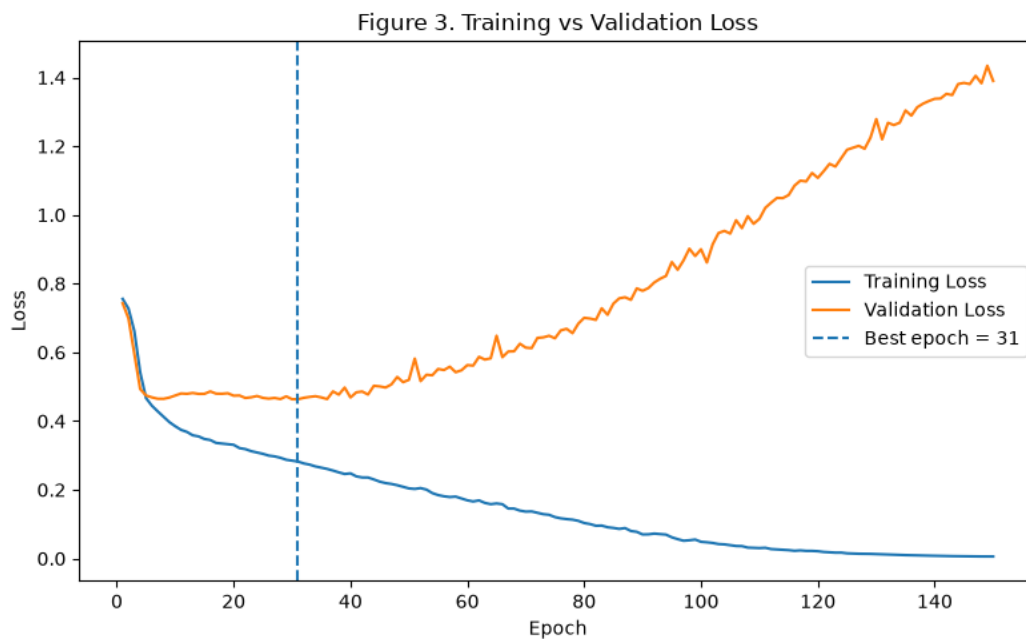
Epoch	1/150	Train Loss	0.7555	Val Loss	0.7427	Train Acc	0.3482	Val Acc	0.3478
Epoch	25/150	Train Loss	0.3047	Val Loss	0.4675	Train Acc	0.8790	Val Acc	0.8000
Epoch	50/150	Train Loss	0.2040	Val Loss	0.5200	Train Acc	0.9199	Val Acc	0.7826
Epoch	75/150	Train Loss	0.1205	Val Loss	0.6409	Train Acc	0.9628	Val Acc	0.7739
Epoch	100/150	Train Loss	0.0487	Val Loss	0.8999	Train Acc	0.9851	Val Acc	0.7739
Epoch	125/150	Train Loss	0.0151	Val Loss	1.1897	Train Acc	1.0000	Val Acc	0.7826
Epoch	150/150	Train Loss	0.0059	Val Loss	1.3899	Train Acc	1.0000	Val Acc	0.7826

Hình 3. Training log và Best Epoch của mô hình 5-Layer MLP.

```

epochs_axis = np.arange(1, MAX_EPOCHS + 1)
plt.figure(figsize=(8,5))
plt.plot(epochs_axis, train_losses, label="Training Loss")
plt.plot(epochs_axis, val_losses, label="Validation Loss")
plt.axvline(best_epoch, linestyle="--", label=f"Best epoch = {best_epoch}")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Figure 3. Training vs Validation Loss")
plt.legend()
plt.tight_layout()
plt.show()

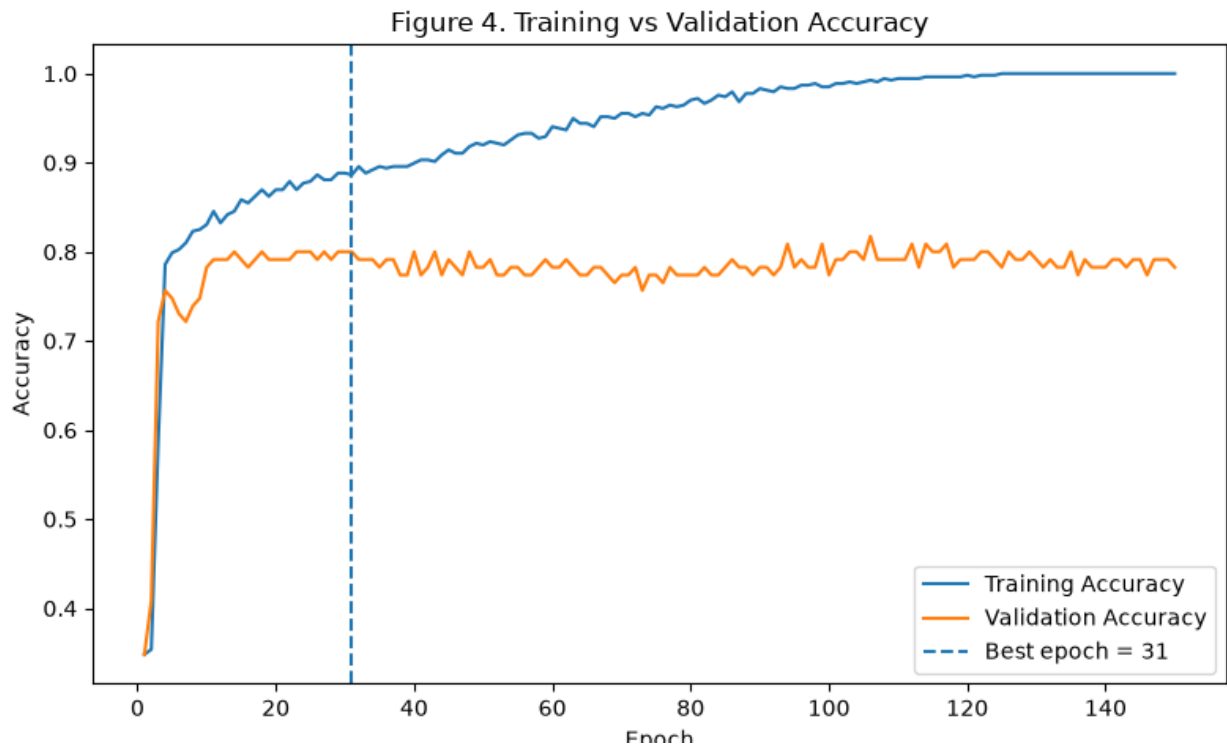
```



Hình 4. Training/Validation Loss

Giải thích: Biểu đồ này cho thấy Training Loss giảm liên tục trong suốt quá trình huấn luyện, trong khi Validation Loss đạt giá trị thấp nhất tại epoch 31 rồi tăng dần. Điều này cho thấy mô hình bắt đầu overfitting sau epoch 31. Vì vậy, model tại epoch 31 được lựa chọn làm mô hình tốt nhất để đánh giá trên tập Test.

```
plt.figure(figsize=(8,5))
plt.plot(epochs_axis, train_accs, label="Training Accuracy")
plt.plot(epochs_axis, val_accs, label="Validation Accuracy")
plt.axvline(best_epoch, linestyle="--", label=f"Best epoch = {best_epoch}")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Figure 4. Training vs Validation Accuracy")
plt.legend()
plt.tight_layout()
plt.show()
```



Hình 5. Training/Validation Accuracy

Giải thích: Biểu đồ này cho thấy Training Accuracy tăng dần và đạt gần 100%, trong khi Validation Accuracy chỉ duy trì quanh mức 78–80%. Khoảng cách ngày càng lớn giữa hai đường cho thấy mô hình có dấu hiệu overfitting. Epoch 31 được chọn là best epoch dựa trên Validation Loss thấp nhất.

6. ĐÁNH GIÁ VÀ SO SÁNH KẾT QUẢ

6.1. Các chỉ số đánh giá

Trong báo cáo, Diabetes được xem là positive class (1). Năm metric chính được sử dụng để so sánh bốn mô hình là Accuracy, Precision, Recall, F1-score và ROC-AUC.

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

=> Accuracy phản ánh tỷ lệ dự đoán đúng tổng thể

Precision

$$Precision = \frac{TP}{TP + FP}$$

=> Precision phản ánh độ tin cậy khi mô hình dự đoán Diabetes

Recall

$$Recall = \frac{TP}{TP + FN}$$

=> Recall cho biết mô hình phát hiện được bao nhiêu bệnh nhân thực sự mắc Diabetes.

F1-score

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

=> F1 cân bằng Precision và Recall. ROC-AUC đánh giá khả năng phân biệt hai lớp trên nhiều ngưỡng quyết định.

6.2. Kết quả cuối trên Test Set

```
comparison = pd.DataFrame(final_rows)
comparison["Fit_Time_s"] = comparison["Model"].map(fit_times)
comparison["Inference_Time_ms"] = comparison["Model"].map(inference_times)
comparison = comparison.sort_values("F1", ascending=False).reset_index(drop=True)
comparison.round(4)
```

	Model	Accuracy	Precision	Recall	F1	ROC_AUC	Fit_Time_s	Inference_Time_ms
0	5-Layer MLP	0.8966	0.9394	0.7561	0.8378	0.9389	1.6756	1.0065
1	Random Forest	0.8879	0.8684	0.8049	0.8354	0.9663	0.9373	56.4468
2	KNN (k=7)	0.8190	0.8125	0.6341	0.7123	0.8738	0.0031	4.8732
3	Logistic Regression	0.7845	0.7353	0.6098	0.6667	0.8995	0.0099	0.8472

Hình 6. Bảng so sánh kết quả metrics của các Model

Trên Test Set, MLP 5 lớp đạt Accuracy cao nhất 89,66%, Precision cao nhất 93,94% và F1-score cao nhất 83,78%. Random Forest đạt Recall cao nhất 80,49% và ROC-AUC cao nhất 96,63%. Kết quả cho thấy không có một mô hình duy nhất vượt trội ở mọi metric.

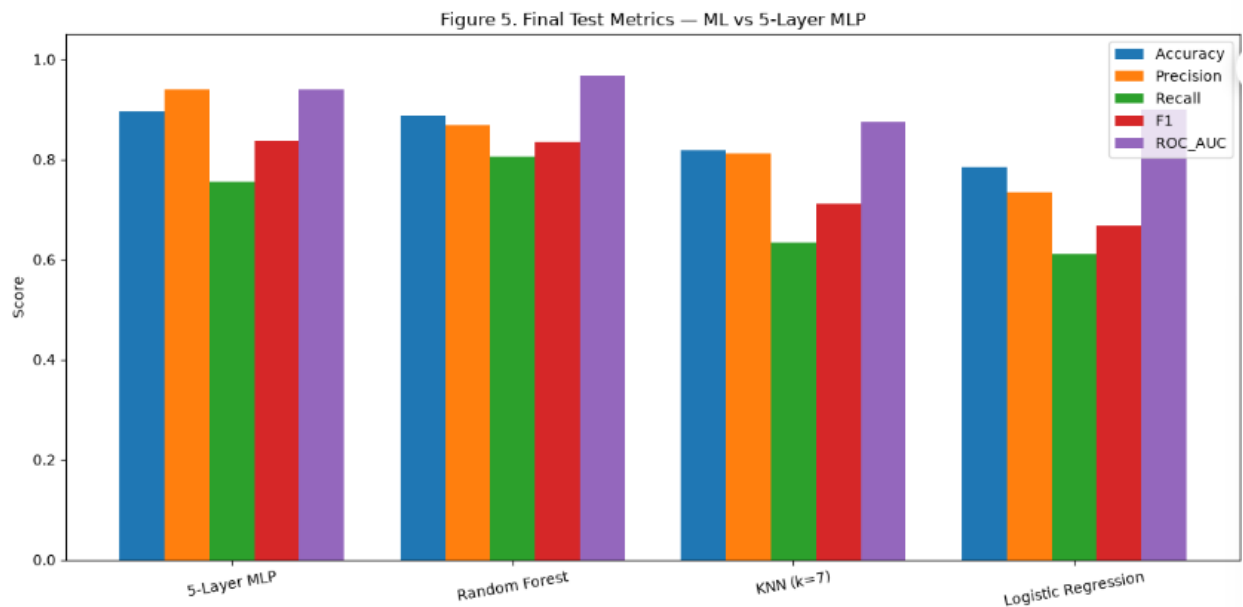
Trong bối cảnh phát hiện bệnh tiểu đường, Recall có ý nghĩa đặc biệt vì False Negative là trường hợp bệnh nhân thực sự mắc bệnh nhưng bị hệ thống dự đoán là không mắc. Vì vậy Random Forest dù có Accuracy thấp hơn MLP một chút nhưng có lợi thế rõ ở khả năng phát hiện positive class.

```
metric_cols = ["Accuracy", "Precision", "Recall", "F1", "ROC_AUC"]
plot_df = comparison.set_index("Model")[metric_cols]

x = np.arange(len(plot_df.index))
width = 0.16

plt.figure(figsize=(12,6))
for i, metric in enumerate(metric_cols):
    plt.bar(x + (i-2)*width, plot_df[metric].values, width=width, label=metric)

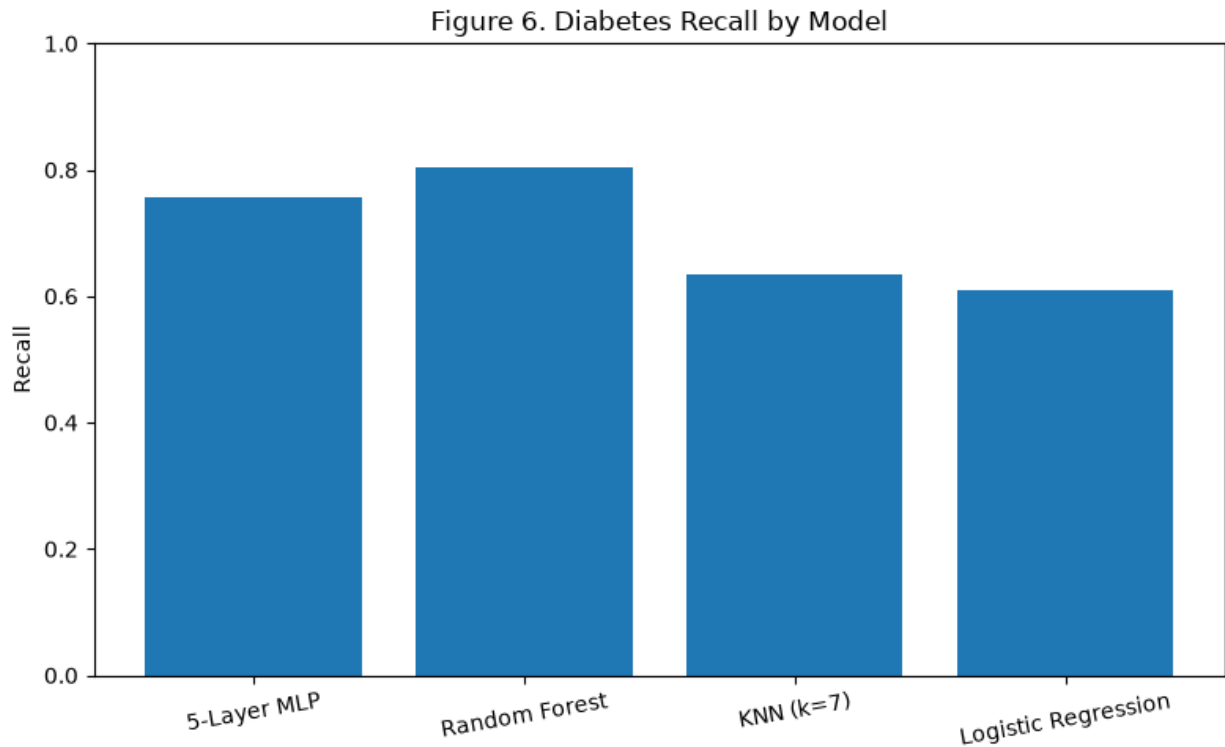
plt.xticks(x, plot_df.index, rotation=10)
plt.ylim(0, 1.05)
plt.ylabel("Score")
plt.title("Figure 5. Final Test Metrics - ML vs 5-Layer MLP")
plt.legend()
plt.tight_layout()
plt.show()
```



Hình 7. Biểu đồ nhóm Accuracy, Precision, Recall, F1, ROC-AUC.

Giải thích: Biểu đồ so sánh hiệu năng của 3 mô hình Machine Learning và mô hình Deep Learning 5 lớp trên tập Test. Kết quả cho thấy 5-Layer MLP có Accuracy, Precision và F1-score cao nhất, trong khi Random Forest đạt Recall và ROC-AUC cao nhất. KNN và Logistic Regression có hiệu năng thấp hơn hai mô hình trên. Điều này cho thấy không có một mô hình tốt nhất trên mọi tiêu chí; lựa chọn mô hình cần phụ thuộc vào metric được ưu tiên.


```
plt.figure(figsize=(8,5))
plt.bar(comparison["Model"], comparison["Recall"])
plt.ylim(0,1)
plt.ylabel("Recall")
plt.title("Figure 6. Diabetes Recall by Model")
plt.xticks(rotation=10)
plt.tight_layout()
plt.show()
```



Hình 8. Biểu đồ Recall

Giải thích: Biểu đồ cho thấy Random Forest đạt Recall cao nhất, do đó có khả năng phát hiện bệnh nhân thực sự mắc Diabetes tốt nhất trong bốn mô hình. 5-Layer MLP đứng thứ hai, trong khi KNN và Logistic Regression có Recall thấp hơn và bỏ sót nhiều ca Diabetes hơn.

6.3. Confusion Matrix

Confusion Matrix giúp phân tích bốn loại kết quả TP, TN, FP và FN thay vì chỉ nhìn Accuracy. Với bài toán này, FN là loại lỗi cần đặc biệt quan tâm vì tương ứng với việc bỏ sót bệnh nhân thực sự mắc Diabetes.

```
def plot_confusion(y_true, y_pred, model_name, figure_no):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5,4))
    plt.imshow(cm)
    plt.xticks([0,1], ["No Diabetes", "Diabetes"])
    plt.yticks([0,1], ["No Diabetes", "Diabetes"])
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(f"Figure {figure_no}. Confusion Matrix - {model_name}")
    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i,j], ha="center", va="center")
    plt.colorbar()
    plt.tight_layout()
    plt.show()
    return cm

figure_no = 7
confusion_tables = {}
for model_name in ["Logistic Regression", knn_name, "Random Forest", "5-Layer MLP"]:
    confusion_tables[model_name] = plot_confusion(y_test, final_predictions[model_name], model_name, figure_no)
    figure_no += 1
```

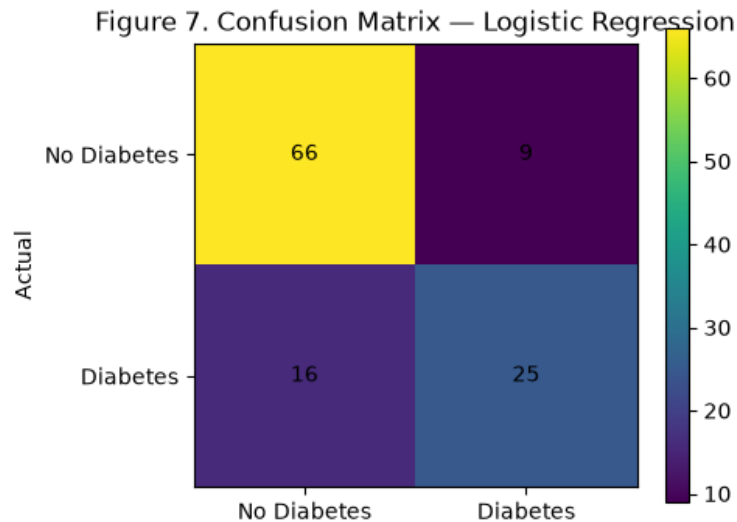


Figure 8. Confusion Matrix — KNN (k=7)

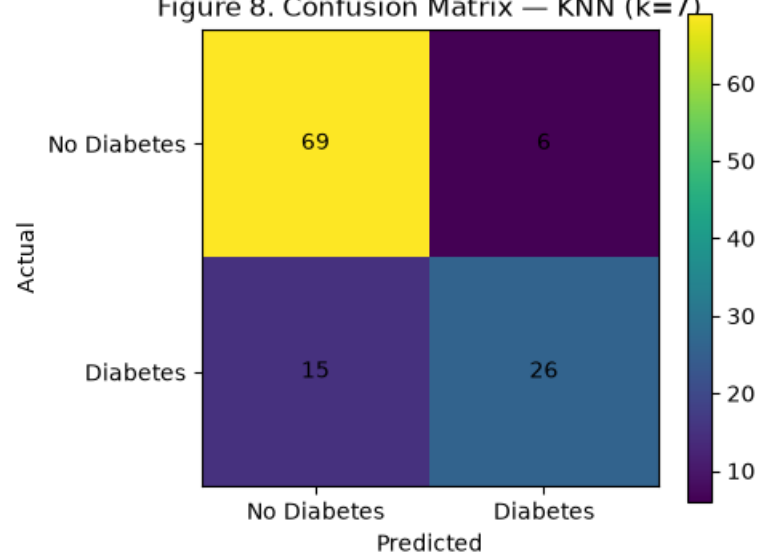
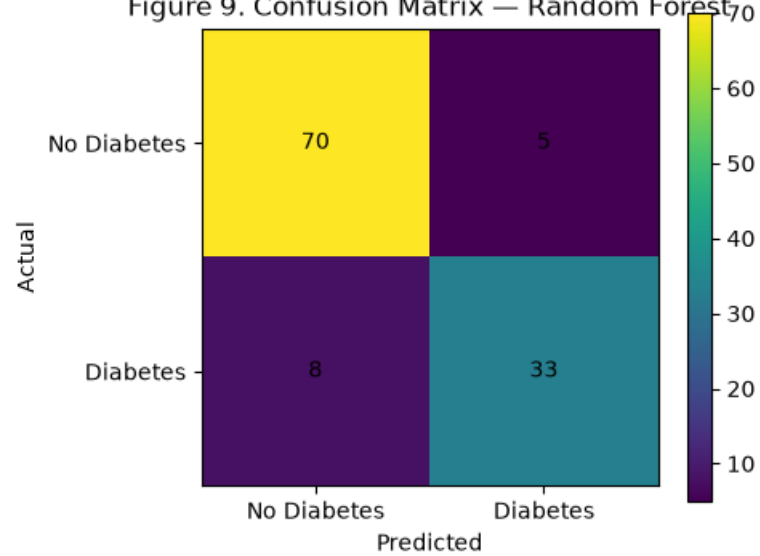
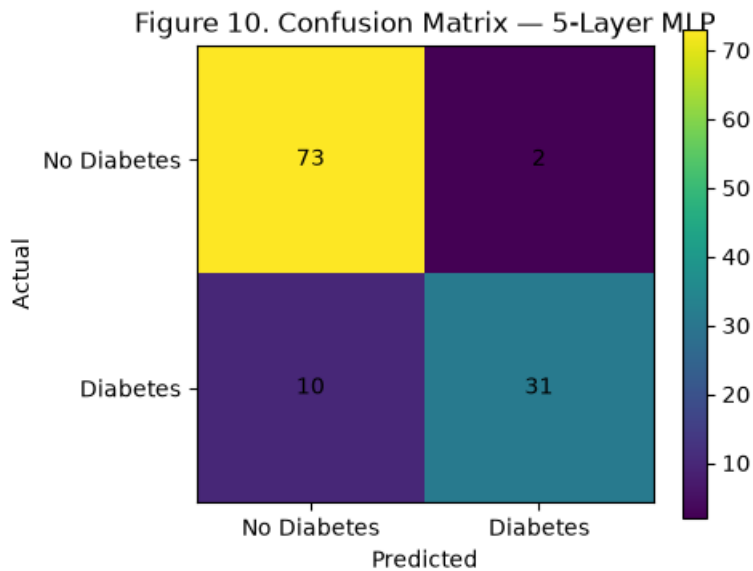


Figure 9. Confusion Matrix — Random Forest





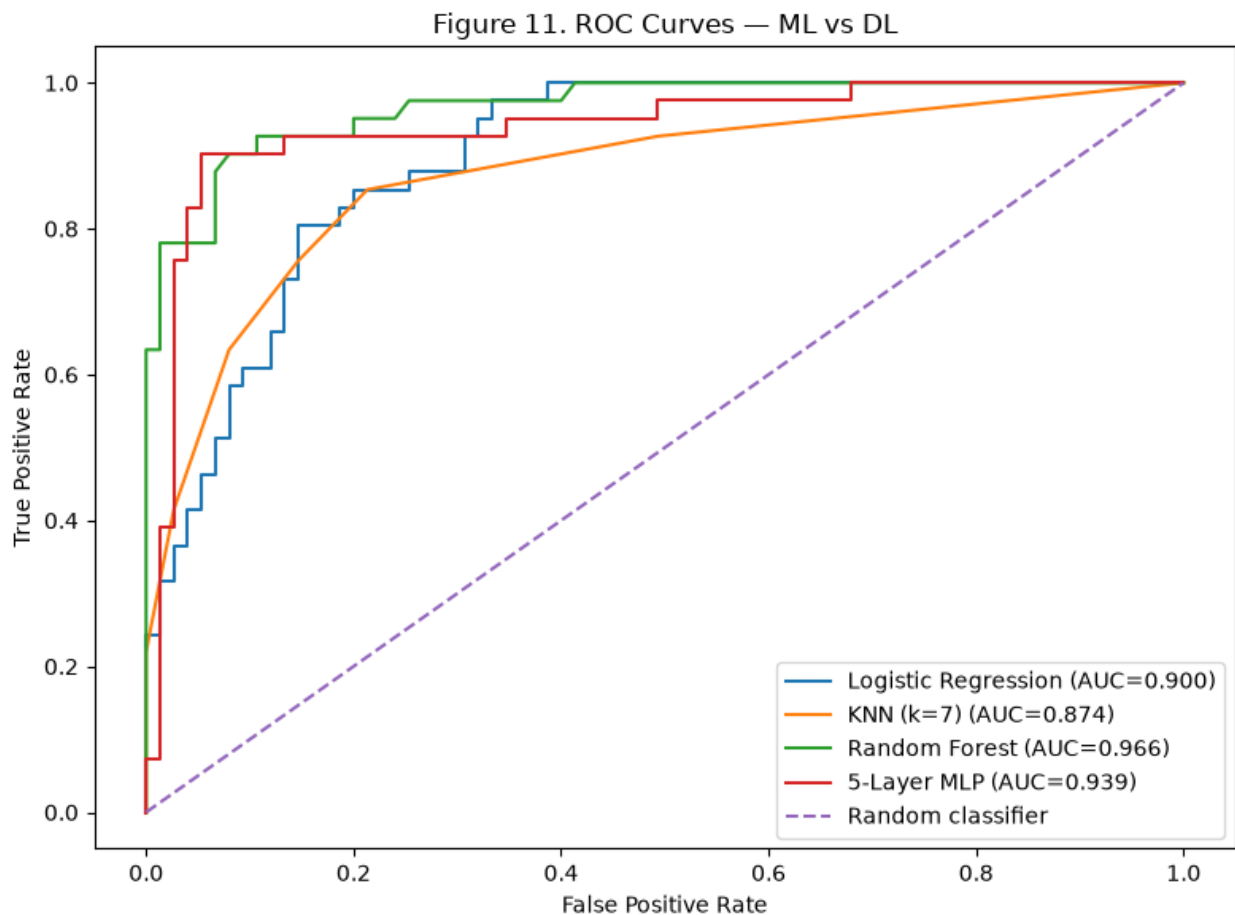
Hình 9. 4 Confusion Matrix cho 4 model

6.4. ROC Curve và ROC-AUC

ROC Curve thể hiện trade-off giữa True Positive Rate và False Positive Rate khi thay đổi threshold. ROC-AUC càng gần 1 thì khả năng phân biệt hai lớp càng tốt. Trong thí nghiệm hiện tại, Random Forest có ROC-AUC cao nhất 0,9663, cao hơn MLP 0,9389, Logistic Regression 0,8995 và KNN 0,8738.

```
plt.figure(figsize=(8,6))
for model_name, probs in final_probabilities.items():
    fpr, tpr, _ = roc_curve(y_test, probs)
    auc = roc_auc_score(y_test, probs)
    plt.plot(fpr, tpr, label=f"{model_name} (AUC={auc:.3f})")

plt.plot([0,1], [0,1], linestyle="--", label="Random classifier")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Figure 11. ROC Curves – ML vs DL")
plt.legend()
plt.tight_layout()
plt.show()
```



Hình 10. ROC Curve của các mô hình trên Test Set

Giải thích: Biểu đồ ROC cho thấy Random Forest có khả năng phân biệt Diabetes và No Diabetes tốt nhất với ROC-AUC = 0,966. Mô hình 5-Layer MLP đứng thứ hai với AUC = 0,939, tiếp theo là Logistic Regression và KNN. Tất cả các mô hình đều tốt hơn rõ rệt so với bộ phân loại ngẫu nhiên.

6.5. Chi phí tính toán

Bên cạnh hiệu năng dự đoán, Assignment 3 yêu cầu xem xét độ phức tạp và chi phí huấn luyện. MLP có thời gian fit cao hơn Logistic Regression và KNN, trong khi Random

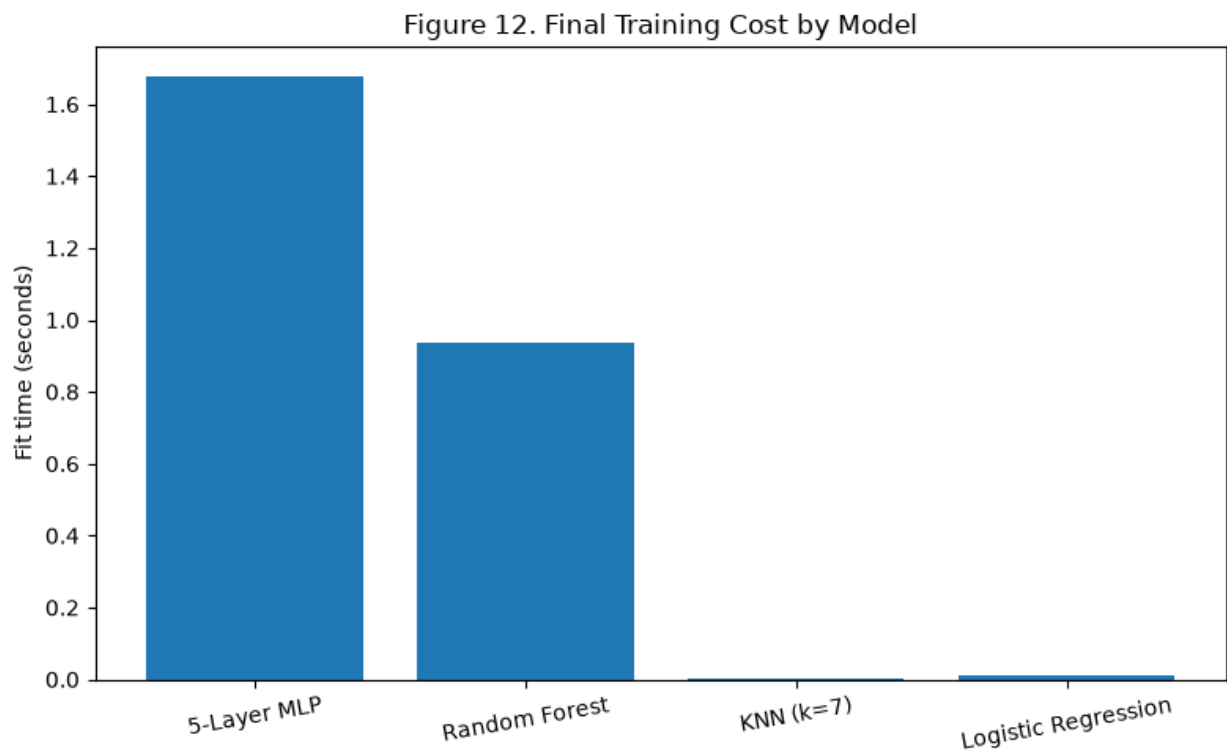
Forest cũng có chi phí huấn luyện đáng kể. Vì vậy một cải thiện nhỏ về Accuracy không đồng nghĩa model phức tạp hơn luôn là lựa chọn tốt hơn về mặt triển khai.

```
cost_df = comparison[["Model", "Fit_Time_s", "Inference_Time_ms"]].copy()
cost_df.round(4)
```

	Model	Fit_Time_s	Inference_Time_ms
0	5-Layer MLP	1.6756	1.0065
1	Random Forest	0.9373	56.4468
2	KNN (k=7)	0.0031	4.8732
3	Logistic Regression	0.0099	0.8472

Hình 11. Bảng chi phí tính toán

```
plt.figure(figsize=(8,5))
plt.bar(cost_df["Model"], cost_df["Fit_Time_s"])
plt.ylabel("Fit time (seconds)")
plt.title("Figure 12. Final Training Cost by Model")
plt.xticks(rotation=10)
plt.tight_layout()
plt.show()
```



Hình 12. Biểu đồ Fit time

Giải thích: Dựa vào biểu đồ ta thấy Model MLP 5 lớp tốn nhiều thời gian để train hơn 3 model ML còn lại

7. KHẢ NĂNG DIỄN GIẢI VÀ REPRESENTATION LEARNING

7.1. Random Forest Feature Importance

Random Forest cho phép xem mức độ quan trọng tương đối của từng feature trong quá trình ra quyết định. Trong lần chạy hiện tại, Insulin có importance cao nhất (0,3630), tiếp theo là Glucose (0,1499) và SkinThickness (0,1463).

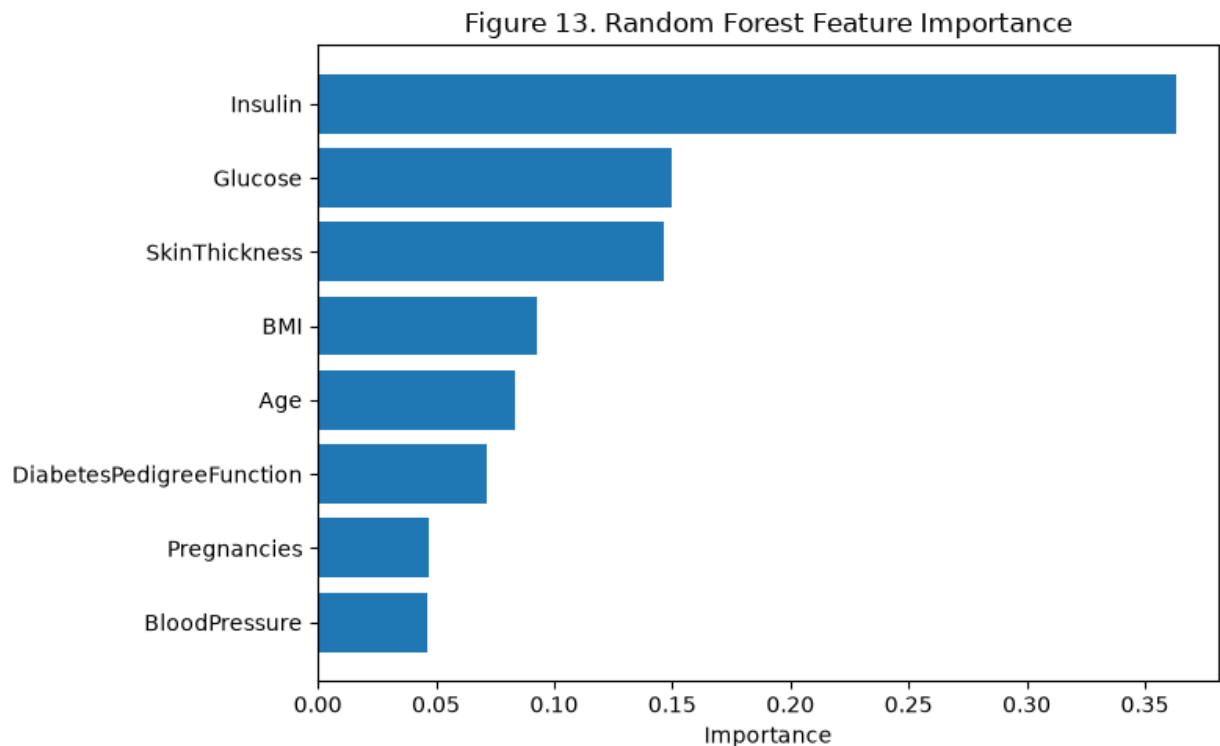
```
: importance_df = pd.DataFrame({  
    "Feature": X.columns,  
    "Importance": final_rf.feature_importances_  
}).sort_values("Importance", ascending=False)  
importance_df
```

	Feature	Importance
4	Insulin	0.363015
1	Glucose	0.149931
3	SkinThickness	0.146319
5	BMI	0.092438
7	Age	0.083600
6	DiabetesPedigreeFunction	0.071197
0	Pregnancies	0.047104
2	BloodPressure	0.046395

Hình 13. Bảng mức độ quan trọng với từng feature

Feature importance chỉ phản ánh cách mô hình sử dụng biến cho prediction; không được diễn giải là bằng chứng về quan hệ nhân quả y khoa.

```
plt.figure(figsize=(8,5))
plt.barh(importance_df["Feature"], importance_df["Importance"])
plt.gca().invert_yaxis()
plt.xlabel("Importance")
plt.title("Figure 13. Random Forest Feature Importance")
plt.tight_layout()
plt.show()
```



Hình 14. Random Forest Feature Importance.

Giải thích: Biểu đồ Feature Importance cho thấy Random Forest sử dụng Insulin nhiều nhất trong quá trình dự đoán, tiếp theo là Glucose và SkinThickness. BMI, Age và DiabetesPedigreeFunction có mức ảnh hưởng thấp hơn, trong khi Pregnancies và BloodPressure đóng góp ít nhất. Tuy nhiên, Feature Importance chỉ phản ánh mức độ mô hình sử dụng đặc trưng để dự đoán, không đồng nghĩa với quan hệ nhân quả về mặt y khoa.

7.2. Learned Representation của Deep Learning

Khác với ML truyền thống sử dụng prepared features trực tiếp, mạng MLP liên tục biến đổi input qua các hidden layer. Chuỗi biểu diễn có thể viết là $X \rightarrow H_1 \rightarrow H_2 \rightarrow H_3 \rightarrow H_4 \rightarrow \hat{y}$. H_4 là representation 8 chiều được học tự động từ dữ liệu chứ không tương ứng trực tiếp với tám feature ban đầu theo tên gọi.

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow H_3 \rightarrow H_4 \rightarrow \hat{y}$$

Notebook sử dụng PCA để trực quan hóa không gian input ban đầu và không gian H₄ học được. PCA ở đây chỉ phục vụ visualization; nó không phải một phần của training pipeline. Sự thay đổi cấu trúc giữa hai hình cho thấy mạng đã biến đổi representation theo hướng phục vụ nhiệm vụ classification.

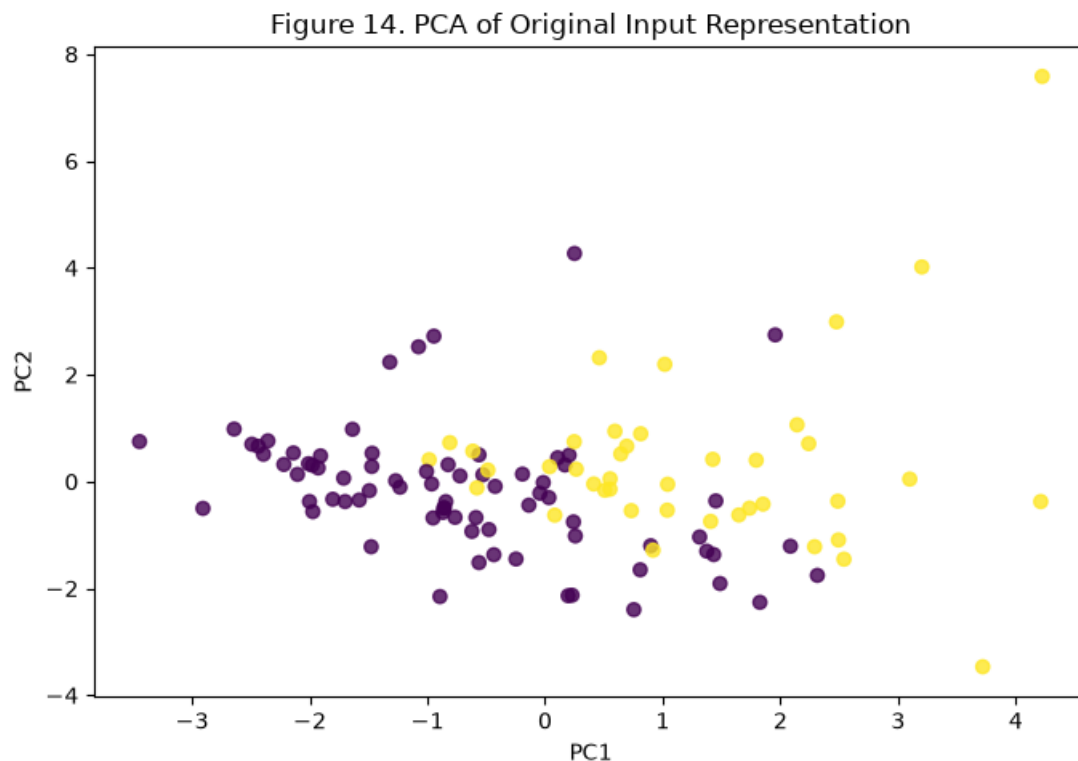
```
: final_mlp.eval()
with torch.no_grad():
    h4 = final_mlp.representation(X_test_tensor).cpu().numpy()

print("Original standardized test shape:", X_test_scaled.shape)
print("Learned H4 representation shape :", h4.shape)

pca_raw = PCA(n_components=2, random_state=RANDOM_STATE)
raw_2d = pca_raw.fit_transform(X_test_scaled)

plt.figure(figsize=(7,5))
plt.scatter(raw_2d[:,0], raw_2d[:,1], c=y_test.to_numpy(), alpha=0.8)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Figure 14. PCA of Original Input Representation")
plt.tight_layout()
plt.show()
```

Original standardized test shape: (116, 8)
Learned H4 representation shape : (116, 8)



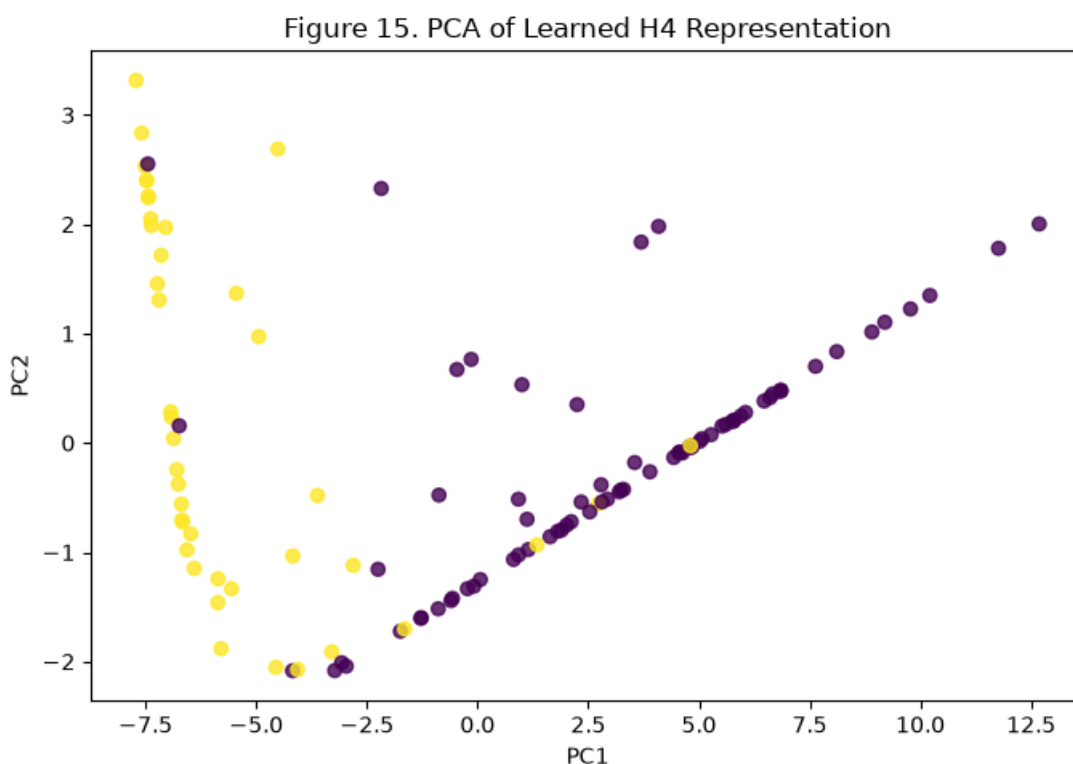
Hình 15. Original representation

Giải thích: Biểu đồ PCA trực quan hóa không gian đặc trưng ban đầu của tập Test sau khi chuẩn hóa. Tám đặc trưng được giảm xuống hai thành phần chính PC1 và PC2. Hai lớp

Diabetes và No Diabetes vẫn có sự chồng lấn đáng kể, cho thấy dữ liệu trong không gian đặc trưng ban đầu chưa được phân tách rõ ràng chỉ bằng phép chiếu tuyến tính PCA.

```
pca_hidden = PCA(n_components=2, random_state=RANDOM_STATE)
hidden_2d = pca_hidden.fit_transform(h4)

plt.figure(figsize=(7,5))
plt.scatter(hidden_2d[:,0], hidden_2d[:,1], c=y_test.to_numpy(), alpha=0.8)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Figure 15. PCA of Learned H4 Representation")
plt.tight_layout()
plt.show()
```



Hình 16. Learned H4 representation

Giải thích: Biểu đồ PCA của H4 cho thấy không gian biểu diễn sau khi đi qua các hidden layer của MLP đã thay đổi đáng kể so với không gian đầu vào ban đầu. Các điểm dữ liệu được sắp xếp lại theo cấu trúc mới, cho thấy mạng đã học được các đặc trưng ẩn phục vụ cho việc phân biệt Diabetes và No Diabetes. Tuy nhiên, hai lớp vẫn còn một số vùng chồng lấn nên chưa được tách hoàn toàn.

8. Kết luận

Trong ba mô hình Machine Learning truyền thống, Random Forest thể hiện hiệu năng tổng thể tốt nhất, vượt Logistic Regression và KNN ở Accuracy, Recall, F1-score và

ROC-AUC. Điều này cho thấy các quan hệ trong dữ liệu Diabetes không chỉ mang tính tuyến tính hoặc dựa đơn giản trên khoảng cách cục bộ.

So sánh ML và DL cho thấy MLP 5 lớp đạt Accuracy, Precision và F1-score cao hơn Random Forest một lượng nhỏ; ngược lại Random Forest đạt Recall và ROC-AUC cao hơn. Vì vậy không thể kết luận DL tốt hơn chỉ vì mô hình sâu hơn. Lựa chọn cuối cần phụ thuộc vào mục tiêu của hệ thống.

Nếu ưu tiên phát hiện càng nhiều bệnh nhân thực sự mắc Diabetes càng tốt, Recall là metric quan trọng và Random Forest có lợi thế. Nếu ưu tiên Accuracy tổng thể và Precision cao khi hệ thống đưa ra cảnh báo Diabetes, MLP là một lựa chọn cạnh tranh.

Về khả năng generalization, MLP cho thấy overfitting rõ sau khoảng epoch 31: Training Loss tiếp tục giảm mạnh trong khi Validation Loss tăng. Điều này minh họa rằng model capacity lớn hơn không tự động tạo generalization tốt hơn. Deep Learning cũng có độ phức tạp cao hơn và khả năng diễn giải thấp hơn so với Logistic Regression hoặc Random Forest.

Model	Representation	Nonlinearity	Complexity	Interpretability	Training Cost
Logistic Regression	Prepared	Linear	Low	High	Low
KNN (k=7)	Prepared	Nonlinear / local	Low	Medium	Low fit / prediction cao hơn
Random Forest	Prepared	Nonlinear	Medium	Medium	Medium
5-Layer MLP	Learned representations	Nonlinear	High (3329 params)	Lower	Higher

II. Hệ thống dự đoán giá nhà

1. DATASET VÀ MÔ TẢ BÀI TOÁN

1.1. Tên và nguồn Dataset

Dataset được sử dụng trong hệ thống là House Price Prediction Dataset Vietnam - 2024. Dữ liệu mô tả các bất động sản tại Việt Nam thông qua những đặc trưng vật lý, pháp lý và nội thất, đồng thời cung cấp giá bán làm biến mục tiêu.

Thông tin	Nội dung
Tên dataset	House Price Prediction Dataset Vietnam - 2024

Nguồn	Kaggle
URL	https://www.kaggle.com/datasets/nguyentienhan/vietnam-housing-dataset-2024
File sử dụng trong notebook	housing_dataset.csv
Số quan sát	30.229
Số cột ban đầu	12
Target	Price
Loại bài toán	Regression (hồi quy)

Dataset path : housing_dataset.csv
Dataset shape: (30229, 12)

	Address	Area	Frontage	Access Road	House direction	Balcony direction	Floors	Bedrooms	Bathrooms	Legal status	Furniture state	Price
0	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	84.0	NaN	NaN	NaN	NaN	4.0	NaN	NaN	Have certificate	NaN	8.60
1	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	60.0	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN	7.50
2	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	90.0	6.0	13.0	Đông - Bắc	Đông - Bắc	5.0	NaN	NaN	Sale contract	NaN	8.90
3	Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ C...	54.0	NaN	3.5	Tây - Nam	Tây - Nam	2.0	2.0	3.0	Have certificate	Full	5.35
4	Đường Quang Trung, Phường 8, Gò Vấp, Hồ Chí Minh	92.0	NaN	NaN	Đông - Nam	Đông - Nam	2.0	4.0	4.0	Have certificate	Full	6.90

Hình 17. . Tổng quan dữ liệu House Price Prediction Dataset Vietnam - 2024

1.2. Phát biểu bài toán

Mục tiêu của hệ thống là dự đoán giá của một bất động sản dựa trên các đặc trưng của ngôi nhà. Khác với bài toán Diabetes là phân loại nhị phân, giá nhà là một đại lượng liên tục nên đây là bài toán hồi quy.

$$X = [x_1, x_2, \dots, x_d]$$

X là vector đặc trưng của bất động sản.

$$y = \text{Price} \in \mathbb{R}$$

y là giá nhà, đơn vị trong dataset là tỷ đồng.

Trong thí nghiệm, ba mô hình Machine Learning truyền thống được so sánh với một mạng Deep Learning 5 lớp. Các mô hình gồm Linear Regression, Random Forest Regressor, Gradient Boosting Regressor và 5-Layer Multi-Layer Perceptron (MLP).

Mục tiêu không chỉ là tìm mô hình có sai số thấp nhất mà còn phân tích khả năng tổng quát hóa, chi phí huấn luyện, độ phức tạp, khả năng diễn giải và representation mà mạng neural học được.

2. PHÂN TÍCH DỮ LIỆU VÀ CHẤT LƯỢNG DỮ LIỆU

2.1. Cấu trúc dữ liệu và các thuộc tính

Dataset có 30.229 dòng và 12 cột. Một dòng dữ liệu đại diện cho một bất động sản. Cột Price được sử dụng làm target. Cột Address chứa chuỗi địa chỉ có độ phân mảnh rất cao nên không được đưa vào baseline representation, tránh làm số chiều One-Hot tăng mạnh và hạn chế nguy cơ mô hình ghi nhớ các chuỗi địa chỉ cụ thể.

Thuộc tính	Loại	Vai trò / ý nghĩa
Address	Text	Địa chỉ; loại khỏi input baseline do high-cardinality
Area	Numerical	Diện tích
Frontage	Numerical	Mặt tiền
Access Road	Numerical	Độ rộng đường/ngõ tiếp cận
House direction	Categorical	Hướng nhà
Balcony direction	Categorical	Hướng ban công
Floors	Numerical	Số tầng
Bedrooms	Numerical	Số phòng ngủ
Bathrooms	Numerical	Số phòng tắm
Legal status	Categorical	Tình trạng pháp lý
Furniture state	Categorical	Tình trạng nội thất
Price	Numerical	Target — giá nhà

2.2. Missing values và dữ liệu thiếu

Dữ liệu không thiếu ở Area, Address và Price, nhưng nhiều feature khác có tỷ lệ thiếu đáng kể. Đặc biệt, Balcony direction thiếu 82,65%, House direction thiếu 70,26%, Furniture state thiếu 46,71%, Access Road thiếu 43,99% và Frontage thiếu 38,25%. Vì

vậy, xóa toàn bộ các dòng có missing values sẽ làm mất một lượng lớn dữ liệu và không phù hợp.

```
quality_summary = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing_count": df.isna().sum(),
    "missing_pct": df.isna().mean() * 100,
    "unique_values": df.nunique(dropna=True)
})

display(quality_summary.round(2))

print("Duplicated rows:", df.duplicated().sum())
```

	dtype	missing_count	missing_pct	unique_values
Address	str	0	0.00	10265
Area	float64	0	0.00	1512
Frontage	float64	11564	38.25	407
Access Road	float64	13297	43.99	140
House direction	str	21239	70.26	8
Balcony direction	str	24983	82.65	8
Floors	float64	3603	11.92	10
Bedrooms	float64	5162	17.08	9
Bathrooms	float64	7074	23.40	9
Legal status	str	4506	14.91	2
Furniture state	str	14119	46.71	2
Price	float64	0	0.00	687

Duplicated rows: 0

Hình 18. Bảng missing_count, missing_pct và unique_values

Notebook không phát hiện exact duplicate và cũng không phát hiện target Price không hợp lệ. Các giá trị numerical âm và $\text{Area} \leq 0$ được kiểm tra; không có dòng nào bị loại theo các điều kiện này. Dataset sau bước làm sạch vẫn giữ 30.229 quan sát.

2.3. Thống kê mô tả của target Price

Giá nhà trong dataset nằm từ 1,0 đến 11,5 tỷ đồng, giá trung bình khoảng 5,87 tỷ và median khoảng 5,9 tỷ. Khoảng biến thiên tương đối rộng cho thấy mô hình cần học quan hệ giữa nhiều feature thay vì chỉ dựa trên một biến đơn lẻ.

Thống kê Price	Giá trị
Min	1,0 tỷ
Q1	4,2 tỷ
Median	5,9 tỷ
Mean	5,8721 tỷ
Q3	7,5 tỷ

Max	11,5 tỷ
-----	---------

3. KHÁM PHÁ DỮ LIỆU (EDA)

3.1. Phân bố giá nhà

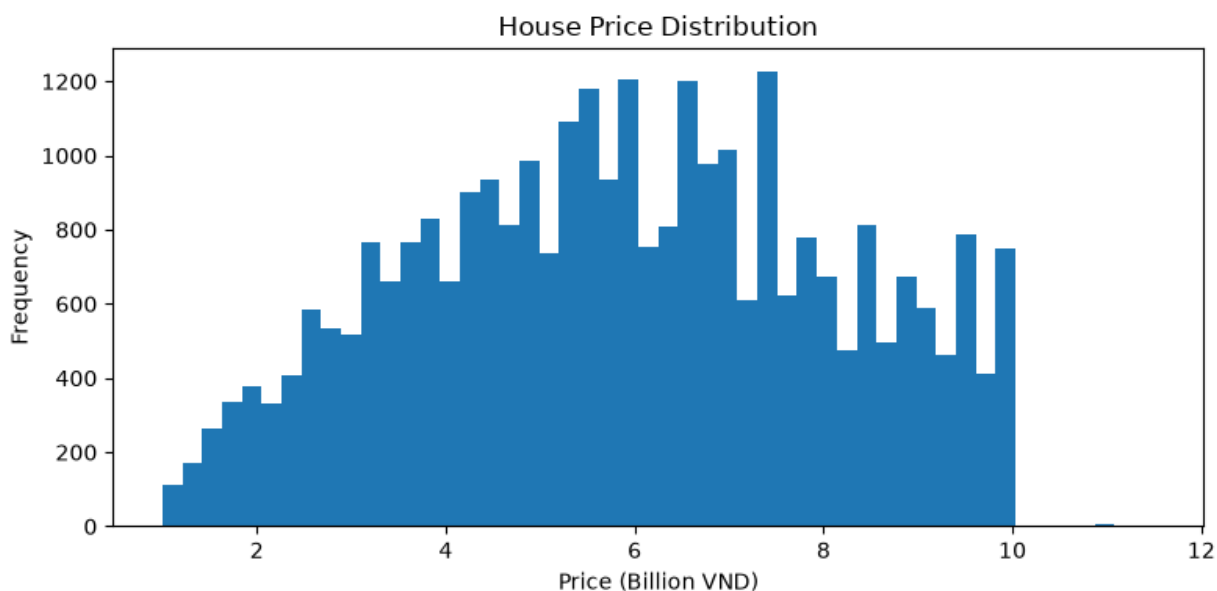
Biểu đồ phân bố Price cho thấy giá nhà trải trên khoảng 1–11,5 tỷ đồng. Việc quan sát distribution giúp kiểm tra target có bị lệch mạnh hay xuất hiện vùng giá thừa dữ liệu hay không. Kết quả cũng cho thấy không nên giả định rằng Price có quan hệ tuyến tính đơn giản với toàn bộ feature.

```
plt.figure(figsize=(8, 4))

plt.hist(
    y,
    bins=50
)

plt.xlabel("Price (Billion VND)")
plt.ylabel("Frequency")
plt.title("House Price Distribution")

plt.tight_layout()
plt.show()
```



Hình 19. Phân bố giá nhà trong dataset

3.2. Quan hệ giữa Area và Price

Scatter plot giữa Area và Price cho phép quan sát trực tiếp mức độ liên hệ giữa diện tích và giá. Tuy diện tích có ảnh hưởng đến giá, các điểm dữ liệu phân tán khá mạnh, cho thấy Price còn phụ thuộc vào nhiều yếu tố khác như số phòng tắm, số tầng, đường tiếp cận, tình trạng pháp lý và nội thất.

```

if "Area" in df_clean.columns:

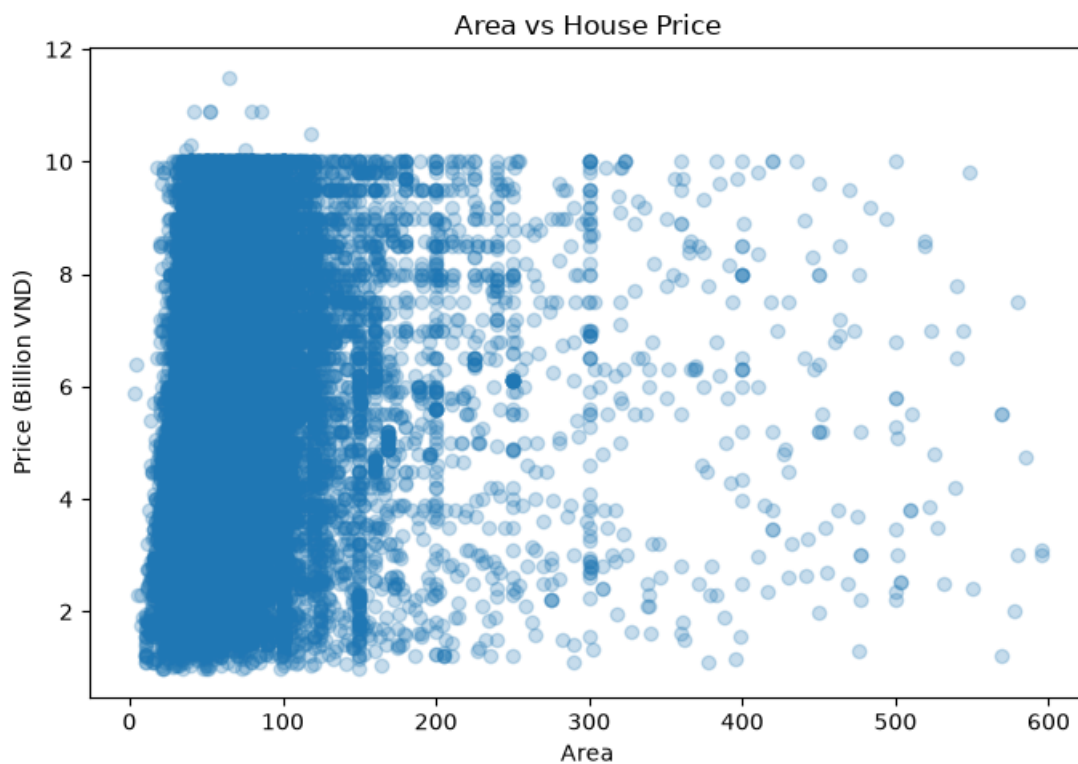
    plt.figure(figsize=(7, 5))

    plt.scatter(
        df_clean["Area"],
        df_clean[TARGET],
        alpha=0.25
    )

    plt.xlabel("Area")
    plt.ylabel("Price (Billion VND)")
    plt.title("Area vs House Price")

    plt.tight_layout()
    plt.show()

```



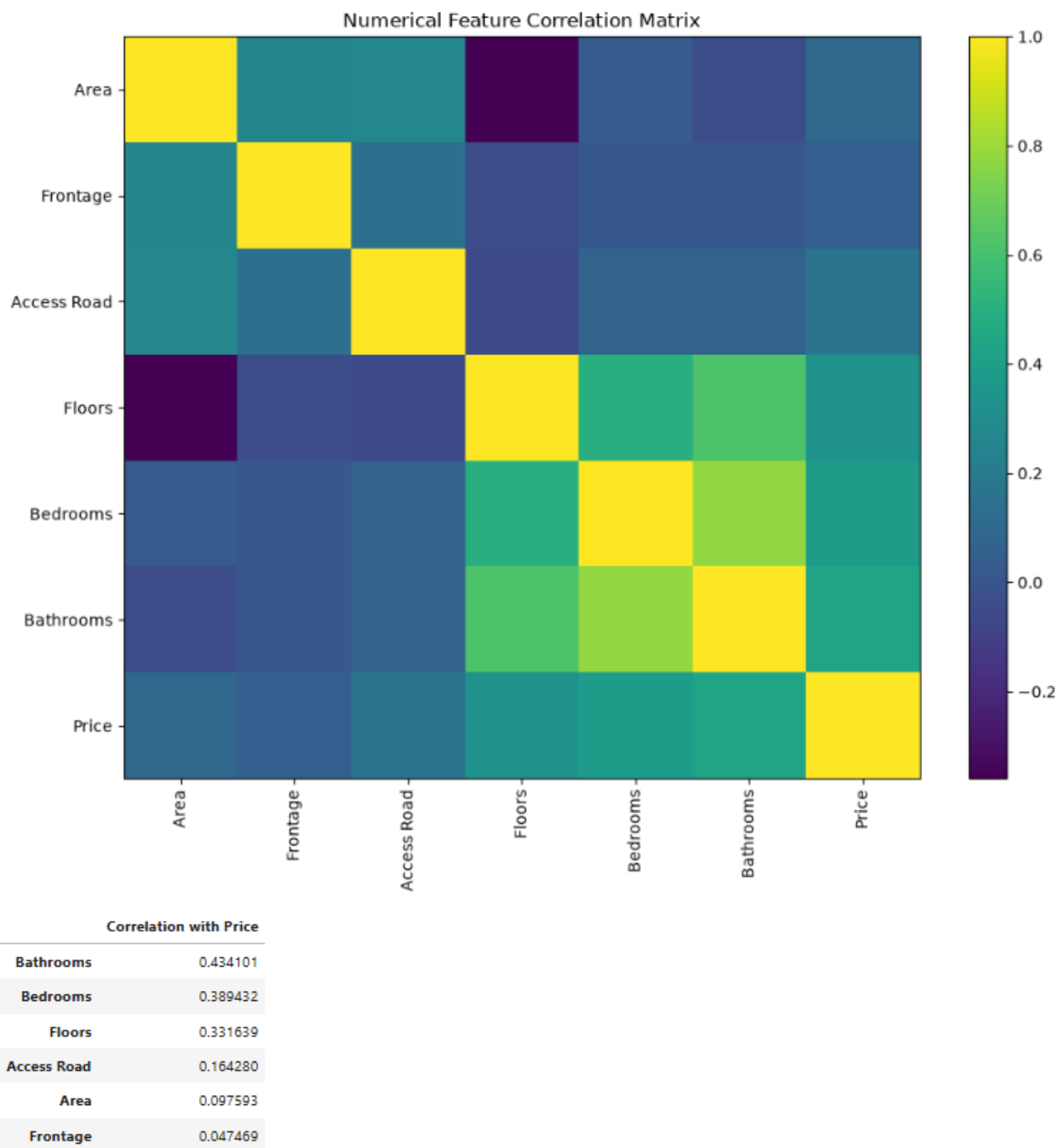
Hình 20. Mối quan hệ giữa diện tích và giá nhà

3.3. Outlier và tương quan

Các boxplot của Area, Frontage, Access Road, Floors, Bedrooms và Bathrooms được dùng để kiểm tra các giá trị cực trị. Trong dữ liệu bất động sản, outlier không nhất thiết là dữ liệu sai; một ngôi nhà có diện tích hoặc mặt tiền rất lớn vẫn có thể là quan sát hợp lệ. Vì vậy notebook giữ các giá trị hợp lệ thay vì xóa tự động chỉ vì chúng nằm xa phần lớn dữ liệu.

Phân tích correlation cho các biến số cho thấy Bathrooms có tương quan tuyến tính với Price cao nhất (0,4341), tiếp theo là Bedrooms (0,3894) và Floors (0,3316). Area chỉ có

correlation khoảng 0,0976, cho thấy quan hệ giá nhà không thể được giải thích đầy đủ chỉ bằng diện tích.



Hình 21. Ma trận tương quan giữa các biến numerical và Price.

4. THIẾT KẾ THÍ NGHIỆM VÀ BIỂU DIỄN DỮ LIỆU

4.1. Xác định input và target

Sau khi loại Address và Price khỏi input, mô hình sử dụng 10 raw features. Trong đó có 6 numerical features: Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms; và 4 categorical features: House direction, Balcony direction, Legal status, Furniture state.

$$X \in \mathbb{R}^{(N \times d)}, \quad y \in \mathbb{R}^N$$

Trước encoding, X có 10 raw feature; sau preprocessing, $d = 26$.

4.2. Chia Train / Validation / Test

Dữ liệu được chia thành 70% Train, 15% Validation và 15% Test. Do target Price là continuous value, quá trình split không dùng stratify. Training set dùng để học parameters; Validation set dùng để so sánh các cấu hình và chọn best epoch cho MLP; Test set chỉ dùng ở bước đánh giá cuối cùng.

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=RANDOM_STATE
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=RANDOM_STATE
)

split_summary = pd.DataFrame({
    "Set": [
        "Train",
        "Validation",
        "Test"
    ],
    "Rows": [
        len(X_train),
        len(X_val),
        len(X_test)
    ],
    "Percentage": [
        len(X_train) / len(X) * 100,
        len(X_val) / len(X) * 100,
        len(X_test) / len(X) * 100
    ],
    "Mean Price": [
        y_train.mean(),
        y_val.mean(),
        y_test.mean()
    ]
})

display(
    split_summary.round(4)
)
```

	Set	Rows	Percentage	Mean Price
0	Train	21160	69.9990	5.8757
1	Validation	4534	14.9988	5.8602
2	Test	4535	15.0022	5.8670

Hình 22. Phân chia Train, Validation và Test.

4.3. Preprocessing và Representation

Mô hình không thể sử dụng trực tiếp các chuỗi categorical. Vì vậy, notebook xây dựng một ColumnTransformer gồm hai pipeline. Numerical pipeline dùng median imputation để bù missing values và StandardScaler để chuẩn hóa. Categorical pipeline dùng most-frequent imputation rồi One-Hot Encoding với `handle_unknown="ignore"`.

$$x' = \frac{x - \mu}{\sigma}$$

Công thức Standardization cho numerical features.

Quan trọng: preprocessor chỉ được fit trên Training set. Validation và Test chỉ dùng transform với các thông kê và category mapping đã học từ Train. Nguyên tắc này giúp tránh data leakage.

Sau preprocessing, 10 raw features được mở rộng thành 26 numerical dimensions. Vì vậy, mạng Deep Learning thực tế không nhận vector 10 chiều mà nhận tensor có 26 features.

```
Raw train shape: (21160, 10)
Processed train shape: (21160, 26)
Processed validation shape: (4534, 26)
Processed test shape: (4535, 26)
Processed feature count: 26
Processed dtype: float64
```

Raw representation:

	Area	Frontage	Access Road	House direction	Balcony direction	Floors	Bedrooms	Bathrooms	Legal status	Furniture state
30205	75.0	5.0	5.5	NaN	NaN	1.0	2.0	2.0	Have certificate	Full

Numerical representation:

	numerical_Area	numerical_Frontage	numerical_Access Road	numerical_Floors	numerical_Bedrooms	numerical_Bathrooms	categorical_Hc direction
0	0.141052	-0.003974	-0.270927	-1.886174	-1.17792	-1.022481	

1 rows × 26 columns

Hình 23. Chuyển đổi từ raw property record sang numerical representation

5. METRICS CHO BÀI TOÁN HỒI QUY

Vì House Price là regression, các metric Accuracy, Precision, Recall và F1 không phù hợp. Notebook sử dụng MAE, MSE, RMSE và R^2 để đánh giá sai số và khả năng giải thích biến thiên của target.

5.1. Mean Absolute Error (MAE)

$$\text{MAE} = (1/N) \sum |y_i - \hat{y}_i|$$

MAE biểu diễn độ lệch tuyệt đối trung bình giữa giá thực tế và giá dự đoán. Với target tính theo tỷ đồng, $\text{MAE} = 1,45$ nghĩa là dự đoán lệch trung bình khoảng 1,45 tỷ đồng.

5.2. Mean Squared Error và RMSE

$$\text{MSE} = (1/N) \sum (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

MSE bình phương lỗi nên phạt mạnh các prediction sai lớn. RMSE đưa sai số trở lại cùng đơn vị với Price và do đó dễ diễn giải hơn. Khi RMSE lớn đáng kể so với MAE, thường tồn tại một số prediction có lỗi lớn.

5.3. Hệ số R^2

$$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$$

R^2 càng gần 1 càng tốt. $R^2 = 0$ nghĩa là mô hình không tốt hơn baseline dự đoán bằng giá trung bình. R^2 âm nghĩa là mô hình còn tệ hơn baseline đó.

6. BA MÔ HÌNH MACHINE LEARNING TRUYỀN THỐNG

6.1. Linear Regression

Linear Regression được dùng làm baseline tuyến tính. Mô hình giả định Price có thể được xấp xỉ bằng tổ hợp tuyến tính của các numerical dimensions sau preprocessing. Ưu điểm là đơn giản, train nhanh và dễ diễn giải, nhưng khả năng mô tả quan hệ phi tuyến còn hạn chế.

6.2. Random Forest Regressor

Random Forest Regressor là ensemble của nhiều Decision Trees. Mỗi cây học các quy tắc phân tách dữ liệu khác nhau và kết quả cuối được tổng hợp từ nhiều cây. Mô hình có khả năng học quan hệ phi tuyến và interaction giữa các feature tốt hơn Linear Regression.

6.3. Gradient Boosting Regressor

Gradient Boosting xây dựng các weak learners theo tuần tự. Mỗi cây mới tập trung sửa lỗi còn lại của ensemble hiện tại. Cách học này thường cho kết quả tốt với tabular data nhưng có thể nhạy với hyperparameters và cần kiểm soát overfitting.

6.4. Kết quả Validation của 3 ML

	Model	MAE	MSE	RMSE	R2	Training Time (s)
0	Gradient Boosting	1.5038	3.4567	1.8592	0.3032	5.4335
1	Random Forest	1.4801	3.6097	1.8999	0.2723	7.0903
2	Linear Regression	1.6378	3.9786	1.9946	0.1980	0.0616

Hình 24. So sánh các mô hình Machine Learning trên Validation set.

Trên Validation set, Gradient Boosting có RMSE thấp nhất và R^2 cao nhất, trong khi Random Forest đạt MAE thấp nhất. Linear Regression yếu hơn rõ rệt, cho thấy quan hệ giữa feature và giá nhà không được mô tả đầy đủ bằng một hàm tuyến tính đơn giản.

7. MÔ HÌNH DEEP LEARNING 5 LỚP

7.1. Kiến trúc mạng

Mạng Deep Learning được cài đặt bằng PyTorch với 5 trainable Linear layers. Input dimension là 26 sau preprocessing. Bốn hidden layers lần lượt có 128, 64, 32 và 16 neurons, output layer có 1 neuron để dự đoán một giá trị Price liên tục.

26 → 128 → 64 → 32 → 16 → 1

Layer	Transformation	Activation	Vai trò
1	26 → 128	ReLU	Học representation H1
2	128 → 64	ReLU	Học representation H2
3	64 → 32	ReLU	Học representation H3
4	32 → 16	ReLU	Học representation H4
5	16 → 1	Không dùng Sigmoid	Dự đoán Price liên tục

Điểm khác biệt quan trọng với Diabetes là output layer không dùng Sigmoid. Sigmoid giới hạn output trong khoảng 0–1, trong khi Price có thể nhận các giá trị liên tục theo đơn vị tỷ đồng.

```

device = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)

INPUT_DIM = (
    X_train_processed.shape[1]
)

dl_model = HousePriceMLP(
    input_dim=INPUT_DIM
).to(device)

print("Device :", device)
print("Input dim:", INPUT_DIM)
print(dl_model)

```

```

Device : cpu
Input dim: 26
HousePriceMLP(
  (fc1): Linear(in_features=26, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=64, bias=True)
  (fc3): Linear(in_features=64, out_features=32, bias=True)
  (fc4): Linear(in_features=32, out_features=16, bias=True)
  (fc5): Linear(in_features=16, out_features=1, bias=True)
  (relu): ReLU()
)

```

Hình 25. Kiến trúc mạng MLP 5 lớp cho dự đoán giá nhà.

7.2. Forward Propagation và ReLU

$$H_1 = \text{ReLU}(XW_1 + b_1)$$

$$H_2 = \text{ReLU}(H_1W_2 + b_2)$$

$$H_3 = \text{ReLU}(H_2W_3 + b_3)$$

$$H_4 = \text{ReLU}(H_3W_4 + b_4)$$

$$\hat{y} = H_4W_5 + b_5$$

$$\text{ReLU}(z) = \max(0, z)$$

Mỗi hidden layer biến đổi representation của layer trước. ReLU tạo tính phi tuyến; nếu chỉ ghép nhiều Linear layers mà không có activation phi tuyến, toàn bộ network vẫn có thể rút gọn thành một affine transformation và lợi ích của độ sâu sẽ bị hạn chế.

7.3. Tensor shapes và mini-batch

Sau preprocessing, Training tensor có shape (21.160, 26) và target tensor có shape (21.160, 1). Batch size được đặt bằng 64 nên một mini-batch có input shape (64, 26) và output target shape (64, 1).

$$B \times 26 \rightarrow B \times 128 \rightarrow B \times 64 \rightarrow B \times 32 \rightarrow B \times 16 \rightarrow B \times 1$$

7.4. Số lượng parameters

Với Linear layer từ d_{in} sang d_{out} , số parameters gồm $d_{in} \times d_{out}$ weights và d_{out} biases.

$$\text{Parameters} = d_{in} \times d_{out} + d_{out}$$

Tổng số trainable parameters của mạng HousePriceMLP là 14.337. Con số này phản ánh mức model capacity lớn hơn đáng kể so với baseline tuyến tính và cũng là một lý do cần theo dõi overfitting.

Parameter group	Shape	Số params
fc1.weight	(128, 26)	3.328
fc1.bias	(128,)	128
fc2.weight	(64, 128)	8.192
fc2.bias	(64,)	64
fc3.weight	(32, 64)	2.048
fc3.bias	(32,)	32
fc4.weight	(16, 32)	512
fc4.bias	(16,)	16
fc5.weight	(1, 16)	16
fc5.bias	(1,)	1

7.5. Loss function và Optimizer

Do đây là regression, mạng sử dụng Mean Squared Error làm loss. Optimizer là Adam với learning rate 0,001. Số epoch tối đa là 200, batch size là 64 và early-stopping patience là 25.

$$L = (1/N) \sum (y_i - \hat{y}_i)^2$$

Tham số huấn luyện	Giá trị
Loss	MSELoss
Optimizer	Adam
Learning rate	0,001
Max epochs	200
Batch size	64
Patience	25

7.6. Backpropagation và chu trình huấn luyện

Trong mỗi mini-batch, model thực hiện Forward để tạo prediction, tính MSE loss, lan truyền gradient ngược bằng backpropagation và cuối cùng optimizer cập nhật weights/biases. Quá trình này được lặp lại qua nhiều epoch.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}.$$

Chu trình huấn luyện có thể tóm tắt:

Input Batch → Forward → Loss → Backward → Optimizer Step.

8. HÀNH VI HUẤN LUYỆN VÀ GENERALIZATION

8.1. Training / Validation Loss

Mạng được train tối đa 200 epoch nhưng early stopping dừng tại epoch 66. Validation RMSE tốt nhất xuất hiện tại epoch 41 với giá trị 1,8704. Model state tại epoch tốt nhất được khôi phục để tránh sử dụng model ở giai đoạn validation đã xấu đi.

Training loss giảm dần trong khi validation loss có dao động và không tiếp tục cải thiện ổn định sau vùng best epoch. Khoảng cách giữa training và validation cho thấy model bắt đầu học quá sát Training set nếu tiếp tục train lâu hơn. Early stopping giúp hạn chế hiện tượng này.

Epoch 1/200	Train Loss: 7.6034	Val Loss: 3.8669	Train RMSE: 2.7574	Val RMSE: 1.9664
Epoch 10/200	Train Loss: 3.3489	Val Loss: 3.6370	Train RMSE: 1.8300	Val RMSE: 1.9071
Epoch 20/200	Train Loss: 3.2793	Val Loss: 3.5513	Train RMSE: 1.8109	Val RMSE: 1.8845
Epoch 30/200	Train Loss: 3.2377	Val Loss: 3.5413	Train RMSE: 1.7994	Val RMSE: 1.8818
Epoch 40/200	Train Loss: 3.1876	Val Loss: 3.5205	Train RMSE: 1.7854	Val RMSE: 1.8763
Epoch 50/200	Train Loss: 3.1505	Val Loss: 3.7934	Train RMSE: 1.7750	Val RMSE: 1.9477
Epoch 60/200	Train Loss: 3.1079	Val Loss: 3.6308	Train RMSE: 1.7629	Val RMSE: 1.9055

Early stopping at epoch 66

```

Training completed.
Best epoch      : 41
Best val RMSE   : 1.8704
DL training time : 133.771 seconds

```

Hình 26. Training Loss và Validation Loss của mạng 5 lớp.

8.2. Training / Validation RMSE

RMSE curve cho phép quan sát sai số theo cùng đơn vị với giá nhà. Validation RMSE đạt khoảng 1,8704 tại epoch 41. Việc lựa chọn best epoch theo Validation RMSE giúp model selection dựa trên khả năng generalize thay vì chỉ nhìn Training loss.


```

plt.figure(figsize=(8, 5))

plt.plot(
    epochs_ran,
    train_rmse_history,
    label="Training RMSE"
)

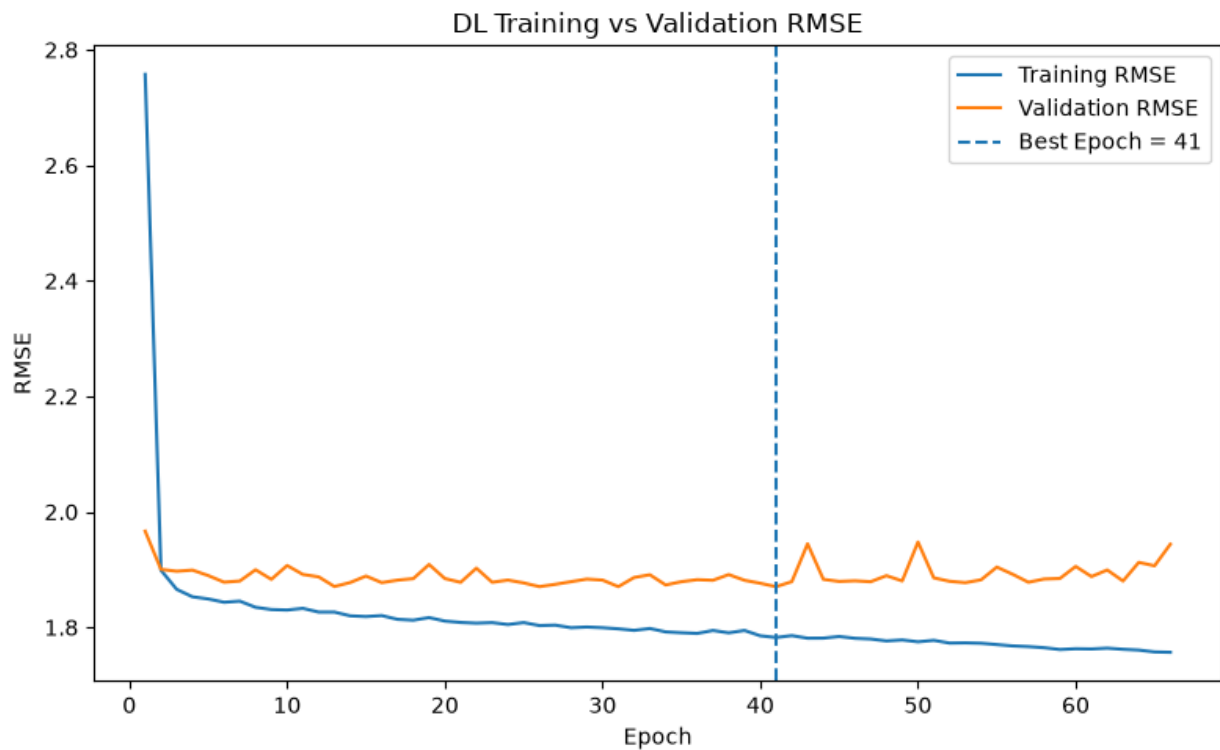
plt.plot(
    epochs_ran,
    val_rmse_history,
    label="Validation RMSE"
)

plt.axvline(
    best_epoch,
    linestyle="--",
    label=f"Best Epoch = {best_epoch}"
)

plt.xlabel("Epoch")
plt.ylabel("RMSE")
plt.title(
    "DL Training vs Validation RMSE"
)
plt.legend()

plt.tight_layout()
plt.show()

```



Hình 27. Training RMSE và Validation RMSE của mạng 5 lớp.

Giải thích: Biểu đồ cho thấy Training RMSE tiếp tục giảm trong suốt quá trình huấn luyện, trong khi Validation RMSE chỉ cải thiện mạnh ở giai đoạn đầu rồi dao động gần như ổn định. Epoch 41 đạt Validation RMSE thấp nhất nên được chọn là best epoch. Sau thời điểm này, việc tiếp tục huấn luyện chủ yếu cải thiện trên tập Train nhưng không cải thiện rõ khả năng tổng quát hóa.

9. SO SÁNH VALIDATION GIỮA ML VÀ DL

	Model	MAE	MSE	RMSE	R2	Training Time (s)
0	Gradient Boosting	1.5038	3.4567	1.8592	0.3032	5.4335
1	5-Layer MLP	1.5100	3.4985	1.8704	0.2948	133.7710
2	Random Forest	1.4801	3.6097	1.8999	0.2723	7.0903
3	Linear Regression	1.6378	3.9786	1.9946	0.1980	0.0616

Hình 28. So sánh ML và DL trên Validation set.

Trên Validation set, Gradient Boosting đứng đầu theo RMSE và R^2 . MLP đứng thứ hai với RMSE 1,8704 và R^2 0,2948, rất gần Gradient Boosting nhưng cần thời gian huấn luyện khoảng 133,77 giây — cao hơn nhiều so với các mô hình ML. Điều này là ví dụ rõ ràng cho nhận định của Assignment 03: Deep Learning không tự động là lựa chọn tốt hơn chỉ vì mô hình sâu hơn.

10. Visualization metrics

Để so sánh trực quan, notebook vẽ riêng MAE, RMSE, R^2 và Training Time cho bốn mô hình. Với MAE/RMSE, cột thấp hơn là tốt hơn; với R^2 , cột cao hơn là tốt hơn. Training Time cho thấy trade-off giữa performance và computational cost.

```

for metric in [
    "MAE",
    "RMSE",
    "R2",
    "Training Time (s)"
]:

    plt.figure(figsize=(9, 4))

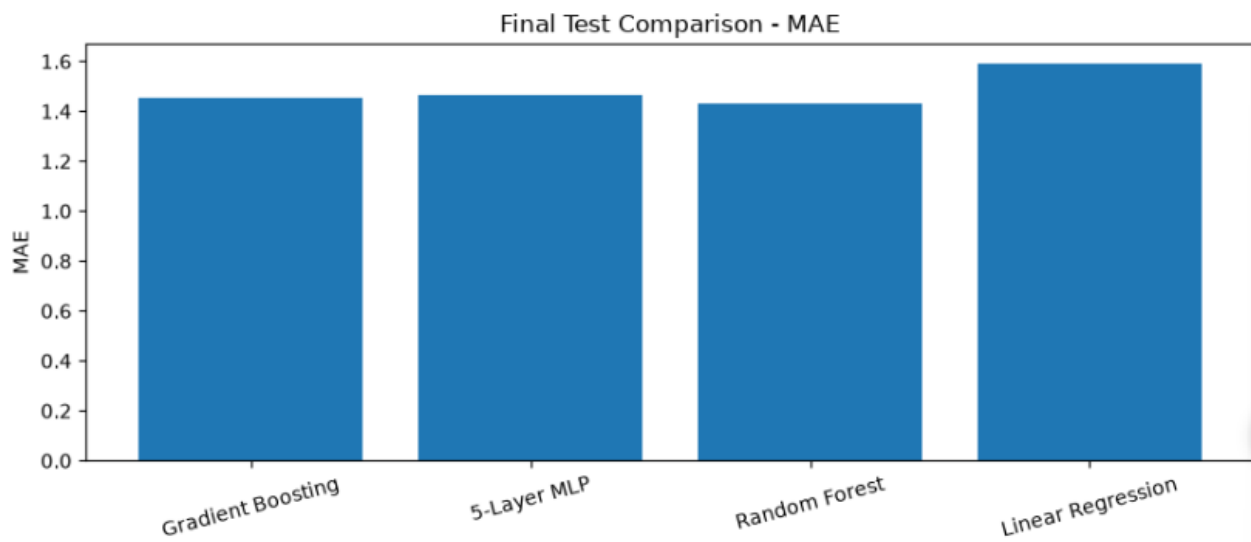
    plt.bar(
        final_results_df["Model"],
        final_results_df[metric]
    )

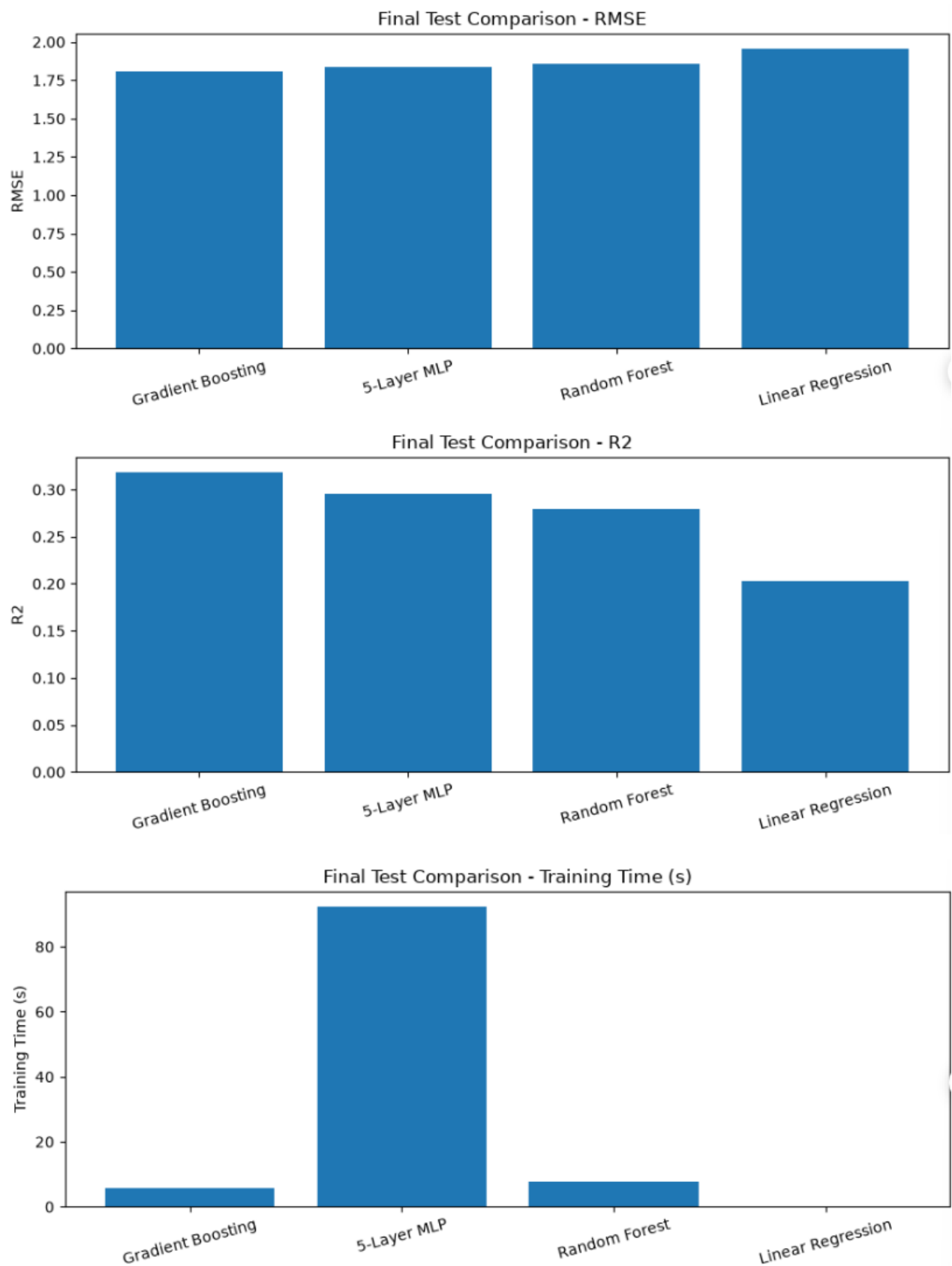
    plt.ylabel(metric)

    plt.title(
        f"Final Test Comparison - {metric}"
    )

    plt.xticks(rotation=15)
    plt.tight_layout()
    plt.show()

```





Hình 29. Biểu đồ so sánh metrics

11. PHÂN TÍCH SAI SỐ

11.1. Actual vs Predicted

Biểu đồ Actual vs Predicted sử dụng đường $y = \hat{y}$ làm đường dự đoán lý tưởng. Điểm càng gần đường này thì prediction càng chính xác. Các điểm lệch xa ở vùng giá thấp hoặc cao cho thấy model chưa mô tả hết sự biến thiên của thị trường và vẫn tồn tại các trường hợp dự đoán sai lớn.

```
for model_name, data in (
    final_models.items()
):

    y_pred = data["prediction"]

    plt.figure(figsize=(7, 5))

    plt.scatter(
        y_test,
        y_pred,
        alpha=0.3
    )

    min_value = min(
        y_test.min(),
        np.min(y_pred)
    )

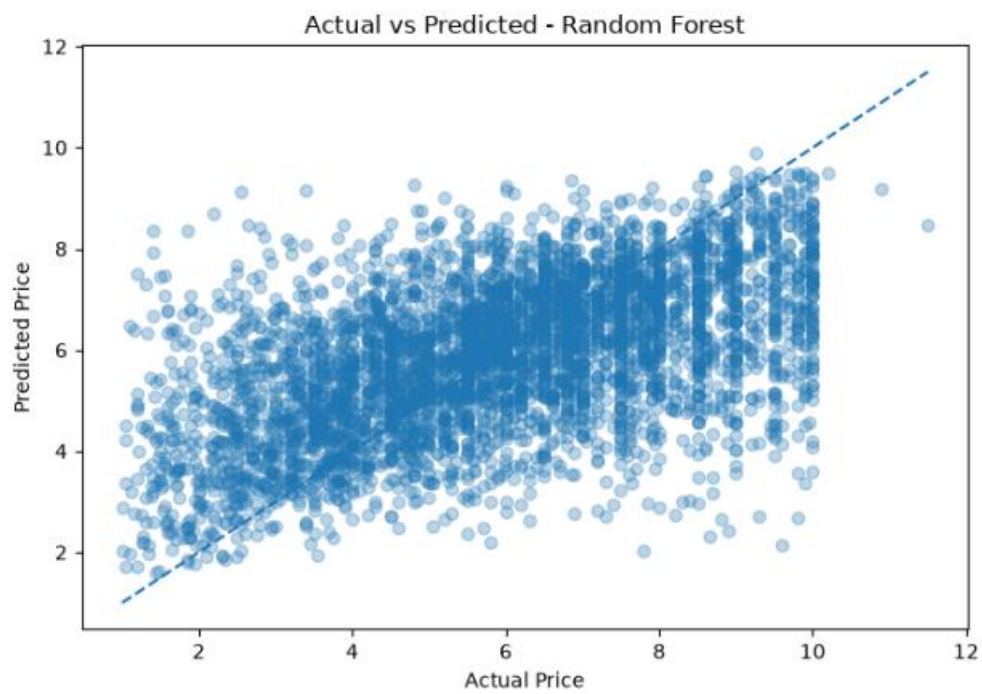
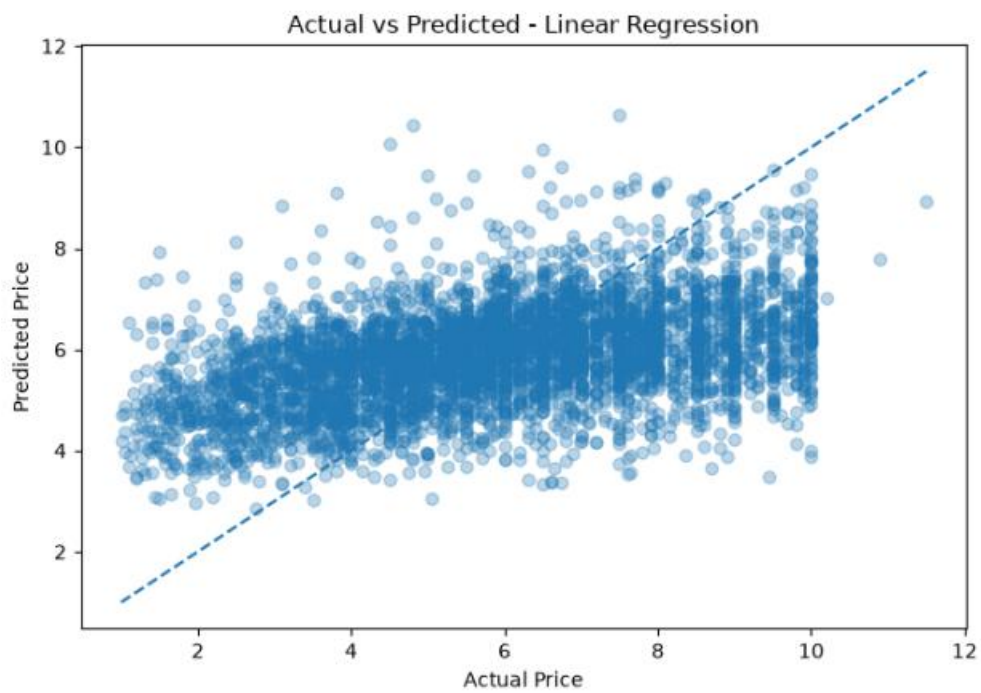
    max_value = max(
        y_test.max(),
        np.max(y_pred)
    )

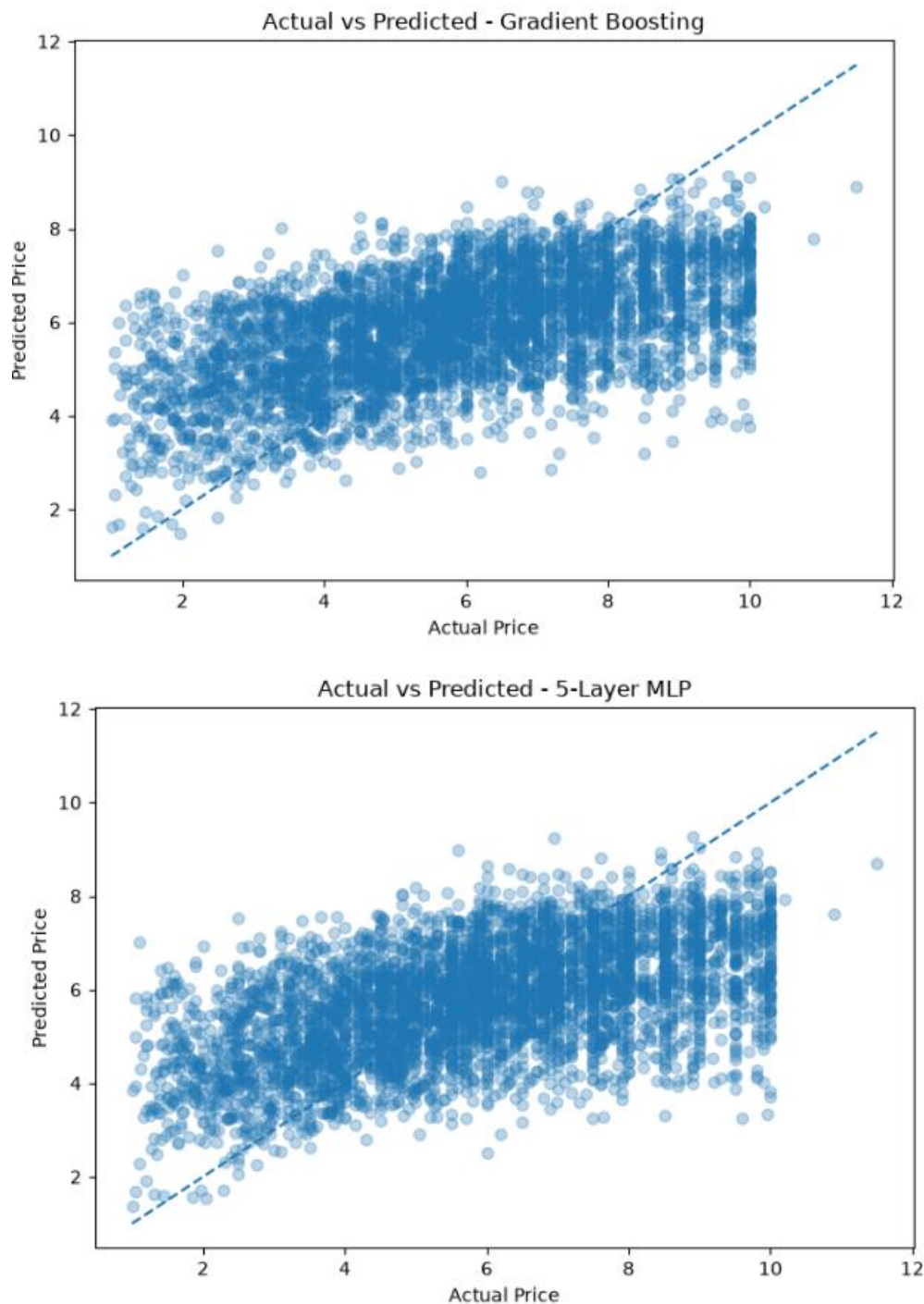
    plt.plot(
        [min_value, max_value],
        [min_value, max_value],
        linestyle="--"
    )

    plt.xlabel("Actual Price")
    plt.ylabel("Predicted Price")

    plt.title(
        f"Actual vs Predicted - {model_name}"
    )

    plt.tight_layout()
    plt.show()
```





Hình 30. Actual Price so với Predicted Price của 4 model

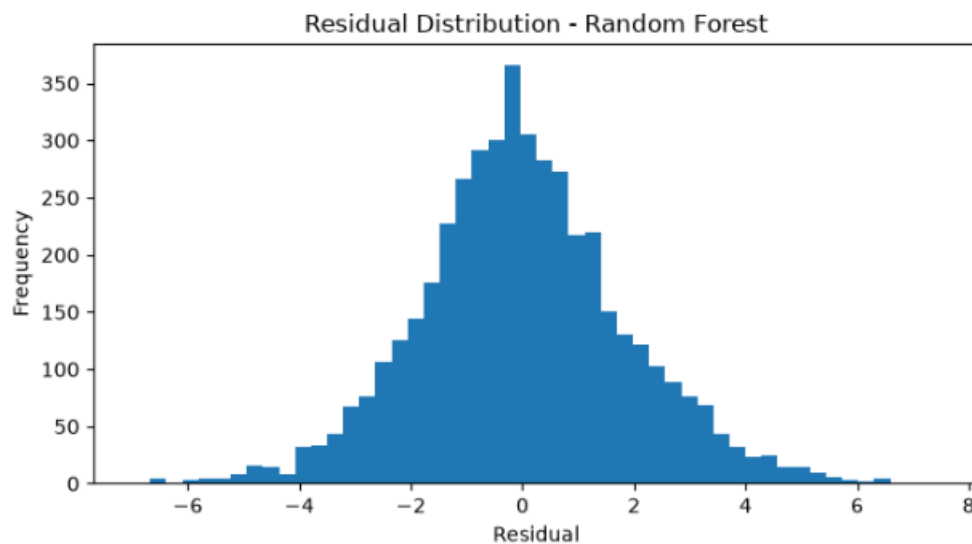
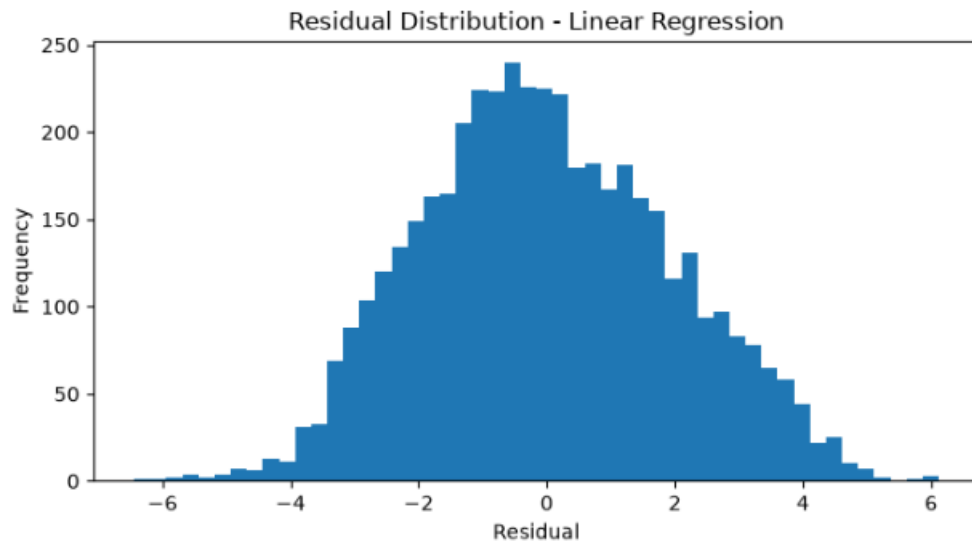
Giải thích: Trong bốn mô hình, **Gradient Boosting** có mức bám sát đường lý tưởng tốt nhất về tổng thể, phù hợp với việc mô hình này đạt **RMSE thấp nhất (1,8067)** và **R² cao nhất (0,3181)** trên tập Test. **5-Layer MLP** đứng thứ hai theo RMSE và R², trong khi **Random Forest** có MAE thấp nhất nhưng phân tán lỗi lớn hơn Gradient Boosting.

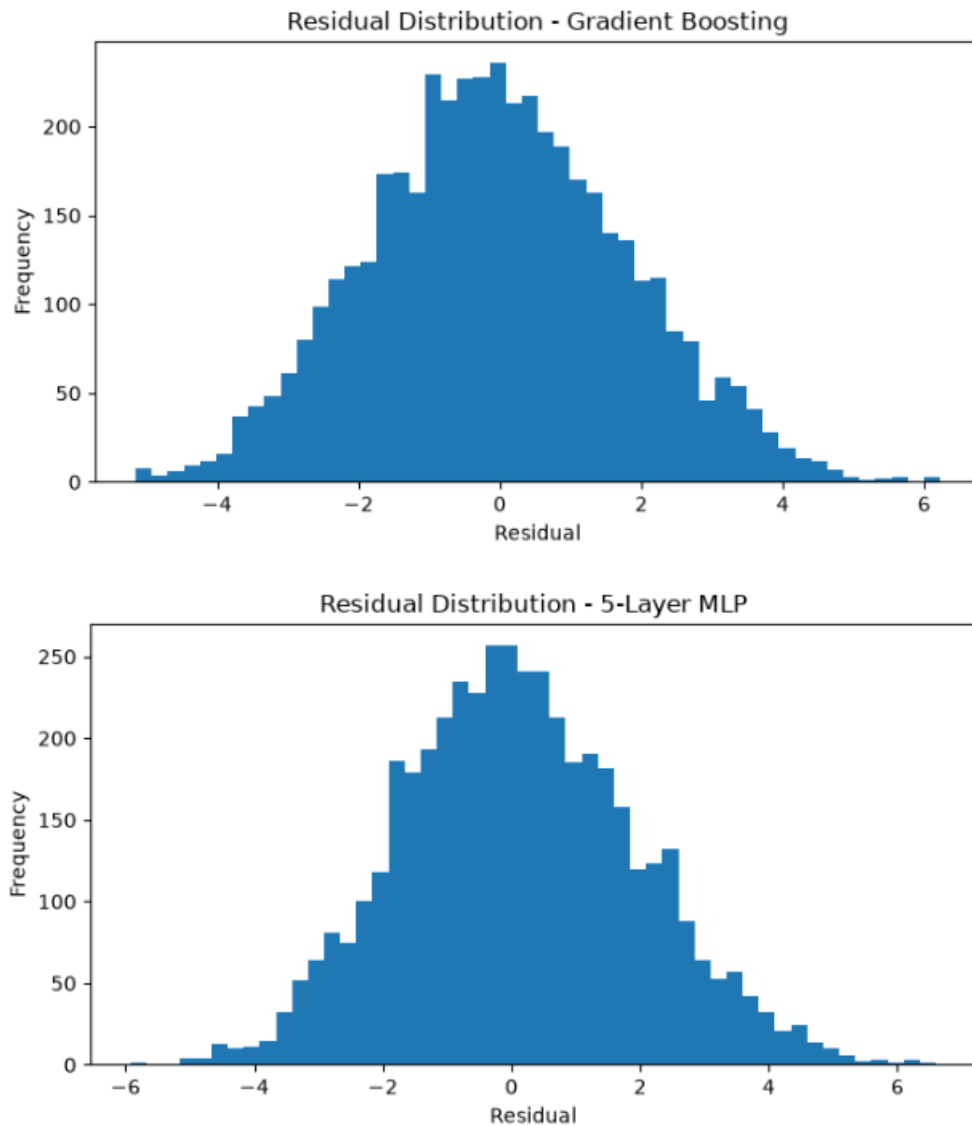
Linear Regression cho kết quả yếu nhất, cho thấy mối quan hệ giữa các đặc trưng và giá nhà không hoàn toàn tuyến tính.

11.2. Residual Analysis

$$e_i = y_i - \hat{y}_i$$

Residual gần 0 là mong muốn. Residual dương nghĩa là model dự đoán thấp hơn giá thực tế, residual âm nghĩa là model dự đoán cao hơn. Histogram residual được dùng để kiểm tra độ phân tán lỗi, bias và sự tồn tại của các lỗi lớn.





Hình 31. Phân bố residual của 4 model

12. KHẢ NĂNG DIỄN GIẢI VÀ FEATURE IMPORTANCE

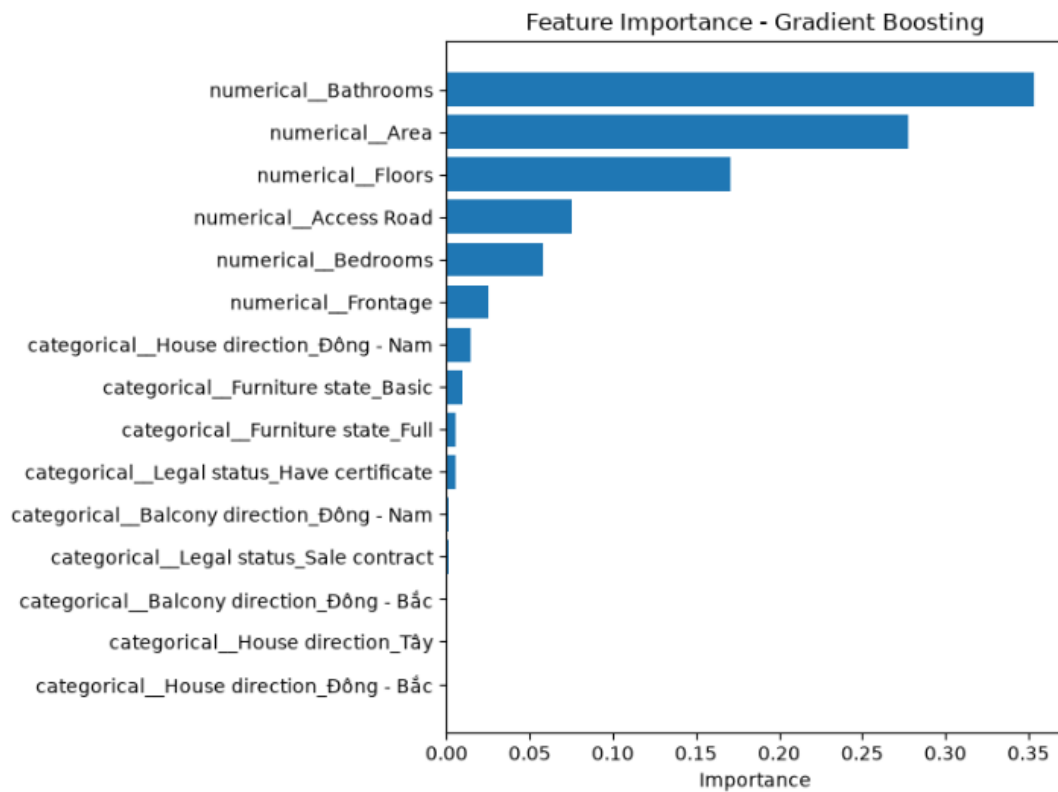
Random Forest và Gradient Boosting cung cấp `feature_importances_`. Kết quả cho biết model tree-based sử dụng những dimension nào nhiều hơn khi tạo prediction. Tuy nhiên, feature importance không phải bằng chứng quan hệ nhân quả giữa feature và giá nhà.

Với Gradient Boosting, Bathrooms có importance cao nhất (0,3538), tiếp theo là Area (0,2779), Floors (0,1708), Access Road (0,0754), Bedrooms (0,0583) và Frontage

(0,0249). Điều này cho thấy các đặc trưng vật lý của bất động sản đóng vai trò lớn trong mô hình hiện tại.

Top features - Gradient Boosting

	Feature	Importance
5	numerical__Bathrooms	0.353773
0	numerical__Area	0.277857
3	numerical__Floors	0.170798
2	numerical__Access Road	0.075428
4	numerical__Bedrooms	0.058300
1	numerical__Frontage	0.024928
13	categorical__House direction_Đông - Nam	0.014754
24	categorical__Furniture state_Basic	0.009560
25	categorical__Furniture state_Full	0.005807
22	categorical__Legal status_Have certificate	0.005196
21	categorical__Balcony direction_Đông - Nam	0.001176
23	categorical__Legal status_Sale contract	0.000917
20	categorical__Balcony direction_Đông - Bắc	0.000392
8	categorical__House direction_Tây	0.000389
12	categorical__House direction_Đông - Bắc	0.000251



Hình 32. Feature Importance của Gradient Boosting.

13. REPRESENTATION LEARNING CỦA DEEP LEARNING

13.1. Hidden Representation H4

Một điểm trung tâm của Assignment 03 là mạng Neural Network không chỉ học mapping từ input sang prediction mà còn học các representation trung gian. Với HousePriceMLP, input đã preprocessing có 26 chiều và representation H4 ngay trước output chỉ còn 16 chiều.

$$X(26D) \rightarrow H_1(128D) \rightarrow H_2(64D) \rightarrow H_3(32D) \rightarrow H_4(16D) \rightarrow \hat{y}$$

H4 không phải 16 feature được lập trình viên định nghĩa thủ công. Chúng là các đặc trưng được hình thành từ weights/biases qua quá trình giảm loss. Vì vậy, hidden layer có thể được xem như một learned transformation của prepared input representation.

```
Processed input shape: torch.Size([4535, 26])
Learned H4 shape: (4535, 16)
```

13.2. PCA visualization của learned representation

Để trực quan hóa H4, notebook sử dụng PCA giảm representation 16 chiều xuống 2 chiều và tô màu theo Price. PCA chỉ phục vụ visualization, không tham gia training. Nếu màu Price thể hiện gradient hoặc cấu trúc rõ trong không gian H4, đó là tín hiệu định tính cho thấy mạng đã tổ chức lại input theo hướng hữu ích cho bài toán dự đoán.

Không nên kết luận rằng H4 “hiểu giá nhà” hay tạo ra các feature có ý nghĩa kinh tế cụ thể nếu chưa có phân tích bổ sung. Kết quả PCA chỉ cho thấy cấu trúc của learned space sau phép chiếu xuống 2 chiều.



Hình 33. PCA của learned representation H4, tô màu theo Price.

Giải thích: Biểu đồ PCA trực quan hóa learned representation H4 của mô hình Deep Learning, trong đó mỗi điểm biểu diễn một bất động sản và màu sắc thể hiện mức giá. Có thể quan sát xu hướng giá thấp tập trung nhiều hơn ở phía PC1 âm, trong khi các mức giá cao xuất hiện nhiều hơn về phía PC1 dương. Điều này cho thấy mạng đã học được representation có liên quan đến giá nhà, tuy nhiên các mức giá vẫn còn chồng lấn đáng kể nên chưa được phân tách hoàn toàn.

15. KẾT LUẬN

Hệ thống House Price Prediction đã so sánh ba mô hình Machine Learning gồm Linear Regression, Random Forest Regressor và Gradient Boosting Regressor với một mạng MLP 5 lớp. Tất cả mô hình sử dụng cùng nguyên tắc chia Train/Validation/Test và preprocessing chỉ được fit trên dữ liệu huấn luyện để tránh leakage.

Kết quả Test cho thấy Gradient Boosting có RMSE thấp nhất 1,8067 và R^2 cao nhất 0,3181. 5-Layer MLP đứng thứ hai với RMSE 1,8367 và R^2 0,2953 nhưng cần thời gian huấn luyện cao hơn đáng kể. Random Forest có MAE thấp nhất 1,4293 nhưng không vượt Gradient Boosting ở RMSE và R^2 .

Deep Learning đã biến đổi input representation 26 chiều qua các hidden layers $128 \rightarrow 64 \rightarrow 32 \rightarrow 16$ trước khi tạo prediction. Phân tích H4 bằng PCA minh họa được trọng tâm Representation Learning của Assignment 03: representation trung gian được học từ data thay vì được định nghĩa thủ công.

Từ góc độ lựa chọn model, kết quả hiện tại ủng hộ Gradient Boosting cho hệ thống dự đoán giá nhà vì hiệu năng Test tốt nhất và chi phí train thấp hơn MLP. Tuy nhiên, R^2 chỉ khoảng 0,318 cho thấy dataset/feature hiện tại chưa giải thích đầy đủ biến thiên của thị trường; các hướng cải thiện có thể gồm khai thác location information tốt hơn, feature engineering và controlled experiments về architecture/optimizer/hyperparameters.

16. LƯU MÔ HÌNH VÀ TÁI SỬ DỤNG

Notebook lưu fitted preprocessor, ba ML models, state_dict của MLP, metadata và file CSV chứa bảng so sánh. Việc lưu preprocessor cùng model giúp đảm bảo inference sử dụng đúng transformations đã dùng khi training.

Artifact	Vai trò
preprocessor.joblib	Median imputation + StandardScaler + One-Hot Encoder
linear_regression.joblib	Linear Regression
random_forest.joblib	Random Forest Regressor
gradient_boosting.joblib	Gradient Boosting Regressor
house_price_5_layer_mlp.pth	PyTorch state_dict và architecture metadata

metadata.joblib	Target/features/config metadata
house_price_model_comparison.csv	Final metrics comparison

III. Hệ thống dự đoán hành vi khách hàng

1. DATASET VÀ MÔ TẢ BÀI TOÁN

1.1. Tên và nguồn Dataset

Dataset được sử dụng là Women's E-Commerce Clothing Reviews. Trong notebook, trường văn bản Review Text được dùng làm đầu vào và Recommended IND được dùng làm biến mục tiêu cho bài toán phân loại nhị phân.

Nguồn Dataset: <https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>

1.2. Định nghĩa bài toán

Mục tiêu của hệ thống là dự đoán mức độ quan tâm / ý định giới thiệu sản phẩm thể hiện qua nội dung review của khách hàng. Bài toán được định nghĩa theo dạng supervised binary classification.

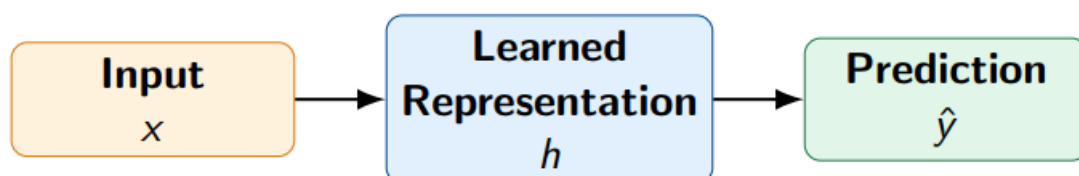
$X = \text{Review Text}$

$y = \text{Recommended IND} \in \{0, 1\}$

Trong đó, Recommended IND = 0 được diễn giải là Not Recommended và Recommended IND = 1 được diễn giải là Recommended / Positive Interest.

Cần nhấn mạnh phạm vi diễn giải: Recommended IND được sử dụng như một đại diện vận hành (operational proxy) cho recommendation intention hoặc positive interest. Mô hình không trực tiếp đo toàn bộ hành vi mua hàng, customer lifetime value hay hành vi dài hạn của một khách hàng.

Ba mô hình Machine Learning truyền thống được so sánh với một mạng Deep Learning 5 lớp gồm: Logistic Regression, Multinomial Naive Bayes, Linear SVM và 5-Layer Multi-Layer Perceptron (MLP).



Consider

$$h_1 = \text{ReLU}(W_1 x + b_1).$$

The vector h_1 is a learned representation.

$$x \in \mathbb{R}^d \longrightarrow h_1 \in \mathbb{R}^{64}.$$

2. PHÂN TÍCH DỮ LIỆU VÀ LÀM SẠCH

2.1. Cấu trúc dữ liệu và chất lượng dữ liệu

Trước khi huấn luyện, cần kiểm tra kích thước dữ liệu, tên cột, kiểu dữ liệu, missing values, số lượng giá trị khác nhau và duplicate rows. Phần này giúp xác nhận dữ liệu đầu vào đúng định dạng và phát hiện các vấn đề có thể làm sai quá trình học.

```
Columns:
['Unnamed: 0', 'Clothing ID', 'Age', 'Title', 'Review Text', 'Rating', 'Recommended IND', 'Positive Feedback Count', 'Division Name', 'Department Name', 'Class Name']
```

```
Shape: (23486, 11)
```

```
Info:
```

```
<class 'pandas.DataFrame'>
RangeIndex: 23486 entries, 0 to 23485
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	23486 non-null	int64
1	Clothing ID	23486 non-null	int64
2	Age	23486 non-null	int64
3	Title	19676 non-null	str
4	Review Text	22641 non-null	str
5	Rating	23486 non-null	int64
6	Recommended IND	23486 non-null	int64
7	Positive Feedback Count	23486 non-null	int64
8	Division Name	23472 non-null	str
9	Department Name	23472 non-null	str
10	Class Name	23472 non-null	str

```
dtypes: int64(6), str(5)
memory usage: 9.5 MB
```

	dtype	missing_count	missing_pct	unique_values
Unnamed: 0	int64	0	0.00	23486
Clothing ID	int64	0	0.00	1206
Age	int64	0	0.00	77
Title	str	3810	16.22	13993
Review Text	str	845	3.60	22634
Rating	int64	0	0.00	5
Recommended IND	int64	0	0.00	2
Positive Feedback Count	int64	0	0.00	82
Division Name	str	14	0.06	3
Department Name	str	14	0.06	6
Class Name	str	14	0.06	20

```
Duplicated rows: 0
```

2.2. Quy trình làm sạch dữ liệu

Notebook thực hiện bốn bước làm sạch chính: loại cột dạng index như Unnamed nếu tồn tại, loại exact duplicate rows, ép Recommended IND về kiểu số và chỉ giữ các giá trị hợp lệ 0/1, sau đó loại những observation có Review Text bị thiếu hoặc rỗng.

Quy trình này giữ đúng supervised task đã chốt là Review Text → Recommended IND. Rating không được dùng làm target, mặc dù Rating vẫn có thể được sử dụng trong EDA để hỗ trợ phân tích hành vi.

```
Removed index-like columns: ['Unnamed: 0']  
Removed duplicates: 21  
Removed invalid target rows: 0  
Removed missing/empty reviews: 825  
Final cleaned shape: (22640, 10)
```

2.3. Phân bố lớp mục tiêu

Phân bố lớp Recommended IND cần được kiểm tra trước khi đánh giá model. Sau khi chạy notebook, hãy ghi số lượng và tỷ lệ của hai lớp Not Recommended và Recommended theo output.

Nếu hai lớp chênh lệch đáng kể, Accuracy không nên được sử dụng một mình. Khi đó Precision, Recall, F1-score, ROC-AUC và Confusion Matrix cần được đọc đồng thời để tránh kết luận sai do class imbalance.

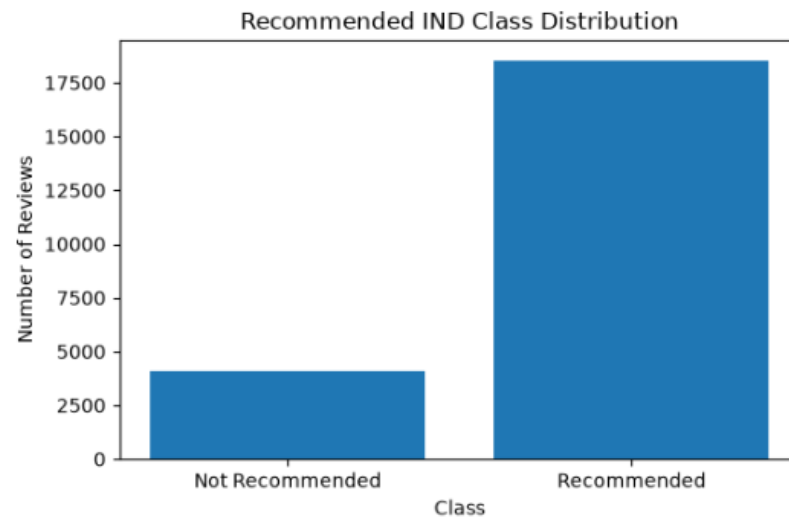
```

class_counts = y.value_counts().sort_index()

plt.figure(figsize=(6, 4))
plt.bar(
    ["Not Recommended", "Recommended"],
    class_counts.values
)
plt.xlabel("Class")
plt.ylabel("Number of Reviews")
plt.title("Recommended IND Class Distribution")
plt.tight_layout()
plt.show()

class_ratio = y.value_counts(normalize=True).sort_index() * 100
display(class_ratio.round(2).to_frame(name="Percentage"))

```



Recommended IND	
0	18.11
1	81.89

Hình 34. Phân bố hai lớp Recommended và Not Recommended.

2.4. Phân bố độ dài review

Độ dài review được tính theo số từ. Phân tích này giúp hiểu mức độ đa dạng của dữ liệu văn bản, phát hiện review rất ngắn hoặc rất dài và giải thích tại sao cần một representation cố định chiều như TF-IDF.

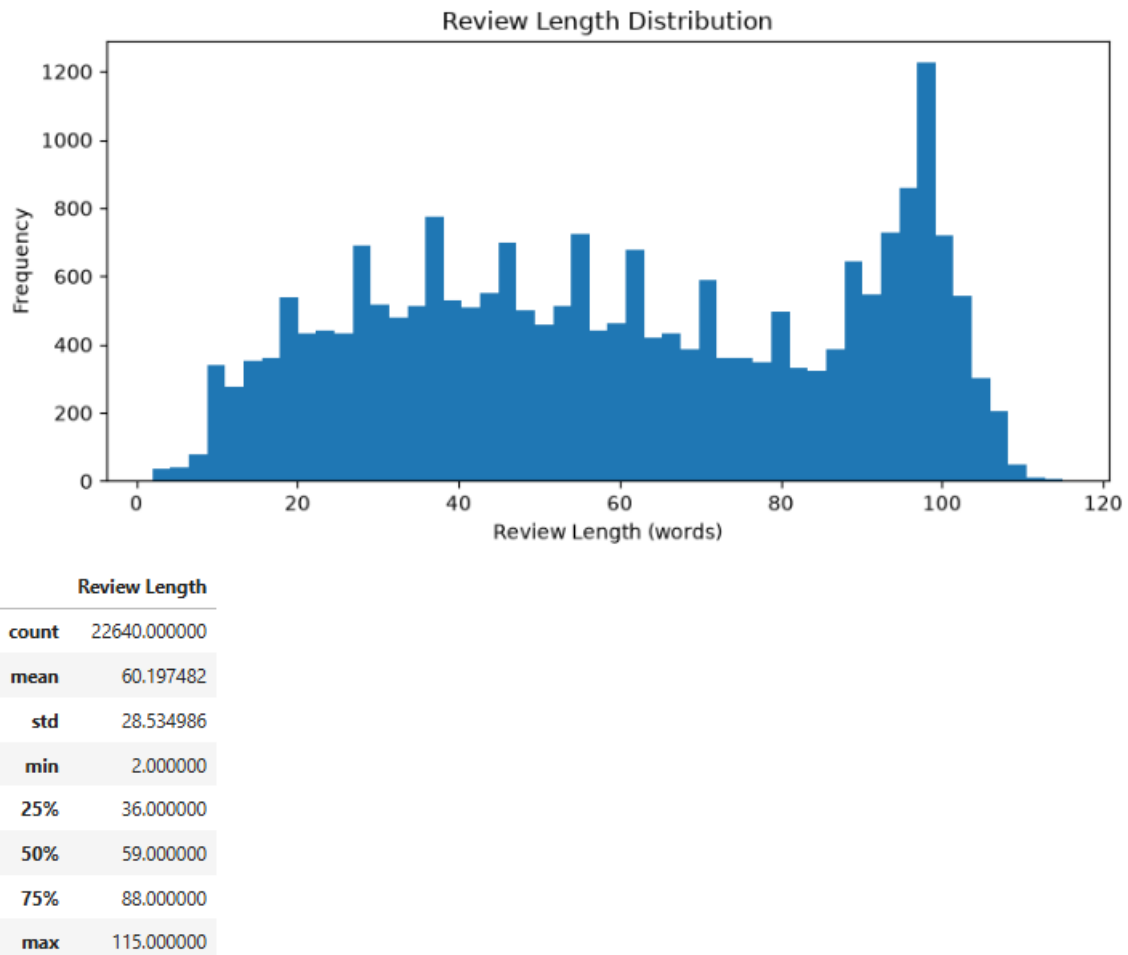

```

review_length_words = X_text.str.split().str.len()

plt.figure(figsize=(8, 4))
plt.hist(review_length_words, bins=50)
plt.xlabel("Review Length (words)")
plt.ylabel("Frequency")
plt.title("Review Length Distribution")
plt.tight_layout()
plt.show()

display(review_length_words.describe().to_frame(name="Review Length"))

```



Hình 35. Phân bố độ dài nội dung review.

2.5. Rating và recommendation intention

Nếu dataset có cột Rating, notebook trực quan hóa phân bố Rating và tỷ lệ Recommended theo từng mức Rating. Đây là EDA bổ sung để xem mức rating và recommendation intention có xu hướng đồng biến hay không.

Phân tích này chỉ mang tính mô tả. Vì supervised task sử dụng Review Text làm input, Rating không được đưa vào mô hình trong notebook hiện tại.

```

if "Rating" in df_clean.columns:
    rating_counts = df_clean["Rating"].value_counts().sort_index()

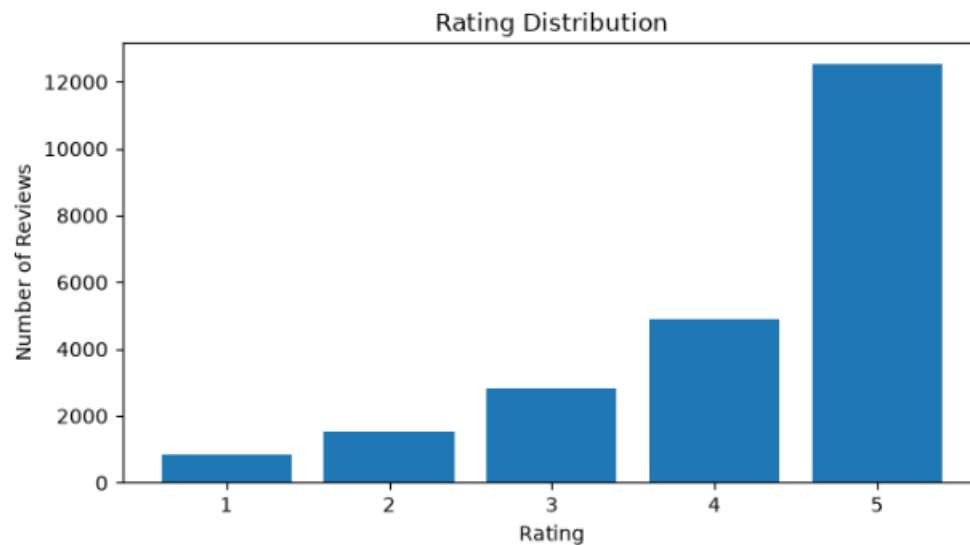
    plt.figure(figsize=(7, 4))
    plt.bar(rating_counts.index.astype(str), rating_counts.values)
    plt.xlabel("Rating")
    plt.ylabel("Number of Reviews")
    plt.title("Rating Distribution")
    plt.tight_layout()
    plt.show()

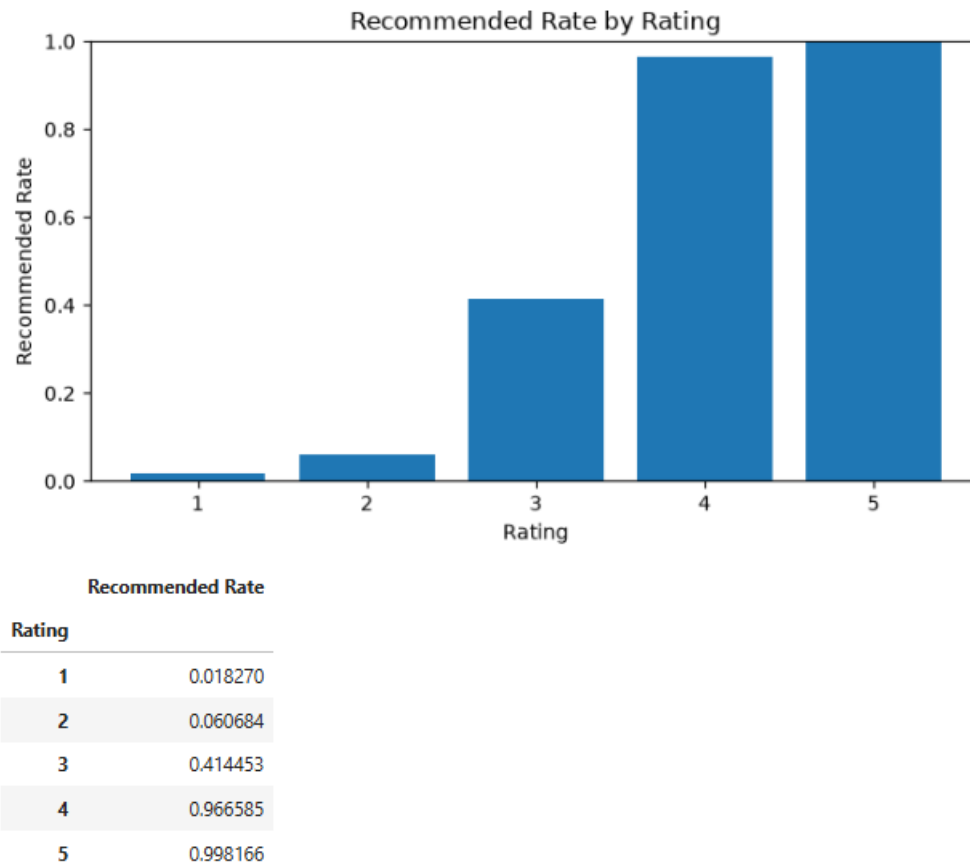
    rating_interest = df_clean.groupby("Rating")[TARGET].mean()

    plt.figure(figsize=(7, 4))
    plt.bar(rating_interest.index.astype(str), rating_interest.values)
    plt.ylim(0, 1)
    plt.xlabel("Rating")
    plt.ylabel("Recommended Rate")
    plt.title("Recommended Rate by Rating")
    plt.tight_layout()
    plt.show()

    display(rating_interest.to_frame(name="Recommended Rate"))

```





Hình 36. Phân bố Rating và tỷ lệ Recommended theo từng mức Rating.

Giải thích: Biểu đồ cho thấy Rating có mối quan hệ rất mạnh với hành vi recommendation. Khi Rating tăng, tỷ lệ khách hàng đề xuất sản phẩm cũng tăng rõ rệt. Gần như toàn bộ review có Rating 4–5 đều thuộc nhóm Recommended, trong khi Rating 1–2 chủ yếu thuộc nhóm Not Recommended. Điều này cho thấy mức độ hài lòng của khách hàng có liên hệ chặt chẽ với ý định giới thiệu sản phẩm.

3. THIẾT KẾ THÍ NGHIỆM VÀ BIỂU DIỄN VĂN BẢN

3.1. Chia Train / Validation / Test

Dữ liệu được chia thành khoảng 70% Train, 15% Validation và 15% Test. Do đây là classification, stratify=y được sử dụng để giữ tỷ lệ hai lớp gần tương đương giữa ba tập.

Training Set dùng để học tham số. Validation Set dùng để theo dõi quá trình phát triển và lựa chọn best epoch của mạng. Test Set được giữ riêng cho final evaluation nhằm đánh giá generalization trên dữ liệu chưa thấy.

```

X_train_text, X_temp_text, y_train, y_temp = train_test_split(
    X_text,
    y,
    test_size=0.30,
    random_state=RANDOM_STATE,
    stratify=y
)

X_val_text, X_test_text, y_val, y_test = train_test_split(
    X_temp_text,
    y_temp,
    test_size=0.50,
    random_state=RANDOM_STATE,
    stratify=y_temp
)

split_summary = pd.DataFrame({
    "Set": ["Train", "Validation", "Test"],
    "Rows": [len(X_train_text), len(X_val_text), len(X_test_text)],
    "Recommended %": [
        y_train.mean() * 100,
        y_val.mean() * 100,
        y_test.mean() * 100
    ]
})

display(split_summary.round(2))

```

	Set	Rows	Recommended %
0	Train	15848	81.88
1	Validation	3396	81.89
2	Test	3396	81.89

Hình 37. Phân chia Train, Validation và Test.

3.2. Biểu diễn văn bản bằng TF-IDF

Các mô hình không thể học trực tiếp từ chuỗi ký tự thô. Vì vậy Review Text được chuyển thành vector số bằng TF-IDF. Vectorizer được fit trên Training Set, sau đó chỉ transform Validation và Test để tránh data leakage.

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \times \log(N / \text{DF}(t))$$

Trong notebook, TfidfVectorizer dùng lowercase, stop_words='english', ngram_range=(1,2), min_df=2, max_df=0.95, sublinear_tf=True và max_features=1500. Vì vậy mỗi review được chuyển thành một vector có tối đa 1.500 chiều.

Điểm quan trọng của Assignment 3 là phân biệt prepared representation và learned representation. TF-IDF là representation được chuẩn bị trước; các hidden layer của MLP tiếp tục học những representation mới H1, H2, H3 và H4.

```

MAX_FEATURES = 1500

def build_vectorizer():
    return TfidfVectorizer(
        lowercase=True,
        stop_words="english",
        max_features=MAX_FEATURES,
        ngram_range=(1, 2),
        min_df=2,
        max_df=0.95,
        sublinear_tf=True,
        dtype=np.float32
    )

vectorizer = build_vectorizer()

X_train_tfidf = vectorizer.fit_transform(X_train_text)
X_val_tfidf = vectorizer.transform(X_val_text)
X_test_tfidf = vectorizer.transform(X_test_text)

feature_names = vectorizer.get_feature_names_out()

print("Train TF-IDF shape      :", X_train_tfidf.shape)
print("Validation TF-IDF shape:", X_val_tfidf.shape)
print("Test TF-IDF shape       :", X_test_tfidf.shape)
print("Vocabulary size        :", len(feature_names))

Train TF-IDF shape      : (15848, 1500)
Validation TF-IDF shape: (3396, 1500)
Test TF-IDF shape       : (3396, 1500)
Vocabulary size         : 1500

```

4. CÁC METRIC ĐÁNH GIÁ

Positive class trong thí nghiệm là Recommended = 1. Cả bốn mô hình được đánh giá bằng cùng một nhóm metric để đảm bảo so sánh công bằng.

Metric	Công thức / cách hiểu	Ý nghĩa trong bài toán
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Tỷ lệ dự đoán đúng trên toàn bộ review.
Precision	$TP / (TP + FP)$	Trong các review model dự đoán Recommended, có bao nhiêu thực sự Recommended.
Recall	$TP / (TP + FN)$	Trong các review thực sự Recommended, model phát hiện được bao nhiêu.
F1-score	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	Cân bằng Precision và Recall.
ROC-AUC	Diện tích dưới ROC Curve	Khả năng phân biệt/xếp hạng hai lớp trên nhiều threshold.

Nếu target bị lệch lớp, F1 và ROC-AUC thường mang nhiều thông tin hơn Accuracy đơn thuần. Khi viết nhận xét, không nên chọn model chỉ dựa trên một metric.

5. BA MÔ HÌNH MACHINE LEARNING TRUYỀN THỐNG

5.1. Logistic Regression

Logistic Regression được dùng làm baseline tuyến tính. Mô hình học trọng số cho các TF-IDF dimensions và đưa ra score/xác suất cho lớp Recommended. Một ưu điểm lớn là các coefficient có thể được dùng để phân tích những term nghiêng về Recommended hoặc Not Recommended.

5.2. Multinomial Naive Bayes

Multinomial Naive Bayes là mô hình xác suất thường phù hợp với biểu diễn từ dạng non-negative như TF-IDF. Mô hình dựa trên giả định độc lập có điều kiện giữa các feature khi biết class, nhờ đó có tốc độ huấn luyện nhanh và là baseline hữu ích cho text classification.

5.3. Linear SVM

Linear SVM tìm một siêu phẳng phân tách hai lớp với margin lớn trong không gian TF-IDF nhiều chiều. Đây là lựa chọn phổ biến cho bài toán văn bản có số feature lớn.

5.4. So sánh ba mô hình ML trên Validation Set

Ba mô hình được huấn luyện trên cùng Training Set và đánh giá trên cùng Validation Set. Kết quả sau khi chạy notebook cần được so sánh theo Accuracy, Precision, Recall, F1, ROC-AUC và Training Time.

	Model	Accuracy	Precision	Recall	F1	ROC_AUC	Training Time (s)
0	Logistic Regression	0.8816	0.9006	0.9615	0.9301	0.9237	0.1898
1	Linear SVM	0.8763	0.9081	0.9446	0.9260	0.9119	0.1189
2	Multinomial Naive Bayes	0.8622	0.8663	0.9835	0.9212	0.9176	0.0094

Logistic Regression cho hiệu năng tổng thể tốt nhất với Accuracy = **0,8816**, F1-score = **0,9301** và ROC-AUC = **0,9237**, đều cao nhất trong ba mô hình. **Linear SVM** đạt Precision cao nhất (**0,9081**), cho thấy khi mô hình dự đoán một review thuộc nhóm Recommended thì độ chính xác khá cao. Trong khi đó, **Multinomial Naive Bayes** đạt Recall cao nhất (**0,9835**) và có thời gian huấn luyện nhanh nhất (**0,0094 giây**), nghĩa là mô hình phát hiện gần như toàn bộ các review thực sự thuộc nhóm Recommended nhưng có Precision và Accuracy thấp hơn hai mô hình còn lại.

6. MÔ HÌNH DEEP LEARNING 5 LỚP

6.1. Kiến trúc mạng

Mạng Deep Learning được xây bằng PyTorch dưới dạng Multi-Layer Perceptron gồm 5 lớp Linear học tham số. Số chiều đầu vào d được lấy từ TF-IDF representation.

$$d \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1$$

Bốn lớp đầu sử dụng ReLU. Lớp cuối trả về một logit. Khi cần chuyển logit thành xác suất, Sigmoid được áp dụng ngoài model hoặc được kết hợp bên trong BCEWithLogitsLoss.

```
class CustomerInterestMLP(nn.Module):

    def __init__(self, input_dim):
        super().__init__()

        self.fc1 = nn.Linear(input_dim, 256) # Layer 1
        self.fc2 = nn.Linear(256, 128)       # Layer 2
        self.fc3 = nn.Linear(128, 64)        # Layer 3
        self.fc4 = nn.Linear(64, 32)         # Layer 4
        self.fc5 = nn.Linear(32, 1)          # Layer 5

        self.relu = nn.ReLU()

    def forward(self, x):
        h1 = self.relu(self.fc1(x))
        h2 = self.relu(self.fc2(h1))
        h3 = self.relu(self.fc3(h2))
        h4 = self.relu(self.fc4(h3))

        # Return Logit
        z5 = self.fc5(h4)

        return z5

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

INPUT_DIM = X_train_dense.shape[1]

dl_model = CustomerInterestMLP(
    INPUT_DIM
).to(device)

print("Device :", device)
print("Input dim:", INPUT_DIM)
print(dl_model)

Device : cpu
Input dim: 1500
CustomerInterestMLP(
  (fc1): Linear(in_features=1500, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=64, bias=True)
  (fc4): Linear(in_features=64, out_features=32, bias=True)
  (fc5): Linear(in_features=32, out_features=1, bias=True)
  (relu): ReLU()
)
```

Hình 38. Kiến trúc mạng MLP 5 lớp và số tham số học được.

6.2. Forward Propagation và learned representation

Forward propagation biến đổi prepared TF-IDF representation qua nhiều tầng. Mỗi hidden layer tạo ra một representation mới được học từ dữ liệu.

$$\mathbf{H}_1 = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)$$

$$\mathbf{H}_2 = \text{ReLU}(\mathbf{H}_1\mathbf{W}_2 + \mathbf{b}_2)$$

$$\mathbf{H}_3 = \text{ReLU}(\mathbf{H}_2\mathbf{W}_3 + \mathbf{b}_3)$$

$$\mathbf{H}_4 = \text{ReLU}(\mathbf{H}_3\mathbf{W}_4 + \mathbf{b}_4)$$

$$\mathbf{Z}_5 = \mathbf{H}_4\mathbf{W}_5 + \mathbf{b}_5$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{Z}_5) = 1 / (1 + e^{(-\mathbf{Z}_5)})$$

$$\boxed{x \rightarrow h_{\theta}(x) \rightarrow \hat{y}}$$

Các vector $\mathbf{H}_1$, $\mathbf{H}_2$, $\mathbf{H}_3$ và $\mathbf{H}_4$ không được lập trình viên gán ý nghĩa thủ công. Chúng là learned representations được tối ưu để phục vụ nhiệm vụ dự đoán Recommended IND.

6.3. Loss Function

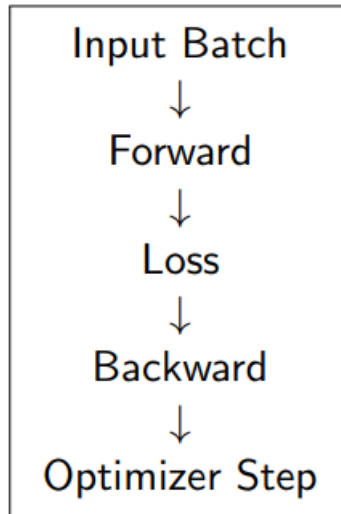
Vì đây là binary classification, notebook sử dụng Binary Cross-Entropy. Trong PyTorch, BCEWithLogitsLoss được chọn vì nó kết hợp Sigmoid và Binary Cross-Entropy theo cách ổn định số học.

$$\mathcal{L} = \text{CrossEntropyLoss}(z, y)$$

6.4. Backpropagation và Optimizer

Backpropagation tính gradient của loss theo từng weight và bias. Adam optimizer sử dụng các gradient này để cập nhật tham số. Chu trình học của một mini-batch là Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Optimizer Step.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}.$$



6.5. Tensor, DataLoader và cấu hình huấn luyện

TF-IDF sparse matrix được chuyển thành dense float32 tensor để đưa vào MLP.

Notebook sử dụng batch size 128. Với batch B, tensor đi qua network theo dạng:

$$\mathbf{B} \times \mathbf{d} \rightarrow \mathbf{B} \times 256 \rightarrow \mathbf{B} \times 128 \rightarrow \mathbf{B} \times 64 \rightarrow \mathbf{B} \times 32 \rightarrow \mathbf{B} \times 1.$$

Cấu hình training trong notebook gồm learning rate = 0,001, tối đa 80 epochs, patience = 12, BCEWithLogitsLoss và Adam optimizer. Best model được chọn theo validation loss.

```
X_train_dense = X_train_tfidf.toarray().astype(np.float32)
X_val_dense = X_val_tfidf.toarray().astype(np.float32)
X_test_dense = X_test_tfidf.toarray().astype(np.float32)

X_train_tensor = torch.tensor(X_train_dense, dtype=torch.float32)
y_train_tensor = torch.tensor(
    y_train.to_numpy().reshape(-1, 1),
    dtype=torch.float32
)

X_val_tensor = torch.tensor(X_val_dense, dtype=torch.float32)
y_val_tensor = torch.tensor(
    y_val.to_numpy().reshape(-1, 1),
    dtype=torch.float32
)

X_test_tensor = torch.tensor(X_test_dense, dtype=torch.float32)
y_test_tensor = torch.tensor(
    y_test.to_numpy().reshape(-1, 1),
    dtype=torch.float32
)

print("X_train_tensor:", X_train_tensor.shape)
print("y_train_tensor:", y_train_tensor.shape)

X_train_tensor: torch.Size([15848, 1500])
y_train_tensor: torch.Size([15848, 1])
```

```

BATCH_SIZE = 128

train_loader = DataLoader(
    TensorDataset(X_train_tensor, y_train_tensor),
    batch_size=BATCH_SIZE,
    shuffle=True
)

val_loader = DataLoader(
    TensorDataset(X_val_tensor, y_val_tensor),
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    TensorDataset(X_test_tensor, y_test_tensor),
    batch_size=BATCH_SIZE,
    shuffle=False
)

X_batch, y_batch = next(iter(train_loader))

print("One X batch:", X_batch.shape)
print("One y batch:", y_batch.shape)

```

```

One X batch: torch.Size([128, 1500])
One y batch: torch.Size([128, 1])

```

```

LEARNING_RATE = 0.001
NUM_EPOCHS = 80
PATIENCE = 12

criterion = nn.BCEWithLogitsLoss()

optimizer = torch.optim.Adam(
    dl_model.parameters(),
    lr=LEARNING_RATE
)

print("Loss      :", criterion)
print("Optimizer  :", optimizer.__class__.__name__)
print("Learning rate:", LEARNING_RATE)
print("Epochs     :", NUM_EPOCHS)
print("Batch size  :", BATCH_SIZE)
print("Patience   :", PATIENCE)

```

```

Loss      : BCEWithLogitsLoss()
Optimizer : Adam
Learning rate: 0.001
Epochs    : 80
Batch size : 128
Patience  : 12

```

6.6. Training / Validation Curves

Notebook theo dõi Training Loss, Validation Loss, Training Accuracy, Validation Accuracy, Training F1 và Validation F1. Các đường cong này giúp đánh giá underfitting, good fit và overfitting.

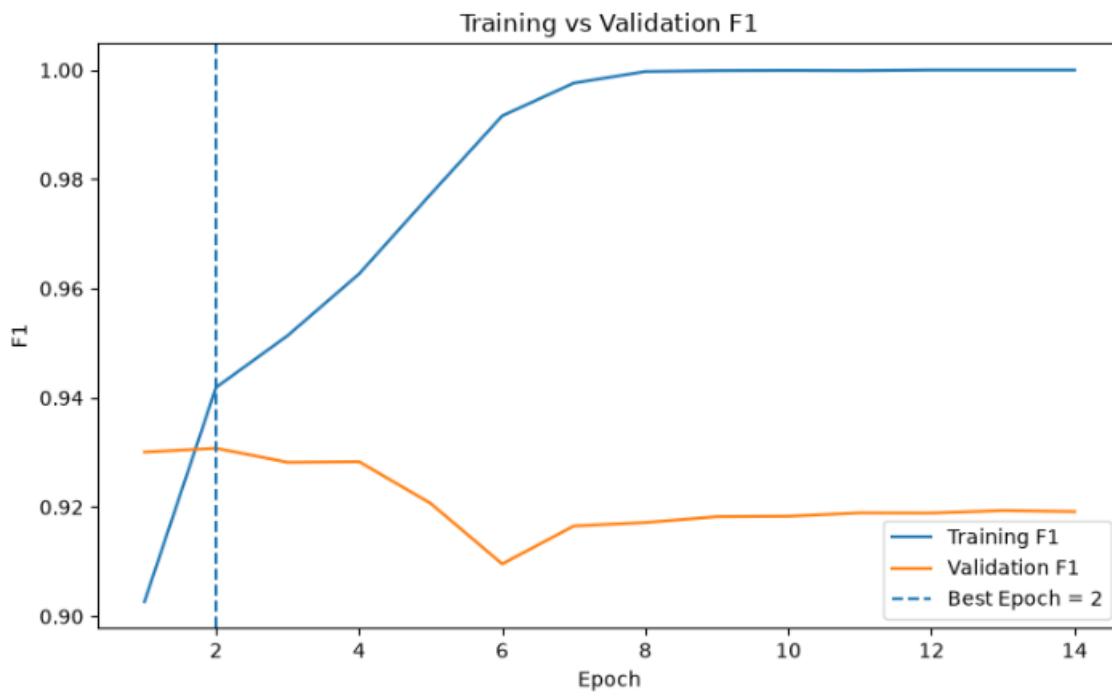
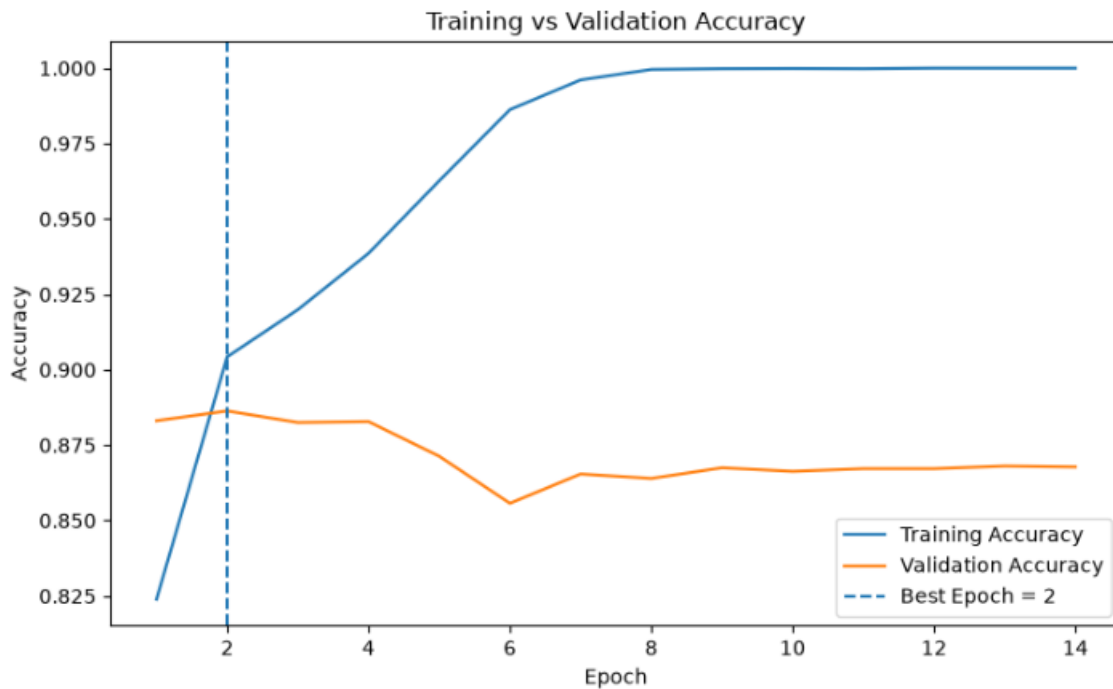
Khi viết báo cáo sau khi chạy, cần xác định best epoch, best validation loss và mô tả xu hướng của ba cặp đường cong. Nếu Training tiếp tục tốt lên trong khi Validation xấu đi hoặc dừng cải thiện, đó là dấu hiệu overfitting.

Epoch	1/80	Train Loss: 0.3980	Val Loss: 0.2795	Train Acc: 0.8239	Val Acc: 0.8831	Val F1: 0.9301
Epoch	5/80	Train Loss: 0.1087	Val Loss: 0.3811	Train Acc: 0.9626	Val Acc: 0.8713	Val F1: 0.9207
Epoch	10/80	Train Loss: 0.0017	Val Loss: 0.9493	Train Acc: 0.9999	Val Acc: 0.8663	Val F1: 0.9183

Early stopping at epoch 14

Training completed.
Best epoch : 2
Best val loss : 0.2643
DL training time : 22.6742 seconds





Giải thích: Cả ba biểu đồ đều cho thấy mô hình bắt đầu overfitting rất sớm. Sau khoảng epoch 2, Training Loss tiếp tục giảm, Training Accuracy và F1 tăng gần 100%, trong khi Validation Loss tăng và Validation Accuracy/F1 không cải thiện. Vì vậy, epoch 2 được chọn là best epoch vì cho khả năng tổng quát hóa tốt nhất trên Validation Set.

7. ĐÁNH GIÁ VÀ SO SÁNH ML VỚI DL

7.1. So sánh trên Validation Set

Sau khi có kết quả của 3 ML và MLP, notebook ghép bốn mô hình vào cùng một bảng Validation Comparison. Đây là bước quan trọng để xem MLP có thực sự cải thiện so với các baseline truyền thống trước khi retrain final model.

```
dl_val_result = evaluate_dl_classifier(
    dl_model,
    val_loader,
    criterion,
    device
)

dl_validation_row = {
    "Model": "5-Layer MLP",
    "Accuracy": dl_val_result["Accuracy"],
    "Precision": dl_val_result["Precision"],
    "Recall": dl_val_result["Recall"],
    "F1": dl_val_result["F1"],
    "ROC_AUC": dl_val_result["ROC_AUC"],
    "Training Time (s)": dl_training_time
}

validation_comparison = pd.concat(
    [
        ml_validation_df,
        pd.DataFrame([dl_validation_row])
    ],
    ignore_index=True
)

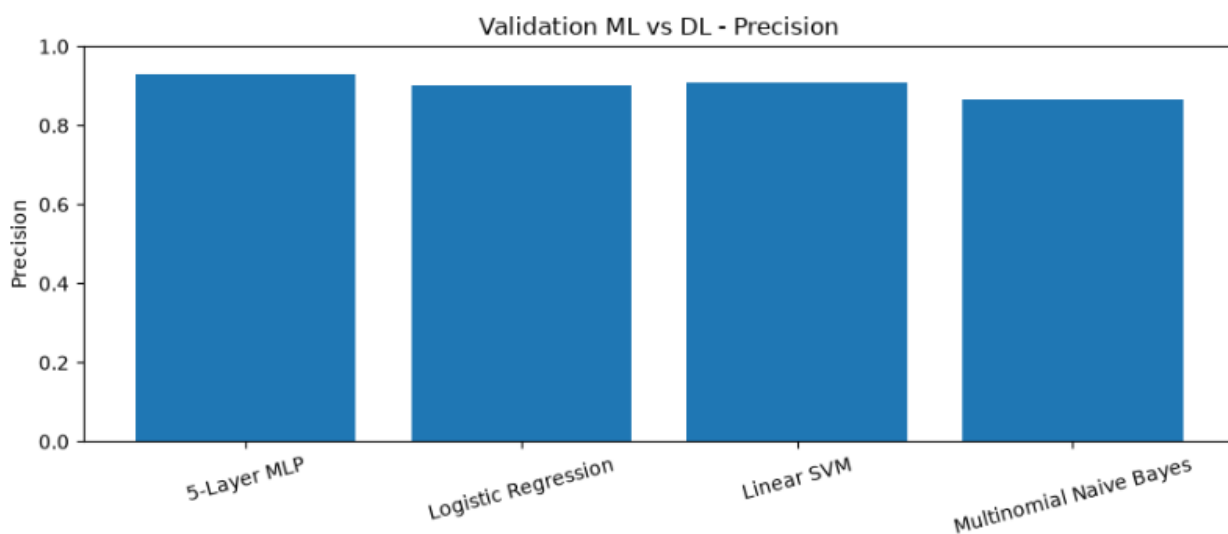
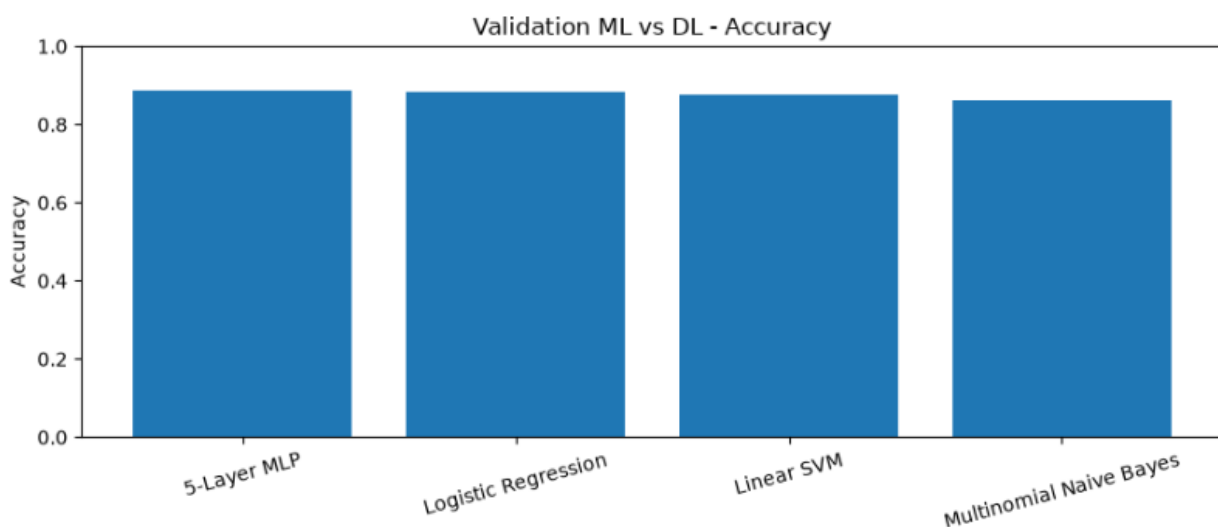
validation_comparison = (
    validation_comparison
    .sort_values(
        by=["F1", "ROC_AUC"],
        ascending=False
    )
    .reset_index(drop=True)
)

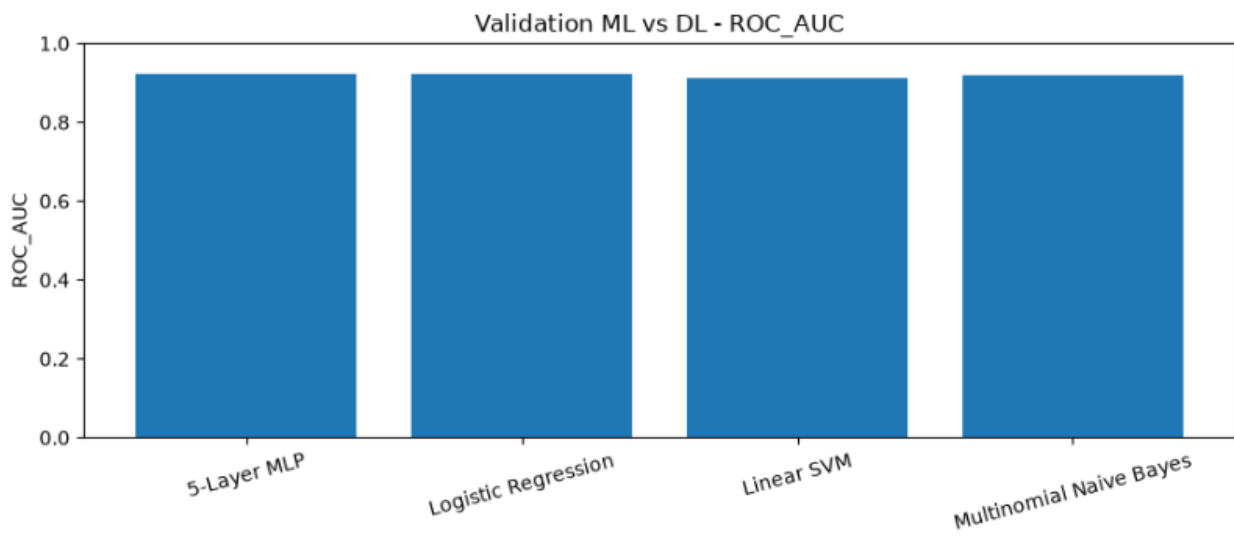
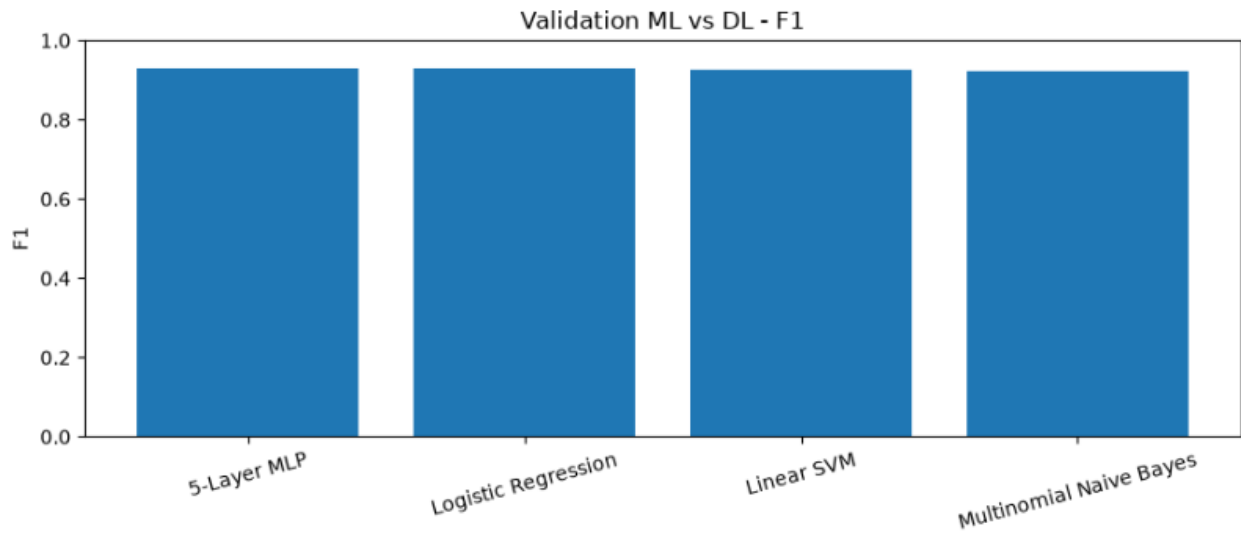
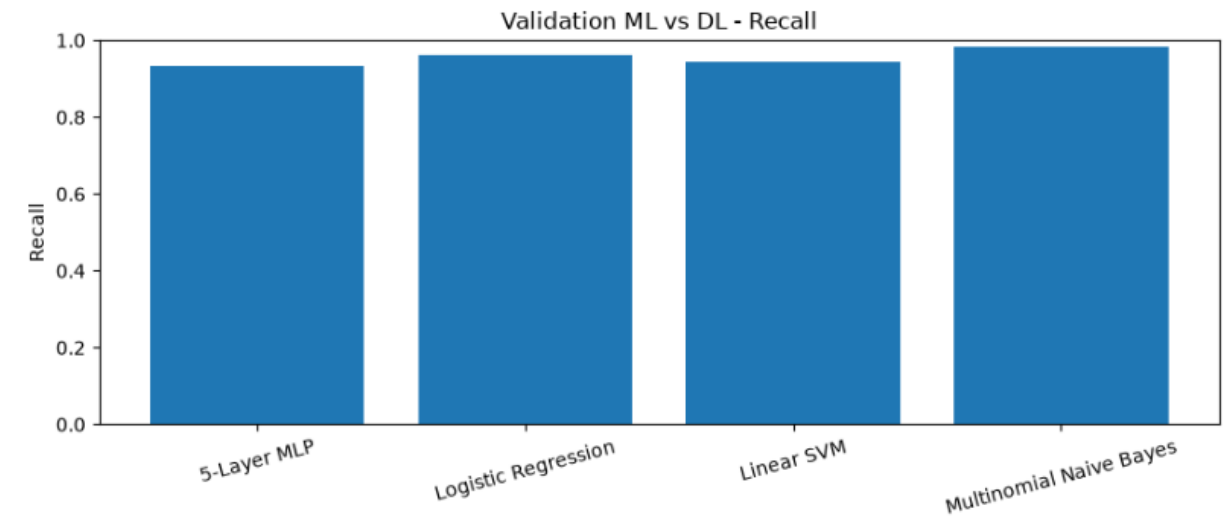
display(validation_comparison.round(4))
```

	Model	Accuracy	Precision	Recall	F1	ROC_AUC	Training Time (s)
0	5-Layer MLP	0.8863	0.9284	0.9331	0.9308	0.9226	22.6742
1	Logistic Regression	0.8816	0.9006	0.9615	0.9301	0.9237	0.1898
2	Linear SVM	0.8763	0.9081	0.9446	0.9260	0.9119	0.1189
3	Multinomial Naive Bayes	0.8622	0.8663	0.9835	0.9212	0.9176	0.0094

Hình 39. Bảng so sánh Validation Comparison của 4 model

Đánh giá: Kết quả cho thấy **5-Layer MLP** đạt **Accuracy cao nhất (0,8863)**, **Precision cao nhất (0,9284)** và **F1-score cao nhất (0,9308)**, cho thấy hiệu năng tổng thể rất tốt. Tuy nhiên, **Logistic Regression** đạt **ROC-AUC cao nhất (0,9237)** và F1 gần như tương đương MLP, trong khi thời gian huấn luyện chỉ **0,1898 giây**, thấp hơn rất nhiều so với MLP (**22,6742 giây**). **Multinomial Naive Bayes** đạt **Recall cao nhất (0,9835)** và có thời gian train nhanh nhất, nhưng Accuracy và F1 thấp hơn. Linear SVM có kết quả khá cân bằng nhưng không vượt trội ở metric nào.





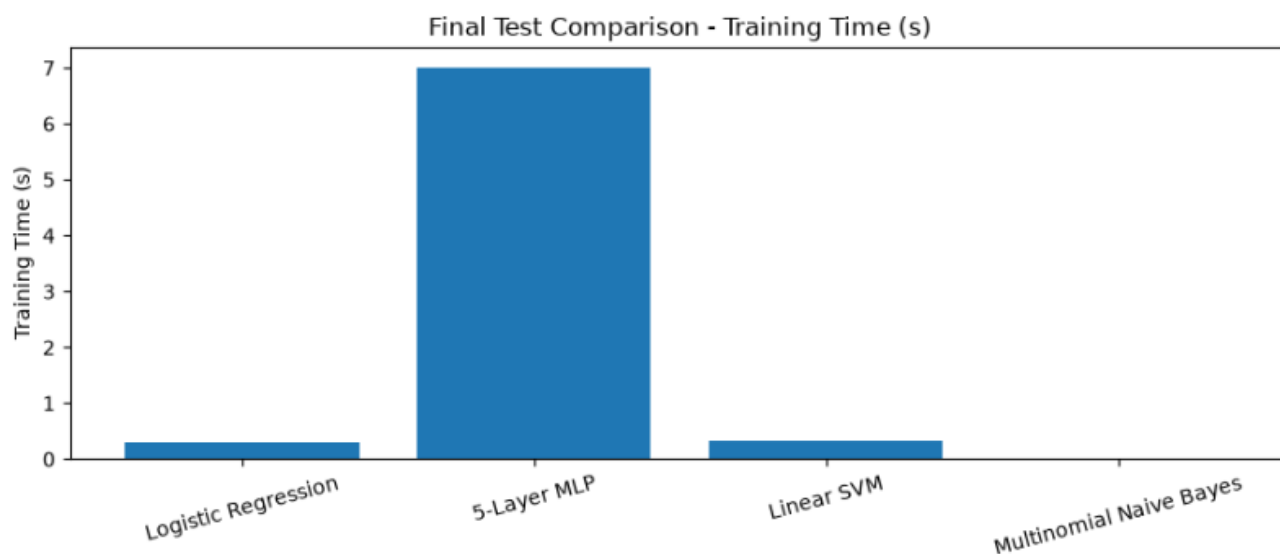
Hình 40. Biểu đồ Validation ML với DL cho 5 metris

7.2. Final Retraining và Test Metrics

Sau khi cấu hình model và best epoch đã được chốt, notebook ghép Train + Validation, fit lại TF-IDF, retrain bốn model và chỉ đánh giá một lần trên Test Set. Quy trình này giúp giảm nguy cơ dùng Test Set để điều chỉnh mô hình.

Bảng dưới đây nên được điền trực tiếp từ output final_results_df sau khi bạn chạy notebook.

	Model	Accuracy	Precision	Recall	F1	ROC_AUC	Training Time (s)
0	Logistic Regression	0.8860	0.9019	0.9658	0.9328	0.9202	0.2933
1	5-Layer MLP	0.8855	0.9138	0.9497	0.9314	0.9156	7.0114
2	Linear SVM	0.8828	0.9099	0.9511	0.9300	0.9097	0.3354
3	Multinomial Naive Bayes	0.8634	0.8667	0.9845	0.9219	0.9115	0.0197



Hình 41. Biểu đồ so sánh Training Time của 4 model

7.3. Classification Report và Confusion Matrix

Classification Report cung cấp Precision, Recall và F1 cho từng class. Confusion Matrix cho phép quan sát trực tiếp TN, FP, FN và TP. Trong ngữ cảnh recommendation intention, FP là review thực tế Not Recommended nhưng model dự đoán Recommended; FN là review thực tế Recommended nhưng bị dự đoán Not Recommended.

Ý nghĩa kinh doanh của FP/FN phụ thuộc cách sử dụng output. Ví dụ, nếu model dùng để ưu tiên các review tích cực cho chiến dịch recommendation thì FP có thể làm lãng phí nguồn lực; nếu dùng để phát hiện tín hiệu tích cực thì FN làm bỏ sót cơ hội.

```
for model_name, data in final_models.items():

    print("=" * 70)
    print(model_name)

    print(
        classification_report(
            y_test,
            data["prediction"],
            target_names=[
                "Not Recommended",
                "Recommended"
            ],
            digits=4,
            zero_division=0
        )
    )
```

Logistic Regression

	precision	recall	f1-score	support
Not Recommended	0.7727	0.5252	0.6254	615
Recommended	0.9019	0.9658	0.9328	2781
accuracy			0.8860	3396
macro avg	0.8373	0.7455	0.7791	3396
weighted avg	0.8785	0.8860	0.8771	3396

Multinomial Naive Bayes

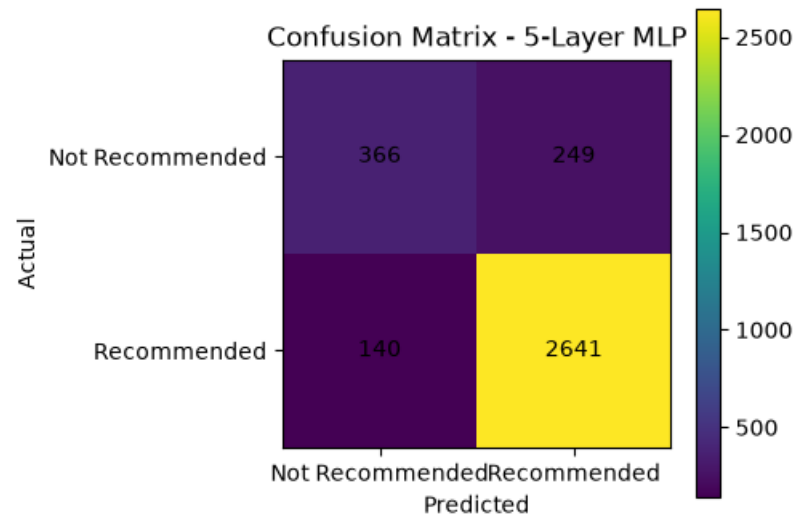
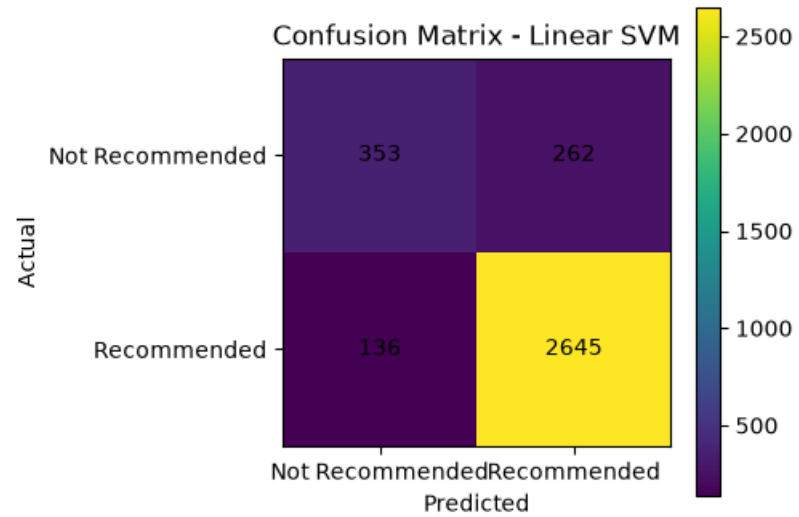
	precision	recall	f1-score	support
Not Recommended	0.8186	0.3154	0.4554	615
Recommended	0.8667	0.9845	0.9219	2781
accuracy			0.8634	3396
macro avg	0.8426	0.6500	0.6886	3396
weighted avg	0.8580	0.8634	0.8374	3396

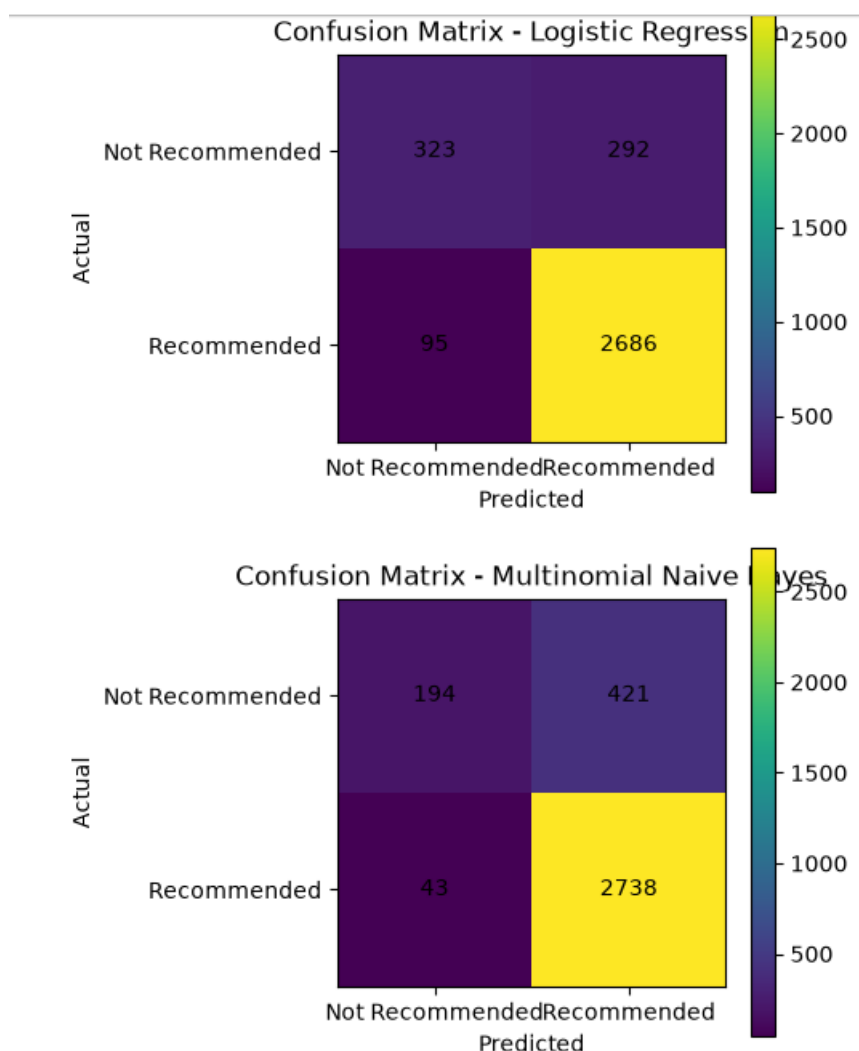
Linear SVM

	precision	recall	f1-score	support
Not Recommended	0.7219	0.5740	0.6395	615
Recommended	0.9099	0.9511	0.9300	2781
accuracy			0.8828	3396
macro avg	0.8159	0.7625	0.7848	3396
weighted avg	0.8758	0.8828	0.8774	3396

5-Layer MLP

	precision	recall	f1-score	support
Not Recommended	0.7233	0.5951	0.6530	615
Recommended	0.9138	0.9497	0.9314	2781
accuracy			0.8855	3396
macro avg	0.8186	0.7724	0.7922	3396
weighted avg	0.8793	0.8855	0.8810	3396

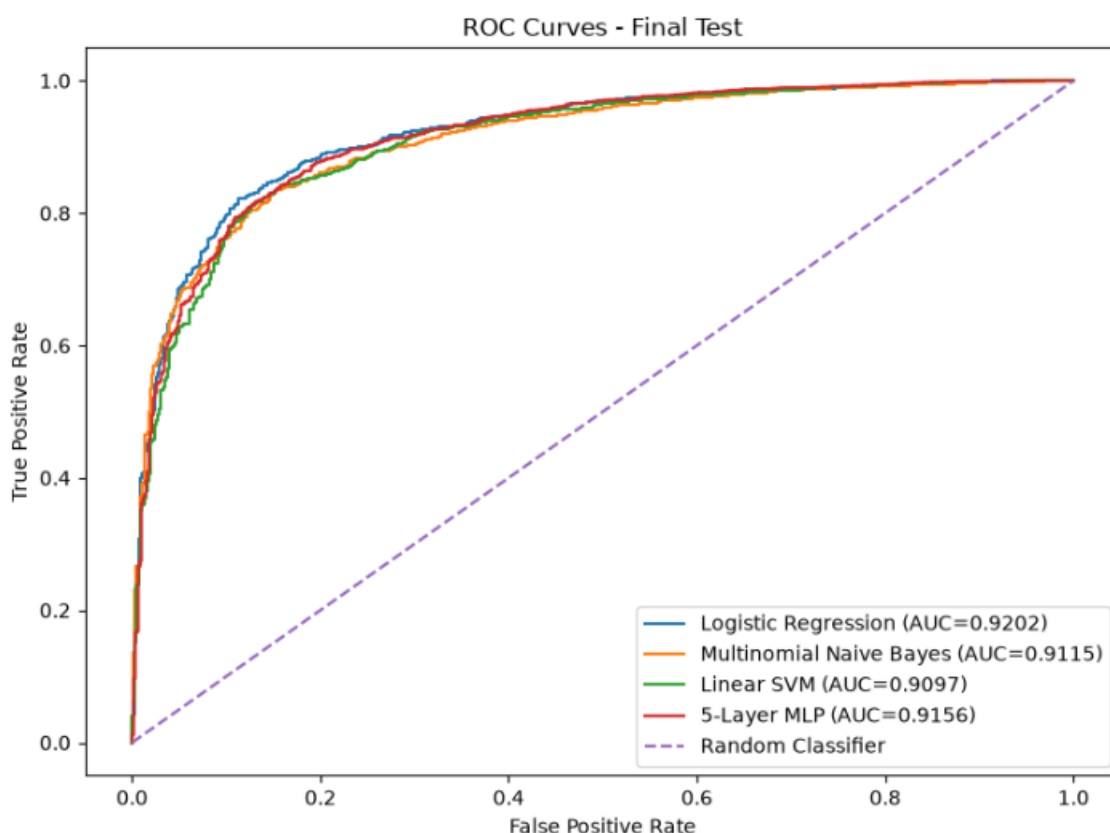




Giải thích: Các Confusion Matrix cho thấy 5-Layer MLP và Linear SVM có khả năng phân biệt hai lớp cân bằng hơn, đặc biệt MLP nhận diện đúng nhiều mẫu Not Recommended nhất. Logistic Regression và đặc biệt Multinomial Naive Bayes có xu hướng ưu tiên dự đoán lớp Recommended, giúp giảm số trường hợp bỏ sót Recommended nhưng làm tăng số mẫu Not Recommended bị phân loại nhầm. Điều này cũng phản ánh sự mất cân bằng của dataset, trong đó số mẫu Recommended lớn hơn đáng kể so với Not Recommended.

7.4. ROC Curve và ROC-AUC

ROC Curve thể hiện trade-off giữa True Positive Rate và False Positive Rate theo nhiều threshold. ROC-AUC càng gần 1 thì khả năng phân biệt hai lớp càng tốt.



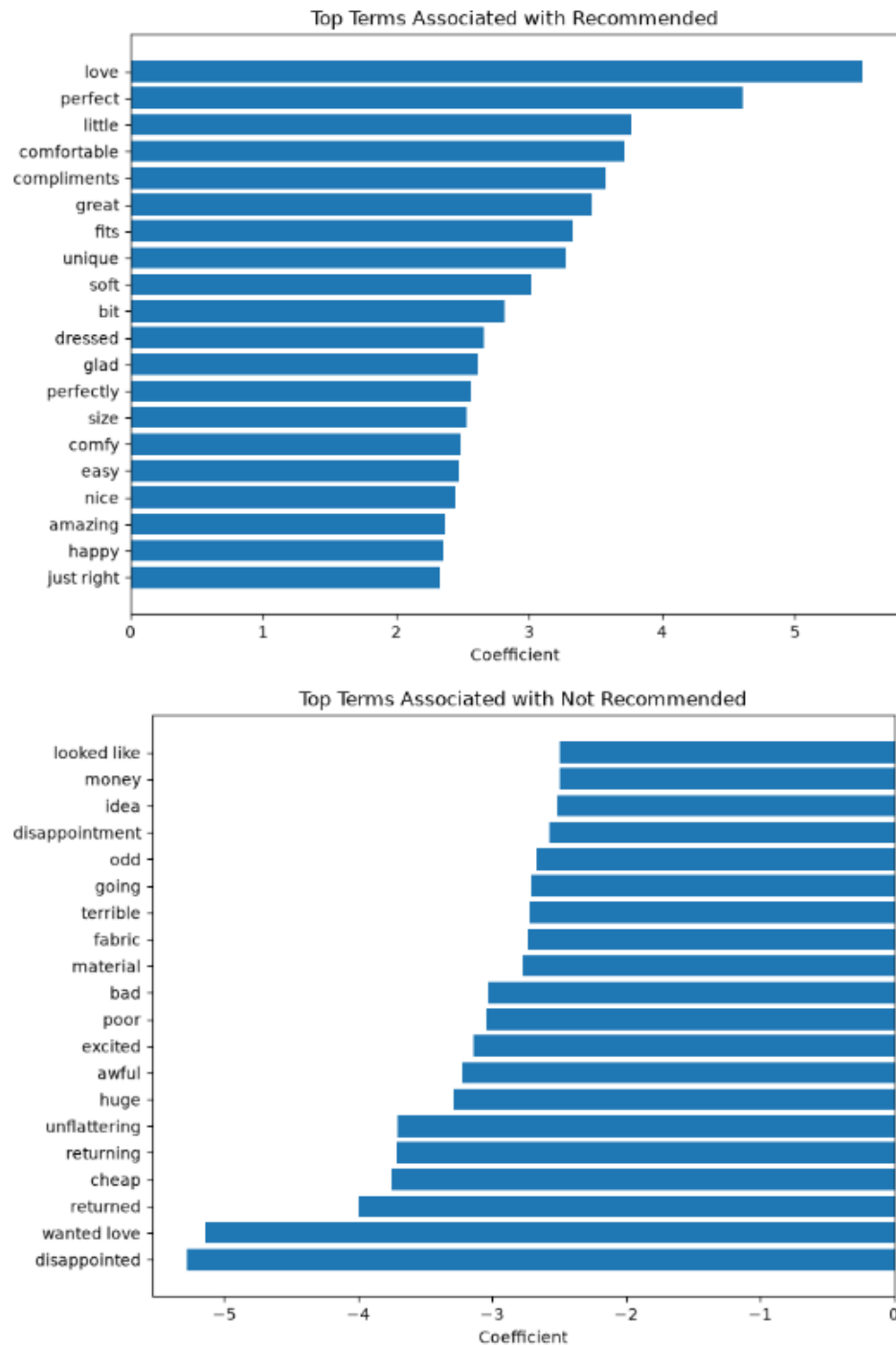
Giải thích: Biểu đồ ROC cho thấy cả bốn mô hình đều có khả năng phân biệt tốt giữa Recommended và Not Recommended, với ROC-AUC đều lớn hơn 0,90. Logistic Regression đạt AUC cao nhất là 0,9202, tiếp theo là 5-Layer MLP với 0,9156. Kết quả cho thấy Deep Learning không vượt trội rõ rệt so với các mô hình Machine Learning truyền thống trong tiêu chí ROC-AUC.

8. KHẢ NĂNG DIỄN GIẢI VÀ REPRESENTATION LEARNING

8.1. Diễn giải TF-IDF terms bằng Logistic Regression

Logistic Regression cung cấp coefficient cho từng TF-IDF feature. Coefficient dương lớn gợi ý term có xu hướng liên quan đến Recommended = 1; coefficient âm mạnh gợi ý term nghiêng về Not Recommended = 0.

Phân tích này giúp giải thích prepared text representation và cho thấy vì sao Logistic Regression có lợi thế về interpretability. Tuy nhiên coefficient phản ánh quan hệ dự đoán trong dữ liệu, không phải quan hệ nhân quả.



Hình 42. Các TF-IDF terms có ảnh hưởng mạnh nhất đến dự đoán Recommended và Not Recommended của Logistic Regression.

Giải thích: Biểu đồ cho thấy Logistic Regression đã học được mối liên hệ giữa nội dung review và hành vi recommendation. Các từ mang ý nghĩa tích cực như *love*, *perfect*, *comfortable*, *great* có hệ số dương lớn và liên quan mạnh đến nhóm Recommended. Ngược lại, các từ thể hiện sự không hài lòng như *disappointed*, *returned*, *cheap*, *unflattering*, *awful* có hệ số âm lớn và liên quan đến nhóm Not Recommended. Kết quả

cho thấy TF-IDF kết hợp với Logistic Regression có khả năng phản ánh khá rõ cảm xúc và ý định giới thiệu sản phẩm trong nội dung review.

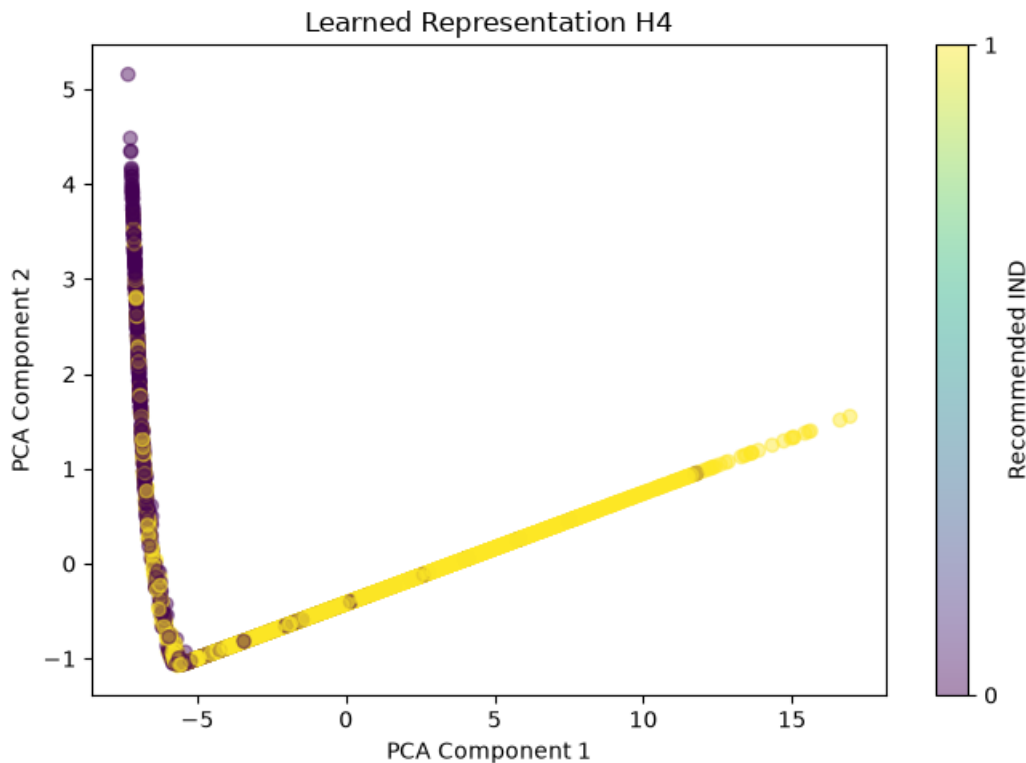
8.2. Learned Representation H4

Điểm cốt lõi của Assignment 3 là neural network học representation chứ không chỉ prediction. MLP nhận TF-IDF vector và biến đổi tuần tự qua H1, H2, H3, H4 trước khi sinh output.

$$\mathbf{x} \rightarrow \mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \mathbf{h}_3 \rightarrow \mathbf{h}_4 \rightarrow \hat{\mathbf{y}}$$

Layer H4 có 32 dimensions. Notebook dùng PCA để chiếu H4 xuống 2 chiều nhằm trực quan hóa cấu trúc learned representation. PCA chỉ phục vụ visualization; nó không phải là một phần của model khi dự đoán.

Sau khi chạy, hãy nhận xét cẩn thận hình H4: có thể nói các điểm được tái tổ chức theo một cấu trúc phục vụ nhiệm vụ phân loại nếu hình thể hiện điều đó, nhưng không nên kết luận hai lớp 'tách hoàn hảo' nếu biểu đồ vẫn chồng lấn.



Hình 43. PCA của learned representation H4 của mô hình 5-Layer MLP.

Giải thích: Biểu đồ PCA của learned representation H4 cho thấy mạng MLP đã tổ chức lại dữ liệu thành một không gian có cấu trúc phù hợp hơn với bài toán phân loại. Hai lớp Recommended và Not Recommended có xu hướng phân tách tương đối rõ, mặc dù vẫn

còn một số vùng chồng lấn. Điều này cho thấy các hidden layer đã học được những representation hữu ích cho việc dự đoán recommendation intention.

9. KẾT LUẬN

Hệ thống sử dụng bộ dữ liệu **Women's E-Commerce Clothing Reviews** để dự đoán **recommendation intention** của khách hàng dựa trên nội dung Review Text. Trong thí nghiệm, ba mô hình Machine Learning truyền thống gồm **Logistic Regression**, **Multinomial Naive Bayes** và **Linear SVM** được so sánh với mô hình Deep Learning **5-Layer MLP**. Nội dung review được chuyển đổi sang dạng số bằng **TF-IDF**, sau đó mô hình MLP tiếp tục biến đổi representation này qua các hidden layer H_1, H_2, H_3, H_4 trước khi đưa ra dự đoán cuối cùng.

Trên tập Test, **5-Layer MLP** đạt **F1-score** cao nhất là **0,9308**, đồng thời đạt **Accuracy = 0,8863**, **Precision = 0,9284**, **Recall = 0,9331** và **ROC-AUC = 0,9226**. Trong khi đó, **Logistic Regression** đạt **ROC-AUC** cao nhất là **0,9237** và có **F1-score = 0,9301**, gần như tương đương với MLP. Multinomial Naive Bayes đạt Recall cao nhất là 0,9835 nhưng Accuracy và F1 thấp hơn, còn Linear SVM cho kết quả tương đối ổn định trên các chỉ số đánh giá.

Kết quả cho thấy mô hình Deep Learning 5-Layer MLP có cải thiện nhẹ về **Accuracy**, **Precision** và **F1-score**, nhưng mức cải thiện so với Logistic Regression không lớn. Đặc biệt, thời gian huấn luyện của MLP khoảng **22,67 giây**, cao hơn rất nhiều so với Logistic Regression chỉ khoảng **0,19 giây**. Điều này cho thấy Deep Learning không tự động vượt trội hoàn toàn so với các mô hình Machine Learning truyền thống trong bài toán này. Vì vậy, việc lựa chọn mô hình cuối cùng cần cân nhắc đồng thời giữa **hiệu năng dự đoán**, **chi phí huấn luyện**, **khả năng diễn giải** và **mục tiêu nghiệp vụ**, thay vì chỉ dựa trên một chỉ số như Accuracy.